FINAL YEAR PROJECT INTERIM REPORT

FYP02-DS-T2530-0022

A Dashboard for Evaluating Federated Learning in Book Recommender Systems

1211104426
SOULTAN TOURE SALIFOU

BACHELOR OF COMPUTER SCIENCE B.CS (HONS) DATA SCIENCE

February 2026

FYP02-DS-T2530-0022
A Dashboard for Evaluating Federated Learning in Book Recommender Systems

BY

Soultan Toure Salifou

PROJECT INTERIM REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE

REQUIREMENT FOR THE DEGREE OF

Computer Science

in the

Faculty of Computing and Informatics

MULTIMEDIA UNIVERSITY
MALAYSIA

February 2026

Copyright

2026 Universiti Telekom Sdn. Bhd. ALL RIGHTS RESERVED.

Copyright of this report belongs to Universiti Telekom Sdn. Bhd. as qualified by Regulation 7.2 (c) of the Multimedia University Intellectual Property and Commercialisation Policy. No part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Universiti Telekom Sdn. Bhd. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

Declaration

I hereby declare that the work has been done by myself and no portion of the work contained in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institution of learning.

Name of candidate: Soultan Toure Salifou

Faculty of Computing & Informatics

Multimedia University

Date: 8/02/26

Acknowledgements

I would like to extend my deepest gratitude to my supervisor, Mr. Sharaf El-Deen Sami Mohammed Al-Horani, for his unwavering guidance, encouragement, and support throughout the course of this work. His expertise, patience, and dedication have been instrumental in shaping the direction and quality of this project. I am truly grateful for the opportunity to learn under his supervision and for the valuable insights he generously shared along the way.

My sincere appreciation also goes to Multimedia University and the Faculty of Computing and Informatics for providing the resources, opportunities, and a conducive learning environment that enabled me to grow academically and successfully complete this work. The support and infrastructure offered by the university have been fundamental to my development and the realization of this project.

Abstract

Privacy regulations such as GDPR and CCPA increasingly constrain how recommender systems collect and process user data, yet the accuracy cost of privacy-preserving alternatives remains poorly characterized. This project presents both a rigorous empirical study and a production-ready implementation quantifying the privacy-accuracy trade-off in horizontal federated recommendation systems. Using controlled experiments on the Goodreads book dataset with identical model architectures and hyperparameters across 18 experimental runs, we establish that standard Federated Averaging incurs a modest 1.35% RMSE privacy premium ($p < 0.001$) compared to centralized matrix factorization—statistically significant but practically acceptable for most content discovery applications. Scalability analysis across 2–20 client configurations reveal predictable linear degradation (0.016% RMSE per additional client, $R^2 = 0.75$) with computational parity (0.99× training time ratio).

To validate these findings and enable practical deployment, we developed FedRec Dashboard—a full-stack federated learning platform implementing clean architecture principles with FastAPI, PyTorch Lightning, and the Flower framework. The system features asynchronous database persistence, real-time metrics tracking, comprehensive experiment orchestration, and RESTful APIs for integration with production environments. The architecture strictly separates domain logic from infrastructure concerns, employing repository patterns, dependency injection, and domain-driven state machines to ensure testability and maintainability. The implementation supports both centralized and federated training modes with configurable hyperparameters, automatic convergence detection, and detailed per-epoch metrics visualization.

These findings demonstrate that privacy-preserving collaborative filtering is achievable without prohibitive costs, enabling practitioners to make informed architectural decisions when regulatory constraints require alternatives to centralized data collection. The open-source dashboard provides a reference implementation for organizations seeking to deploy federated recommendation systems while maintaining the user experience quality of traditional centralized approaches.

Table of Contents

Copyright iii
Declaration iv
Acknowledgements v
Abstract vi
Table of Contents vii
List of Tables xi
List of Figures xii
List of Appendices xiii
1 Chapter 1: Introduction 1
1.1 Overview 1
1.2 Problem Statement 1
1.3 Project Aims and Objectives 3
1.4 Hypotheses and Research Questions 4
1.5 Project Boundaries and Focus 5
1.6 Project Deliverables 5
1.6.1 Research Publication 5
1.6.2 FedRec Dashboard Application 6
1.7 Chapter Summary 7
2 Chapter 2: Literature Review 8
2.1 Introduction 8
2.2 The Rise of Digital Book Platforms and need for Recommender Systems 8
2.2.1 Emergence of Book-Related Platforms 9
2.2.2 The Central Role of Recommender Systems Across Book Digital Platforms 10
2.3 Fundamentals of Recommender Systems 12
2.3.1 Core Recommendation Approaches 12
2.3.2 Key Implementation Techniques 15
2.3.3 Evaluating Recommendation Performance 16
2.4 Privacy Challenges in Centralized Recommender Systems 18
2.4.1 Data Centralization Risks and Threat Types 18
2.4.2 User Concerns and the Regulatory Landscape 19
2.4.3 Introduction to Privacy-Preserving Recommender Systems (PRecSys) 20
2.5 Federated Learning: Principles and Privacy Advantages 21
2.5.1 Definition and Core Principles 21
2.5.2 Inherent Privacy Advantages 22
2.5.3 Federated Learning vs. Distributed Learning 22

2.5.4 Horizontal vs. Vertical Federated Learning 23
2.6 Federated Aggregation Algorithms 24
2.6.1 Federated Averaging (FedAvg) 24
2.6.2 FedProx 24
2.6.3 Other Relevant Aggregation Strategies 26
2.7 Horizontal Federated Learning Recommender Systems (HFedRec) 26
2.7.1 Taxonomy of HFedRec Approaches 26
2.7.2 Common challenges in HFedRec (Non-IID data, communication) 27
2.8 State-of-the-Art in Horizontal Federated Book Recommendations 29
2.8.1 Overview of HFedRec Research 29
2.8.2 State of Art Book Recommendation Systems (e.g., using Goodreads) 30
2.9 Synthesis and Research Gap Identification 31
2.10 Summary 32
3 Chapter 3: Requirements Elicitation and Analysis 34
3.1 Introduction 34
3.2 System Actors 34
3.3 Functional Requirements 35
3.4 Non-functional Requirements 37
3.5 Use Case Modelling 38
3.5.1 Use Case Diagram 38
3.5.2 Use Case Descriptions 40
3.6 Dynamic Modelling 44
3.6.1 Sequence Diagrams 45
3.6.2 Activity Diagrams 56
3.6.3 State Diagrams 60
3.7 Structural Modelling 67
3.7.1 Class Identification 67
3.7.2 Class Diagram 69
3.7.3 ER Diagram 72
4 Chapter 4: Design 74
4.1 Introduction 74
4.2 Overview of Software Architectures 75
4.2.1 Architecture Option 1: Client-Server Web Application (Monolithic Backend) .. 75
4.2.2 Architecture Option 2: Layered Monolithic Desktop Application 76
4.2.3 Architecture Option 3: Layered Architecture with Modular Microservices (and a Web Frontend) 77
4.3 Selected Software Architecture 79
4.4 Design Model 80

4.5 Deployment ... 83
4.6 Summary ... 85
5 Chapter 5: Implementation ... 87
5.1 Development Environment Setup ... 87
5.1.1 Tools and Platforms ... 87
5.1.2 Programming Languages and Libraries ... 88
5.1.3 Software Stack and Architecture ... 89
5.1.4 Key Configuration Files ... 91
5.2 Data Collection and Preprocessing ... 93
5.2.1 Dataset Characteristics ... 93
5.2.2 Data Preprocessing Pipeline ... 94
5.2.3 FedAvgItemsOnly Aggregation Strategy ... 95
5.3 Domain Layer Implementation ... 96
5.3.1 Experiment Entities ... 96
5.3.2 Configuration Value Objects ... 97
5.3.3 Repository Interfaces ... 98
5.4 Infrastructure Layer Implementation ... 98
5.4.1 Database Setup and Session Management ... 98
5.4.2 ORM Models ... 99
5.4.3 Repository Implementations ... 100
5.4.4 Database Migrations ... 100
5.5 Application Layer Implementation ... 101
5.5.1 Experiment Manager ... 101
5.5.2 Data Handler ... 102
5.5.3 Metrics Calculator and Logger ... 103
5.6 Model Implementation ... 103
5.6.1 Matrix Factorization Lightning Module ... 103
5.6.2 Centralized Training Implementation ... 104
5.6.3 Federated Training Implementation ... 105
5.6.4 Centralized vs. Federated Training Differences ... 106
5.7 Federated Learning Components ... 106
5.7.1 FedAvgItemsOnly Strategy ... 106
5.7.2 Client Application ... 107
5.7.3 User-Based Partitioner ... 108
5.8 API Layer Implementation ... 108
5.8.1 FastAPI Application Setup ... 108
5.8.2 Experiment Endpoints ... 109
5.8.3 Request/Response Schemas ... 109

5.8.4 Dependency Injection 110
5.9 Evaluation Methodology and Statistical Rigor 110
5.9.1 Performance Metrics 110
5.9.2 Statistical Testing Framework 111
5.9.3 Controlled Experimental Protocol 113
5.9.4 Scalability Analysis Methodology 115
5.10 Version Control and Schema Management 115
5.10.1 Git and GitHub Workflow 116
5.10.2 Database Schema Migrations with Alembic 117
5.11 Summary 120
6 Chapter 6: Results and Discussion 123
6.1 Experimental Results 123
6.1.1 Privacy-Accuracy Trade-off 123
6.1.2 Scalability Analysis 124
6.1.3 Training Efficiency 125
6.2 Discussion 126
6.2.1 Interpretation of Results 126
6.2.2 Implications for Deployment 126
6.3 System Demonstration 127
7 Chapter 7: Conclusion and Future Work 130
7.1 Conclusion 130
7.2 Future Work 131
References xiii
Appendix A: Gantt Chart xv
Appendix B: Meeting Logs xvi
Appendix C: Turnitin Similarity Index Page li

List of Tables

Table 2.2.1: Overview of Key Digital Content Platform Types 10
Table 2.3.1Comparison between Collaborative Filtering (CF) and Content-based Filtering (CBF) 14
Table 2.5.1Comparison of Federated Learning (FL) and Distributed Learning (DL) 22
Table 2.6.1Comparison of FedAvg and FedProx Algorithms 25
Table 3.3.1 Functional Requirements for the System Actor 35
Table 3.3.2 Functional Requirements for the User Actor 37
Table 3.4.1 Key NFRs for this project. 37
Table 3.5.1 UC-01 Load and Preprocess Dataset. 40
Table 3.5.2 UC-02 Train Centralized Model 40
Table 3.5.3 UC-03 Configure Training Parameters 41
Table 3.5.4 UC-04: Partition Dataset per Client 41
Table 3.5.5 UC-05: Run Federated Learning Simulation 42
Table 3.5.6 UC-06: Store Performance Metrics 42
Table 3.5.7 UC-07: View Performance Visualizations 43
Table 3.5.8 UC-08 Compare Centralized vs Federated Models 43
Table 3.5.9 UC-09 Export Results 44
Table 5.1.1 Development Tools and Platforms 87
Table 5.1.2 Programming Languages and Technology Stack 88
Table 5.1.3 Software Architecture Layers 89
Table 5.1.4 Configuration Files and Their Purposes 91
Table 5.2.1 Goodreads Poetry Dataset Characteristics 93
Table 5.2.2 Data Cleaning and Preprocessing Steps 94
Table 5.6.1 Centralized vs. Federated Training Implementation 106
Table 5.9.1 Evaluation Metrics and Formulations 110
Table 5.9.2 Statistical Testing Hierarchy 112
Table 5.9.3 Experimental Control Variables 114
Table 5.10.1 Git Version Control Configuration 116
Table 5.10.2 Alembic Migration Management 117
Table 5.11.1 Implementation Summary 120
Table 6.1.1 Privacy Premium Analysis Across Client Configurations 123
Table 6.1.2 Computational Efficiency Analysis 125

List of Figures

Figure 2.7.1 Proposed HFedRec taxonomy. 27
Figure 3.5.1 Use Case Diagram for the Federated Learning Dashboard 39
Figure 3.6.1 Sequence Diagram for Load and Preprocess Dataset 45
Figure 3.6.2 Sequence Diagram for Train Centralized Model 46
Figure 3.6.3 Sequence Diagram for Configure Parameters 48
Figure 3.6.4 Sequence Diagram for Partition Dataset per Client 49
Figure 3.6.5 Sequence Diagram for Run Federated Learning Simulation 51
Figure 3.6.6 Sequence Diagram for Store Performance Metrics 52
Figure 3.6.7 Sequence Diagram for View Performance Visualizations 53
Figure 3.6.8 Sequence Diagram for Compare Centralized vs Federated Models 54
Figure 3.6.9 Sequence Diagram for Export Results 55
Figure 3.6.10 Activity Diagram for UC-01: Load and preprocess Dataset 56
Figure 3.6.11 Activity diagram for UC-02: Train Centralized Model 57
Figure 3.6.12 Activity Diagram for UC-03: Configure Training Parameters 57
Figure 3.6.13 Activity Diagram for UC-05 Run Federated Learning Simulation 58
Figure 3.6.14 Activity Diagram for UC-08: Compare Centralized vs Federated Models 59
Figure 3.6.15 Activity Diagram for UC-09: Export Results 59
Figure 3.6.16 State Diagram for Centralized Training Process 63
Figure 3.6.17 State Diagram for a Client within one FL Round 65
Figure 3.6.18 State Diagram for Dashboard View 66
Figure 3.7.1 Class Diagram 71
Figure 3.7.2 System Entity-Relationship Diagram 72
Figure 4.2.1 Architecture Design Option 1 75
Figure 4.2.2 Architecture Design option 2 76
Figure 4.2.3 Architecture Design option 3 78
Figure 4.3.1 Architecture Design option 3 80
Figure 5.4.4.4.1 Refined Class Diagram with packages and Design pattern 82
Figure 6.1.1 Performance comparison between centralized and federated learning across client configurations 124
Figure 6.1.2 Box plot comparison of test RMSE distributions over 18 runs] 124
Figure 6.1.3 Training dynamics comparison between centralized and federated learning . 125
Figure 6.1.4 Scalability analysis showing individual run results with mean lines 125
Figure 6.3.1 Dashboard Overview and Experiment Management 127
Figure 6.3.2 Dataset Statistics and Preprocessing Validation 128
Figure 6.3.3 Real-time Dataset Preprocessing Progress 128

List of Appendices

Appendix A: Gantt Chart xv
Appendix B: Meeting Logs xvi
Appendix C: Turnitin Similarity Index Page li

Chapter 1: Introduction

1.1 Overview

As readers increasingly turn to digital platforms like Goodreads for discovering and engaging with books, effective and trustworthy recommendation systems become crucial. In the current data-driven landscape, recommender systems are crucial for personalizing user experiences across various platforms, yet traditional centralized methods often compromise user privacy. Federated learning (FL) emerges as a key approach to mitigate these privacy risks by enabling decentralized model training without sharing raw user data. This project investigates using Federated Learning (FL) to develop a privacy-preserving book recommender system with the Goodreads dataset. A crucial deliverable is an interactive dashboard to evaluate the FL approach's effectiveness. This dashboard will visualize performance metrics like accuracy and scalability as the number of clients increases. Ultimately, this research aims to clarify FL's feasibility and trade-offs for secure recommendations, specifically within book systems, using the developed dashboard to visualize and analyze the key findings.

1.2 Problem Statement

Centralized book recommendation systems, while widely adopted, present significant challenges concerning user privacy and system scalability. Traditional approaches necessitate the collection and storage of vast amounts of sensitive user data, including reading history, preferences, and demographic information, on central servers. This practice raises serious privacy concerns as users become increasingly aware of how their data is being utilized and the potential for misuse (Asad et al., 2023). High-profile privacy disclosure events in recent years have further amplified these dangers; for example, millions of LinkedIn accounts were reportedly sold publicly on the black market, and Google+ ultimately had to shut down services partly due to potential privacy risks involving user data. This severe situation, demonstrating the vulnerabilities of large, centralized datasets, has spurred international action, leading to the promotion of stringent data protection laws and regulations such as the GDPR in Europe, CCPA in California, and China's Personal Information Protection Law (Wang et al., 2024). Consequently, the very consolidation of extensive user datasets required by traditional systems creates a tempting target for cyberattacks and data breaches, exposing users to the risk of identity theft and other harms.

Beyond privacy, centralized systems face inherent limitations in scalability and efficiency as the user base expands. The computational demands of processing and analyzing large-scale datasets for generating personalized recommendations can lead to performance bottlenecks, increased latency, and higher infrastructure costs. This challenge is particularly pronounced in rapidly growing platforms where the volume of data and user interactions continuously increases.

In the specific domain of book recommendation, the scale of user interaction on platforms such as Goodreads presents a significant challenge. These platforms generate vast and highly sparse datasets of user ratings, which are essential for training effective collaborative filtering models like Matrix Factorization. However, the traditional centralized approach requires aggregating this sensitive data, creating a direct conflict with modern privacy principles and regulations, thus necessitating the exploration of decentralized alternatives that can maintain recommendation quality without compromising user data.

To address these gaps, this project investigates the potential of Federated Learning (FL) as a decentralized alternative for book recommendation. FL enables collaborative model training such as matrix factorization using frameworks like Flower and algorithms like FedAvg across multiple clients without centralizing their raw data. By keeping user data localized and sharing only model updates, FL offers a promising avenue to mitigate privacy risks in book recommendation systems. Moreover, its distributed nature holds potential for improved scalability.

Therefore, this research focuses on implementing and evaluating a FL-based book recommender system using the Goodreads dataset. The core objectives are to develop a functional federated system, analyze its performance concerning recommendation accuracy, scalability under an increasing number of clients, and the impact of privacy-enhancing techniques. A key component of this project includes the development of an interactive dashboard, which serves as a tool to visualize these performance metrics and facilitate a clearer understanding of the inherent trade-offs. Ultimately, this work aims to provide practical insights into the effectiveness, challenges, and applicability of Federated Learning for creating more robust, secure, and scalable book recommendation solutions.

1.3 Project Aims and Objectives

To achieve the project's aim of evaluating the effectiveness, scalability, and privacy trade-offs of Federated Learning in book recommender systems, the following SMART objectives have been formulated:

1. **Develop a Federated Book Recommender System:** Implement a federated book recommender system using Matrix Factorization on the Goodreads dataset, leveraging the FedAvg algorithm for model aggregation.

2. **Compare Centralized vs. Federated Models:** Conduct a direct, side-by-side comparison of the performance metrics (accuracy and training time) between the federated recommendation approach and the traditional centralized training baseline.

3. **Create an Interactive Evaluation Dashboard:** Design and build a comprehensive, interactive dashboard to visualize key performance metrics, simulation outcomes, and comparative analyses of both centralized and federated approaches.

4. **Analyze Federated Learning Performance:** Evaluate the accuracy (RMSE), scalability (client count vs. runtime), and training efficiency of the federated system, and crucially, analyze the privacy-accuracy and performance trade-offs by systematically varying the number of participating clients and communication rounds..

5. **Enhance Transparency and Accessibility:** Ensure the complex dynamics of federated learning performance are understandable to a broad audience through intuitive dashboard visualizations, facilitating clear insights into the system's behavior and trade-offs.

1.4 Hypotheses and Research Questions

Building upon the problem statement and project objectives, this study transitions from foundational discussions to empirical investigation. The primary goal is to present tangible evidence regarding the real-world trade-offs involved in applying Horizontal Federated Learning (HFL) to recommendation systems.

At the heart of this investigation lies a fundamental question that drives the need for privacy-preserving recommendation technologies: Can users get better recommendations and contribute to a global recommendation engine whilst keeping their data local?
This question encapsulates the core tension between personalization quality and privacy protection that modern recommendation systems must navigate.

To establish a concrete and testable foundation for addressing this question, the project introduces the following primary hypothesis: *A Horizontal Federated Recommender System (HFedRec), utilizing Matrix Factorization and the Federated Averaging (FedAvg) algorithm, can achieve recommendation accuracy comparable to that of a traditional centralized model.*
If validated, this would position HFedRec as a viable, privacy-preserving alternative for book recommendation platforms and directly answers our central research question.

This central hypothesis is evaluated through two specific, measurable claims. The first claim assesses accuracy comparability, proposing that the HFedRec model will achieve recommendation performance—measured using Root Mean Square Error (RMSE)—that is statistically similar to, or only marginally different from, a centralized Matrix Factorization model.
The second claim examines scalability and training efficiency, aiming to demonstrate the practical viability of the HFedRec system for real-world book recommendation applications by analyzing the predictable impact of varying client numbers and communication rounds on these performance metrics.

By structuring the investigation around a clearly defined hypothesis and supporting claims, the project ensures a focused and rigorous empirical evaluation. The interactive dashboard developed for this study visualizes experimental outcomes, directly supporting the validation process. This approach guarantees that the conclusions drawn are grounded in data and reflect the practical considerations necessary for deploying federated recommender systems in real-world environments.

1.5 Project Boundaries and Focus

This project focuses on the implementation and evaluation of a Federated Learning (FL) approach for book recommendation systems. The project will specifically employ the Federated Averaging (FedAvg) algorithm for orchestrating the federated learning process. The core recommendation algorithm utilized will be Matrix Factorization.

The primary dataset for this undertaking is the publicly available Goodreads dataset. The implementation of the federated learning environment and algorithms will leverage the Flower framework.

System performance is evaluated based on recommendation accuracy, measured using Root Mean Square Error (RMSE), and scalability, assessed through the relationship between the number of participating clients and execution runtime.

The federated learning environment is simulated by distributing the Goodreads dataset across multiple virtual clients. Deployment on real user devices is not included within the scope of this project.

An interactive dashboard has been developed to visualize key performance metrics, including accuracy, scalability, and privacy impact, derived from the FL system evaluation. This dashboard will also facilitate a comparison with a traditional centralized baseline, serving as a tool for analysis and demonstration.

The scope of this project does not extend to exploring alternative FL aggregation algorithms beyond FedAvg. Furthermore, it will not involve the implementation of other types of recommendation models, such as deep learning models, within the federated setting. The sole focus remains on assessing the feasibility and performance characteristics of the specified Matrix Factorization-based FL approach for book recommendations within a simulated environment.

1.6 Project Deliverables

This project has produced two principal deliverables that contribute to both academic knowledge and practical implementation in the domain of federated learning for recommender systems:

1.6.1 Research Publication

A scholarly research paper titled "Privacy-Preserving Book Recommendations: A Federated Learning Approach Using Matrix Factorization" has been prepared for publication. This

paper presents a comprehensive empirical evaluation of horizontal federated learning applied to collaborative filtering in book recommendation systems. The research provides:

Empirical Evidence: Statistical analysis comparing centralized and federated approaches across multiple client configurations (2, 10, and 20 clients), demonstrating that federated learning achieves recommendation accuracy within approximately 1.35% of centralized models.

Scalability Analysis: Quantitative assessment of the relationship between client participation and model performance, revealing a modest linear trend of RMSE increase (0.000161 per additional client) that is practically negligible for real-world deployment scenarios.

Privacy-Utility Trade-off Characterization: Documentation of the marginal accuracy cost associated with privacy-preserving federated approaches, providing evidence-based insights for practitioners evaluating federated learning adoption.

Methodological Framework: Detailed description of the implementation architecture, experimental design with multiple random seeds for statistical rigor, and reproducible evaluation protocols that can guide future research in federated recommender systems.

The publication contributes to the growing body of literature on privacy-preserving machine learning by demonstrating that federated learning is a viable alternative to centralized training for recommendation systems, particularly in domains where data privacy is paramount.

1.6.2 FedRec Dashboard Application

A complete, software system has been developed to implement and evaluate federated learning for book recommendations. The system architecture encompasses:

Backend Infrastructure: A FastAPI-based REST API implementing clean architecture principles with clear separation between domain logic (core), application services, infrastructure (database persistence), and presentation layers. The backend supports asynchronous operations for efficient experiment execution and metrics collection.

Federated Learning Engine: Integration of the Flower framework for federated simulation, supporting configurable client counts, communication rounds, and hyperparameters. The engine implements the FedAvg aggregation strategy with user-based data partitioning to simulate realistic federated scenarios.

Data Persistence Layer: PostgreSQL database with SQLAlchemy ORM for storing experiment configurations, training metrics, and results. The schema supports both centralized and federated experiment types with comprehensive metric tracking across epochs and rounds.

Interactive Dashboard: A visualization platform (implementation details in subsequent chapters) that enables users to:

- Create and configure experiments with custom hyperparameters
- Monitor training progress in real-time
- Compare centralized versus federated approaches
- Analyze scalability trends and performance trade-offs
- Export results for further analysis

Together, these deliverables address both the theoretical understanding and practical implementation challenges of federated learning in recommendation systems, providing

contributions that are valuable to academia, industry practitioners, and the broader privacy-preserving machine learning community.

1.7 Chapter Summary

This chapter has established the foundation for investigating federated learning as a privacy-preserving alternative to centralized book recommendation systems. The introduction contextualized the growing importance of recommendation systems in digital reading platforms while highlighting the fundamental tension between personalization quality and user privacy protection.

The problem statement articulated the dual challenges facing centralized recommendation systems: (1) privacy vulnerabilities arising from concentrated user data storage in an era of stringent data protection regulations (GDPR, CCPA), and (2) scalability limitations as platforms grow. These challenges necessitate exploration of decentralized approaches that can maintain recommendation quality while preserving user privacy.

The project objectives were clearly defined to develop, implement, and evaluate a horizontal federated learning system for book recommendations using Matrix Factorization and the FedAvg algorithm. The central research question–"Can users get better recommendations and contribute to a global recommendation engine whilst keeping their data local?"–guides the empirical investigation.

A testable hypothesis was formulated: that federated learning can achieve recommendation accuracy comparable to centralized models. This hypothesis is operationalized through two measurable claims focusing on accuracy comparability and scalability characteristics.

The project scope was delineated to focus specifically on Matrix Factorization with FedAvg aggregation using the Goodreads dataset in a simulated federated environment. The boundaries explicitly exclude alternative aggregation algorithms, deep learning models, and real-world deployment to maintain focus on rigorous evaluation of the core federated approach and finally, the project deliverables were outlined.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews literature pertinent to federated learning for book recommenders, providing the necessary background context and identifying the research gap this project addresses. The review traces the evolution from digital content consumption and the need for recommendations, through core recommender system concepts (including Collaborative Filtering and Matrix Factorization) and their associated privacy challenges. It then introduces Federated Learning (FL), focusing on the Horizontal FL paradigm, key algorithms like FedAvg and FedProx. Finally, it surveys the state-of-the-art in applying these techniques to book recommendations before synthesizing the findings to pinpoint the specific research gap guiding this study.

2.2 The Rise of Digital Book Platforms and need for Recommender Systems

The landscape of reading and literature consumption has undergone a profound transformation over the past few decades, marked by a significant shift from predominantly physical media to diverse digital formats and platforms. This transition, driven by technological advancements and changing user behaviors, has reshaped how individuals discover, access, and engage with books (Hurkens, 2022). Key advantages fueling this transition include the enhanced portability and convenience of carrying entire libraries on a single device, potential cost-effectiveness and accessibility, and the ability for interactive content.

This abundance of digital content, however, has amplified the challenge of discovering relevant reading material. Navigating this vast digital library and helping users find content that aligns with their interests has created a critical need for effective recommender systems and personalized approaches to information management.

2.2.1 Emergence of Book-Related Platforms

Beyond the simple digitization of text, the internet has fostered the growth of specialized platforms centered around books, transforming reading into a more connected and interactive experience.

Social Reading Platforms: Prominent among these are social networking sites dedicated to readers, with Goodreads being the largest and most influential example (Hurkens, 2022). It functions as a hybrid platform, combining social networking features like friend connections, updates, and groups with book-centric activities. Core functionalities allow users to track reading progress, rate books, write reviews, and organize virtual libraries using customizable "shelves," which often act as collaborative tagging. These platforms facilitate "digital social reading," generating archives of "amateur criticism" and reader responses that offer insights into reader engagement (Rebora et al., 2021).

Content Summary Services: These address the needs of time-constrained individuals seeking efficient knowledge acquisition. Services such as Headway, Blinkist, Shortform, and Instaread have gained popularity. They provide condensed summaries of books, typically available in text and audio formats, focusing on extracting key takeaways and actionable insights within minutes.

Digital Libraries: These have evolved beyond simple digitized collections, functioning as dynamic hubs for e-books, journals, databases, and archives. They play a crucial role in preserving cultural heritage and supporting research, with many integrating technologies like AI for personalized experiences and immersive learning. They also face challenges related to the digital divide, information overload, and data privacy.

Streaming Services and Other Platforms: The digital ecosystem also encompasses a wide array of other platforms. This includes video streaming services, such as Netflix and Amazon Prime Video, offering movies, documentaries, and series, some of which are based on books. It also includes news aggregators and portals delivering current events, music streaming platforms, and general social media platforms where content of all types is shared and discussed.

The table below provides a comparative overview of these digital platform types.

Table 2.2.1: Overview of Key Digital Content Platform Types

PLATFORM TYPE	PRIMARY FUNCTION	EXAMPLE CONTENT	KEY USER NEED ADDRESSED	RELEVANCE TO BOOK DOMAIN
SOCIAL READING SITES (E.G., GOODREADS)	Community interaction, discovery, reviews	Book reviews, ratings, discussions, user profiles	Social connection, peer recommendations, discovery	Very High (Primary focus)
DIGITAL LIBRARIES(EG PROJECT GUTENBERG)	Access to curated/archived resources, research support	E-books, journals, databases, digitized archives	In-depth information access, preservation	High (E-book lending, scholarly articles)
CONTENT SUMMARY SERVICES (E.G., HEADWAY, BLINKIST)	Efficient knowledge acquisition, key takeaways	Summaries of non-fiction books (text/audio)	Time-saving, core concept understanding	Moderate (Focus on non-fiction, alternative consumption mode)
STREAMING SERVICES (E.G., NETFLIX, PRIME VIDEO)	Entertainment, passive consumption	Movies, TV shows, documentaries (some book-related)	Entertainment, leisure	Low-Moderate (Adaptations, documentaries about authors/books)
NEWS AGGREGATORS/PORTALS	Current events, information updates	Articles, reports, multimedia news	Staying informed	Low (Book reviews, author interviews possible but not primary)

2.2.2 The Central Role of Recommender Systems Across Book Digital Platforms

The proliferation and diversification of digital book platforms like Goodreads and the vast catalogs within digital libraries, as previously mentioned, present users with an overwhelming volume of choices. This information overload creates a significant challenge for content discovery, making it difficult for readers to find books aligned with their interests.

Recommender systems, often referred to as RecSys, have consequently emerged as essential tools to address this challenge, acting as intelligent filters and guides. These systems function by analyzing a wide array of user data. This includes past behavior such as clicks, views, ratings, and purchases, as well as explicit preferences and potentially contextual or demographic information. This analysis is done alongside item attributes, such as genre, author, or keywords for books, to predict user interest and suggest items they are likely to find

valuable. Specifically, within the context of book platforms, RecSys play a crucial role by surfacing books users might not otherwise encounter through manual browsing or searching, thereby potentially introducing them to new authors or genres. By tailoring suggestions to individual tastes, these systems aim to improve user satisfaction and foster engagement, which in turn can enhance users' reading culture and keep them interacting with the platform (Nisha et al., 2024). Platforms frequently optimize these systems using engagement-based metrics like click-through rates (CTR) or conversion rates, interpreting user interaction as a strong signal of preference.

However, while RecSys offer significant benefits for discovery, their underlying mechanisms and optimization strategies have generated considerable debate regarding their societal and individual impacts. A primary concern revolves around the potential creation of "filter bubbles" or "echo chambers." A filter bubble, a term popularized by Pariser, refers to a state of intellectual isolation that can result from personalized searches and recommendations where algorithms selectively guess what information a user would like to see based on information about them—such as location, past click behavior, and search history—thereby isolating them from differing viewpoints. Similarly, echo chambers describe environments where individuals are primarily exposed to beliefs and opinions that coincide with their own, potentially reinforcing existing biases and reducing exposure to diverse perspectives. Flaxman et al. (2016) explore these phenomena in the context of news consumption, noting mixed results regarding segregation and exposure to opposing views. Recent modeling, however, suggests that certain algorithms might paradoxically enhance novelty discovery compared to individual exploration (De Marzo G, Gravino P, Loreto V (2024)). This highlights that the impact of RecSys on discovery and diversity is complex and algorithm dependent.

Beyond the effects on individual discovery patterns, the deep integration of recommender systems into platforms like Goodreads signifies more than mere technological convenience. It represents the emergence of an "algorithmic culture," where algorithms mediate and shape cultural consumption. Goodreads serves as a prime example of the "algorithmic selection of cultural goods," where the visibility, promotion, and perceived value of books are significantly influenced by the platform's underlying recommendation and ranking algorithms. Critically, user activities on such platforms—including ratings, reviews, adding books to shelves, and reading progress tracked via integrations like Kindle—generate vast amounts of structured data. This process, often referred to as datafication, transforms user engagement into quantifiable data points. The resulting data becomes a valuable commodity for the platform owner, Amazon, utilized not only to refine its own recommendation algorithms but also

potentially for targeted advertising and market analysis. These dynamic leverages what some scholars term "free labor," where the opinions, categorizations, and discussions provided by users are harnessed to create economic value for the platform, often without direct compensation to the users themselves. These insights, as discussed by Hurkens (2022), underscore how such systems, while indispensable for navigating vast digital libraries, introduce intricate dynamics related to discovery, potential homogenization or polarization of tastes, user data privacy, and the subtle shaping of cultural consumption by powerful, often opaque, algorithmic forces. A clear understanding of these dynamics is crucial for contextualizing the need for more transparent, fair, and privacy-preserving approaches to recommendation.

2.3 Fundamentals of Recommender Systems

Given the central role and societal impact of recommender systems discussed previously, understanding their underlying technical fundamentals is crucial. Recommender systems (RecSys) are software tools and techniques designed to provide suggestions for items that are most likely of interest to a particular user. In the context of the overwhelming amount of information available on digital platforms, as discussed previously, they serve the critical function of filtering and prioritizing content, thereby assisting users in discovering relevant items and making decisions. To achieve this, RecSys employ various algorithmic approaches, rely on specific implementation techniques which will be elaborated upon, and are evaluated using distinct metrics.

2.3.1 Core Recommendation Approaches

Three main algorithmic paradigms dominate the field of recommender systems: Collaborative Filtering, Content-Based Filtering, and Hybrid approaches.

1. Collaborative Filtering (CF):

This approach generates recommendations based on the principle of homophily the idea that users who have agreed on their preferences in the past are likely to agree in the future (De Marzo G, Gravino P, Loreto V (2024). CF analyses patterns of user behaviour, activities, or preferences (like ratings or interaction history) across a large user group to predict what a specific user might like, without needing to understand the intrinsic content of the items themselves. Instead, it relies primarily on the user-item interaction matrix, which captures data points like ratings, clicks, or purchases. CF methods are typically categorized into:

i. Memory Based CF:

These methods directly operate on the stored user-item interaction data. They compute similarities between users (in user-based CF) or between items (in item-based CF) and then make predictions based on the aggregated preferences or ratings of similar users ('neighbours') or ratings on similar items. These can face scalability challenges with very large datasets and are known to be vulnerable to shilling attacks, where fake profiles are introduced to manipulate recommendations.

ii. Model Based CF:

These methods aim to learn underlying patterns or structures within the interaction data by building a predictive model. Instead of directly using the raw interaction matrix for predictions, they learn parameters that can predict missing ratings or user preferences. Techniques like Matrix Factorization (MF), which decomposes the user-item interaction matrix into lower-dimensional latent factor matrices representing users and items, fall under this category. Matrix Factorization is a particularly common technique explored in modern recommender systems research, including federated settings (Wang et al., 2024).

CF excels at discovering novel and serendipitous items by leveraging the collective preferences of users. However, it faces significant challenges, including the cold start problem for new users or items with insufficient interaction data (Wahab et al., 2022) and the data sparsity problem where performance degrades significantly if the user-item interaction matrix contains few entries.

2. Content Based Filtering (CBF):

In contrast to CF, CBF recommends items based on their intrinsic attributes and a profile of the user's preferences. It operates on the principle of suggesting items like those the user has liked or interacted positively with in the past. This requires creating descriptive profiles for items (e.g., using genre, author, keywords for books) and maintaining a profile for the user that summarizes their preferences based on past interactions or explicit feedback. Machine learning classifiers or similarity metrics are then employed to match item profiles with the user's profile (Nisha et al., 2024). CBF handles new items well, provided their features are available, and is less affected by data sparsity. Its recommendations are often more transparent as they are based on item features. However, CBF tends towards

overspecialization, struggling to recommend items outside a user's established profile and thus offering limited novelty.

The following table compares the above-mentioned recommendation algorithms.

Table 2.3.1Comparison between Collaborative Filtering (CF) and Content-based Filtering (CBF)

FEATURE	COLLABORATIVE FILTERING (CF)	CONTENT-BASED FILTERING (CBF)
DEFINITION	Recommends based on similar users' preferences or item co-occurrence patterns.	Recommends items like those liked previously, based on item features.
STRENGTHS	Serendipity/Novelty, no content analysis needed, Handles complex items.	Handles new items, less affected by sparsity, Transparency (explainable).
WEAKNESSES	Cold start (new users/items), Data sparsity, Scalability issues, Shilling vulnerability.	Overspecialization (limited novelty), Needs item features, new user cold start.
DATA NEEDS	User-item interaction matrix (ratings, clicks, etc.).	Item features/descriptions, User profiles (past interactions/preferences).
COLD START HANDLING	Poor for new users and new items.	Better for new items, requires user profile for new users.
SPARSITY HANDLING	Performance degrades significantly with sparse data.	Less affected by data sparsity.
NOVELTY/DIVERSITY	Can recommend diverse/unexpected items via similar users.	Tends to recommend items like past preferences; limited novelty.
SHILLING VULNERABILITY	More vulnerable to fake profiles/ratings.	Less susceptible as it relies on item content.

3. Hybrid Approaches:

These systems combine CF and CBF techniques to leverage their strengths and mitigate weaknesses. The goal is often to improve accuracy, address limitations like the cold start problem, and provide more robust and diverse suggestions. Hybridization methods vary, from weighting outputs of separate systems to building unified models incorporating both interaction and content data. While potentially offering superior performance, hybrid systems can be more complex to design, implement, and tune compared to single-paradigm systems. They navigate the fundamental tensions between leveraging collaborative wisdom versus content similarity and handling newness versus exploiting known preferences.

Understanding these core approaches and their trade-offs is fundamental to designing effective recommender systems for diverse applications, including book platforms.

2.3.2 Key Implementation Techniques

While the previous section outlined the core paradigms, implementing these approaches relies on specific algorithms and techniques. Within the Model-Based CF paradigm, one of the most influential and widely adopted techniques, particularly relevant to this project, is Matrix Factorization (MF).

MF techniques address the challenge of the often large and sparse user-item interaction matrix (e.g., a matrix R where R_{ui} is the rating user u gave to item *i*, and most entries are unknown) by aiming to uncover latent features or dimensions that explain the observed interactions. The core idea is to decompose or approximate the interaction matrix R into two or more lower-dimensional matrices, most commonly a user-factor matrix (*U*) and an item-factor matrix (*V*), such that $R \approx U \times V^{\mathrm{T}}$ (Asad et al., 2023). Each user u is thus mapped to a latent vector p_u (a row in u), and each item *i* is mapped to a latent vector q_i (a row in *V*), both residing in a joint latent factor space of dimensionality *k*. The elements of these vectors represent latent features – for items, these might implicitly capture underlying characteristics like genres or styles, while for users, they might capture their affinity for those characteristics (Wang et al., 2024). The prediction of user *u*'s rating for item *i*, denoted $\hat{r}_{ui}$, is then typically calculated as the dot product of their latent vectors $\hat{r}_{ui} = qiTpu\ or\ puTqi.$

These latent factor vectors, $p_u$and q_i, for all users and items are learned by fitting the model to the known ratings or observed interactions in the matrix R. This is framed as an optimization problem, where the goal is to minimize the aggregate error, such as squared error, between predicted ratings, $\hat{r}_{ui}$, and actual known ratings, r_{ui} . This process often incorporates regularization terms controlled by parameters like λ_u and λ_v to prevent overfitting to the training data. While related to techniques like Singular Value Decomposition (SVD) from linear algebra, MF methods in RecSys are specifically adapted to handle the high degree of missing data typical in recommendation datasets. To achieve this, they often employ optimization algorithms like Stochastic Gradient Descent (SGD), favored for its computational efficiency on large datasets and its iterative update mechanism suitable for distributed learning scenarios, or Alternating Least Squares (ALS).

MF models, such as Probabilistic Matrix Factorization (PMF, have proven effective because they can implicitly capture complex user-item relationships and underlying preferences from interaction data, often leading to better accuracy than simpler memory-based CF methods, particularly when data is sparse (Wang et.al., 2024). This ability to model latent factors makes

MF a powerful and foundational technique in recommendation. Other implementation techniques exist for different paradigms. These include similarity calculations, such as cosine similarity for Memory-Based CF, and TF-IDF vectors or various classifiers for CBF. However, MF serves as the primary foundational technique relevant to this work's focus.

2.3.3 Evaluating Recommendation Performance

Evaluating the performance of recommender systems is crucial for comparing different algorithms and understanding their effectiveness. Common evaluation metrics can be broadly categorized based on whether they assess prediction accuracy or the quality of the recommendation list.

1. Prediction Accuracy Metrics

These metrics assess how accurately a recommender system can predict the numerical ratings a user would assign to items. They are most relevant when the system outputs predicted ratings (e.g., for star-based interfaces).

1.1 Root Mean Squared Error (RMSE):

RMSE is one of the most widely used metrics for rating prediction. It measures the square root of the average squared difference between the predicted ratings $\hat{r}_{ui}$ and the actual ratings r_{ui} over all user-item pairs in the test set *T*:

$$\mathrm{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (\hat{r}_{ui} - r_{ui})^2}$$

Equation 2.1 Root Mean Squared Error (RMSE)

Lower RMSE values indicate better accuracy. Since the metric squares errors, it penalizes large deviations more heavily than smaller ones.

1.2 Mean Absolute Error (MAE):

MAE measures the average magnitude of the absolute differences between predicted ratings $\hat{r}_{ui}$ and the actual ratings r_{ui}, without considering the direction of the error:

$$\mathrm{MAE} = \frac{1}{|T|} \sum_{(u,i)\in T} |\hat{r}_{ui} - r_{ui}|$$

Equation 2.2 Mean Absolute Error (MAE)

Unlike RMSE, MAE treats all errors equally and does not disproportionately penalize larger errors. It is often easier to interpret but less sensitive to outliers.

2. Ranking Accuracy Metrics

These metrics evaluate the quality of the ranked list of recommended items, focusing on whether relevant items appear near the top of the list. This is often more important in practical scenarios where users are presented with a list rather than predicted ratings.

2.1 Precision: Measures the fraction of recommended items that are relevant to the user. It is calculated as the number of relevant recommended items, denoted as True Positives (TP), divided by the total number of recommended items, which is the sum of TP and (FP).

$$\mathrm{Precision} = \frac{TP}{TP + FP}$$

Equation 2.3 Precision Formula

Precision reflects how accurate the system's recommendations are.

2.2 Recall: Measures the proportion of all relevant items that were successfully recommended by the system. It is calculated as the number of relevant recommended items (TP) divided by the total number of relevant items, which is the sum of TP and False Negatives (FN).

$$\text{Recall} = \frac{TP}{TP + FN}$$

Equation 2.4 Recall Formula

Recall focuses on the system's ability to find all relevant items.

The choice of evaluation metric depends heavily on the specific goal of the recommender system. For instance, if accurately predicting user ratings is paramount, such as for displaying star predictions, RMSE is appropriate. If the goal is to generate a useful list of items for the user to explore, Precision@k and Recall@k are more indicative of performance from the user's perspective. Aligning with the project objectives outlined in Section 1.3, this work will primarily utilize RMSE to evaluate the performance.

2.4 Privacy Challenges in Centralized Recommender Systems

While recommender systems offer significant benefits in personalization and discovery, traditional implementations, which rely on centralized data architectures, introduce substantial privacy challenges. The effectiveness of many recommendation algorithms is contingent on the availability of large volumes of user data, including interaction histories (ratings, clicks, purchases), browsing patterns, and potentially demographic or contextual information (Asad et al., 2023). Aggregating this data in a central repository creates inherent vulnerabilities and risks to user privacy.

2.4.1 Data Centralization Risks and Threat Types

The concentration of sensitive user data in one location makes centralized recommender systems attractive targets and introduces several types of privacy threats which include:

1. **Data Breaches and Leakage:** Centralized databases are susceptible to unauthorized access, leading to large-scale data breaches where sensitive personal information can be exposed (Asad et al., 2023). High-profile privacy disclosure events in recent years, such as incidents involving LinkedIn and Google+, underscore these dangers (Wang et al., 2024). Even anonymized data can be vulnerable, as re-identification techniques may become possible when datasets are large or can be linked with external information.

2. **Inference Attacks:** Beyond direct data exposure, adversaries can often infer sensitive user attributes from the data held by recommender systems or even from the recommendations themselves. Analysing patterns in ratings, purchases, or consumed content can reveal information about a user's gender, age, political leanings, health status, or other private characteristics, sometimes without requiring significant background knowledge (Asad et al., 2023). Membership inference attacks, specifically, attempt to determine if a particular individual's data was used in training the recommendation model.

3. **Cross-Linking and Profiling:** The value, and risk, of collected data increases significantly when it can be linked with information from other sources. Combining recommendation data, such as book ratings on Goodreads, with browsing history, purchase records from e-commerce sites, location data, or social media activity allows for the creation of highly detailed and potentially intrusive user profiles that reveal far more than any single dataset alone. This aggregation facilitates more precise targeting but also heightens the risk of misuse and unwanted exposure (Asad et al., 2023).

4. **Data Misuse:** The entity operating the recommender system may use the collected data for purposes beyond providing recommendations, potentially without clear user consent (Asad et al., 2023). This could include undisclosed selling of data to third parties, using preferences to manipulate user behaviour, such as promoting high-margin products, or other forms of exploitation that prioritize platform goals over user interests. This sometimes leverages free labour. The opacity of many recommendation algorithms can make such misuse difficult for users to detect.

2.4.2 User Concerns and the Regulatory Landscape

There is growing public awareness and concern regarding how personal data is collected, used, and protected online (Wang et al., 2024). Users often lack transparency into the data practices of platforms, finding it difficult to understand what data is collected, how it is used to generate recommendations, or how securely it is stored (Asad et al., 2023).

This environment of heightened concern has coincided with the introduction of comprehensive data protection regulations globally. Key examples have been described by Wang et al. (2024)

The General Data Protection Regulation (GDPR): Enacted by the European Union in 2018, establishes stringent rules for processing the personal data of EU residents. It grants individuals significant rights regarding their data and mandates principles like lawfulness,

fairness, transparency, and data minimization, often requiring explicit user consent and imposing substantial fines for non-compliance.

The California Consumer Privacy Act (CCPA): Effective from 2020 and amended by CPRA, provides California consumers with rights similar to GDPR, including the right to know, delete, and opt-out of the sale or sharing of personal information. It also requires businesses to provide transparency about their data practices.

These regulations fundamentally alter the landscape for recommender systems. They necessitate robust governance frameworks, enhanced transparency, the implementation of privacy-by-design principles, and mechanisms to honor user rights requests. The potential for significant financial penalties and reputational damage due to non-compliance acts as a powerful incentive for organizations to adopt more privacy-conscious approaches. This regulatory pressure, combined with user concerns, drives the need for technological solutions that can provide personalization while respecting privacy. The inherent conflict between the data requirements of traditional centralized recommenders and the principles of data minimization and user control mandated by regulations underscores the urgency for privacy-preserving alternatives.

2.4.3 Introduction to Privacy-Preserving Recommender Systems (PRecSys)

In response to these challenges, the field of Privacy-Preserving Recommender Systems (PRecSys) has emerged. PRecSys aims to deliver useful recommendations while mitigating the privacy risks associated with traditional methods. This involves employing various techniques, often categorized as:

1. **Data Obfuscation/Perturbation:** Adding noise to user data or outputs to prevent exact inference (e.g., using Differential Privacy).

2. **Cryptographic Methods:** Using techniques like Homomorphic Encryption (HE) or Secure Multiparty Computation (SMC) to perform computations on encrypted data or distribute computation securely among parties.

3. **Data Distribution/Decentralization:** Avoiding central data aggregation by processing data locally, as seen in Federated Learning.

Federated Learning, often combined with techniques like Differential Privacy, represents a particularly promising direction for PRecSys, as it directly addresses the risks of data

centralization (Asad et al., 2023; Wang et al., 2024). Subsequent sections will delve deeper into Federated Learning and its application to recommender systems.

2.5 Federated Learning: Principles and Privacy Advantages

Federated Learning (FL) has emerged as a compelling machine learning paradigm designed explicitly to address the privacy and logistical challenges associated with training models on large, distributed datasets, particularly when those datasets contain sensitive information. It offers a way to leverage data from multiple sources without requiring the data to be centralized (Saif et al., 2025).

2.5.1 Definition and Core Principles

Federated Learning is a decentralized machine learning technique where multiple clients, such as devices like smartphones or organizations like hospitals or banks, collaboratively train a shared model (Saif et al., 2025). This process is typically orchestrated by a central server, but crucially, the raw data used for training remains on the local clients. The core principles underpinning FL are:

- **Data Localization/Decentralization:** This is the foundational principle. Training data resides and remains on the client's local device or infrastructure throughout the learning process. Only model-related information, such as updates, parameters, and gradients, is exchanged, not the underlying sensitive data (Saif et al., 2025; Wang et al., 2024). This prevents the need to move potentially large and sensitive datasets to a central location.

- **Collaborative Training:** Despite data decentralization, clients work together under the coordination of a central server to train a common, shared global model (Saif et al., 2025;).

- **Model Aggregation:** Clients perform computations, such as model training, locally on their data to generate updates to the current global model.These local updates are then securely transmitted to the central server. The server aggregates the updates received from multiple clients, for example, by averaging them, potentially weighted by dataset size, to produce an improved global model (Saif et al., 2025; Wang et al., 2024;).

- **Iterative Process:** The cycle of distributing the current global model to clients, performing local training, sending updates back to the server, and aggregating updates to refine the global model is repeated over multiple communication rounds until the model converges or reaches a desired performance level (Saif et al., 2025).

2.5.2 Inherent Privacy Advantages

Federated Learning's core strength lies in its privacy-centric design. By keeping raw data on client devices, FL mitigates many risks associated with centralized data storage, avoiding a single, vulnerable repository of sensitive information. This approach supports data minimization and reduces the attack surface compared to traditional machine learning. FL trains models using intermediate parameters—such as gradients or updates—rather than raw data, enabling collaborative learning while complying with privacy regulations. This decentralized framework significantly reduces data leakage risks and enhances user privacy by default.

2.5.3 Federated Learning vs. Distributed Learning

It is important to distinguish Federated Learning from traditional Distributed Learning, although both involve training models across multiple machines. While Distributed Learning primarily focuses on parallelizing computation to speed up training on large datasets that might still be centrally accessible or easily distributable, Federated Learning's core motivation is to enable training on decentralized data that should not be moved due to privacy, security, regulatory, or communication constraints. As Wang et al. (2024) highlight when comparing Federated Recommender Systems (FedRec) with Distributed Recommender Systems (DRecSys), FedRec can be seen as a special case of DRecSys with an added privacy constraint, where the focus shifts from pure efficiency and accuracy to balancing these with robust privacy protection.

Table 2.5.1Comparison of Federated Learning (FL) and Distributed Learning (DL)

FEATURE	FEDERATED LEARNING (FL)	DISTRIBUTED LEARNING (DL)
PRIMARY GOAL	Privacy preservation, Learning on heterogeneous data.	Parallelization, Training speed/scalability.
DATA LOCATION	Decentralized (remains on client devices/silos).	Often centralized initially, then distributed for processing.
DATA DISTRIBUTION	Assumed Heterogeneous, Non-IID, Unbalanced.	Often Assumed Homogeneous, IID, Balanced.
NETWORK/CLIENTS	Potentially many clients, Unreliable/slow connections (e.g., mobile).	Typically fewer nodes, Reliable/fast connections (e.g., data centre).
MAIN CHALLENGE	Handling heterogeneity, Communication efficiency, Privacy leakage from updates.	Efficient parallelization, Synchronization, Fault tolerance [Implicit from DL definition].

Distributed Learning focuses on speeding up training on large datasets by parallelizing computation across reliable, often IID-assuming nodes like data center servers. In contrast, Federated Learning is driven by privacy concerns and the need to train models on inherently distributed data that should not be centralized. FL tackles challenges posed by non-IID data and system heterogeneity, such as varying device capabilities and unstable networks—common in real-world clients like mobile devices or organizations (Saif et al., 2025). This focus on heterogeneity sets FL apart and introduces significant algorithmic complexity.

2.5.4 Horizontal vs. Vertical Federated Learning

Federated Learning itself can be categorized based on how data is partitioned across the participating clients, primarily into Horizontal Federated Learning (HFL) and Vertical Federated Learning (VFL) (Wang et al., 2024).

- **Horizontal Federated Learning (HFL):** In HFL, clients share the same feature space but hold different sets of samples, or data instances. Imagine multiple hospitals training a disease prediction model; each hospital has records for different patients, but the type of information recorded, such as symptoms and test results, is largely the same. The training protocol typically involves clients training local models on their data and sending model updates, such as parameters or gradients, to the server for aggregation, for example, by averaging. The output is usually a single global model shared by all participants. HFL is common in cross-device scenarios, where many users contribute data for a shared app feature, and cross-silo scenarios, where organizations possess similar types of data about different populations.

- **Vertical Federated Learning (VFL):** In VFL, clients share the same sample space, meaning they have data about the same set of individuals or entities, but possess different feature sets. Consider a bank and an e-commerce company collaborating to build a credit risk model for their common customers; the bank has financial history features, while the e-commerce company has purchase history features for the same set of users. Training in VFL is more complex, often requiring the exchange of intermediate computational results, like encrypted gradients or embeddings, between parties. This frequently employs cryptographic techniques like Homomorphic Encryption (HE) or Secure Multiparty Computation (SMC) to protect feature privacy (Asad et al., 2023). The output often involves split model components held by different parties. VFL is typically applied in cross-silo settings where organizations with complementary datasets wish to collaborate.

For building a book recommender system where individual users (clients) possess their own reading history, such as ratings and interactions, for a common set of books (items/features), the Horizontal Federated Learning (HFL) setting is the relevant paradigm. Each user holds their own sample of interactions, while the features, which are the books being rated, are shared across the system. Surveys like the one by Wang et al. (2024) specifically focus on this HFL application within recommender systems.

2.6 Federated Aggregation Algorithms

A critical step in the Federated Learning process is the aggregation phase, where the central server combines the updates received from participating clients to refine the global model. The choice of aggregation algorithm significantly impacts the efficiency, robustness, and convergence properties of the FL system, especially under heterogeneous conditions (Li et al., 2020).

2.6.1 Federated Averaging (FedAvg)

Federated Averaging (FedAvg), introduced by McMahan et al. (2017), is arguably the most foundational and widely adopted aggregation algorithm in Federated Learning (FL). It serves as a practical and communication-efficient method, particularly for training deep networks on decentralized data. The core idea involves the server distributing the current global model to a subset of selected clients. These clients then perform multiple local updates, such as using Stochastic Gradient Descent (SGD) over several epochs or mini batches on their local data, before sending their updated model parameters or the parameter deltas back to the server. The server then aggregates these updates, typically by computing a weighted average often weighted by the size of each client's local dataset, to produce the next iteration of the global model. FedAvg has demonstrated empirical robustness to unbalanced and non-IID data distributions, significantly reducing the required communication rounds compared to simpler approaches like federated SGD.

2.6.2 FedProx

While FedAvg performs well empirically, it can struggle theoretically and practically under significant statistical and systems heterogeneity. FedProx was proposed by Li et al. (2020) to specifically address these heterogeneity challenges, serving as a generalization and re-parameterization of FedAvg. The key modification in FedProx is the introduction of a proximal term to the local objective function that each client optimizes. This term penalizes large deviations of the local model update from the current global model, effectively restricting local models from drifting too far due to non-IID data. Importantly, FedProx formally allows clients

to perform variable amounts of local work, accommodating systems heterogeneity where some devices might be slower or have fewer resources, often referred to as 'stragglers'. Instead of dropping these stragglers as often happens in synchronous FedAvg implementations, FedProx incorporates their partial updates, weighted appropriately during server aggregation. Li et al. (2020) provide theoretical convergence guarantees for FedProx under heterogeneity and demonstrate empirically that it offers more stable and robust convergence compared to FedAvg, especially in highly heterogeneous settings.

The table below provides a comparison of the FedAvg and FedProx algorithms across various features.

Table 2.6.1Comparison of FedAvg and FedProx Algorithms

FEATURE	FEDERATED AVERAGING (FEDAVG)	FEDERATED PROXIMAL (FEDPROX)
PRIMARY SOURCE	McMahan et al. (2017)	Li et al. (2020)
CORE IDEA	Clients perform local SGD updates for E epochs, server computes weighted average of resulting models.	Extends FedAvg by adding a proximal term to the local objective function to limit local updates.
HANDLING HETEROGENEITY	Can struggle/diverge with high statistical (non-IID) or systems heterogeneity.	Designed to be more robust to statistical and systems heterogeneity; allows variable local work.
KEY MODIFICATION	N/A (Baseline)	Adds proximal term: $\frac{\mu}{2}$
CONVERGENCE NOTES	Convergence requires careful tuning (e.g., local epochs E); may not converge to true optimum in non-IID.	Provable convergence guarantees under heterogeneity (with assumptions); proximal term aids stability.
PROS	Simple, Communication-efficient (fewer rounds than FedSGD).	More robust convergence in heterogeneous settings, Tolerates partial client work.
CONS	Sensitive to non-IID data, Straggler issues (if dropping slow clients).	Introduces hyperparameter μ, can increase local computation cost.

2.6.3 Other Relevant Aggregation Strategies

Beyond FedAvg and FedProx, researchers have developed various aggregation strategies to address FL challenges. Some approaches, like SCAFFOLD, use control variates to correct client drift from non-IID data and improve global model convergence.

Others employ sophisticated matching or averaging techniques for better heterogeneity handling and personalization.

Future research directions include sharing specific model information, using different local training regularization, or employing clustering to group clients before aggregation. Personalized Federated Learning pFL is a growing field focused on tailoring models to individual clients, directly embracing data heterogeneity. These diverse strategies show ongoing efforts to refine FL aggregation for real-world complexities.

2.7 Horizontal Federated Learning Recommender Systems (HFedRec)

Applying the principles of Horizontal Federated Learning (HFL) to the domain of recommender systems gives rise to Horizontal Federated Recommender Systems (HFedRec). As discussed in 2.5.4 in the HFL paradigm, individual users typically act as the clients, each holding their private interaction data locally on their devices. These users collaboratively train a recommendation model, coordinated by a central server, without sharing their raw data, thus aiming to provide personalized recommendations while preserving privacy (Wang et al., 2024;).

2.7.1 Taxonomy of HFedRec Approaches

The research landscape of HFedRec is diverse, integrating concepts from recommendation algorithms, privacy techniques, and distributed systems. Wang et al. (2024) provide a useful taxonomy organizing HFedRec approaches along three primary perspectives: Model, Privacy, and Federated. This hierarchical structure helps systematically categorize the field. The figure below adopted from Wang’s paper presents the proposed taxonomy of HFedRec.

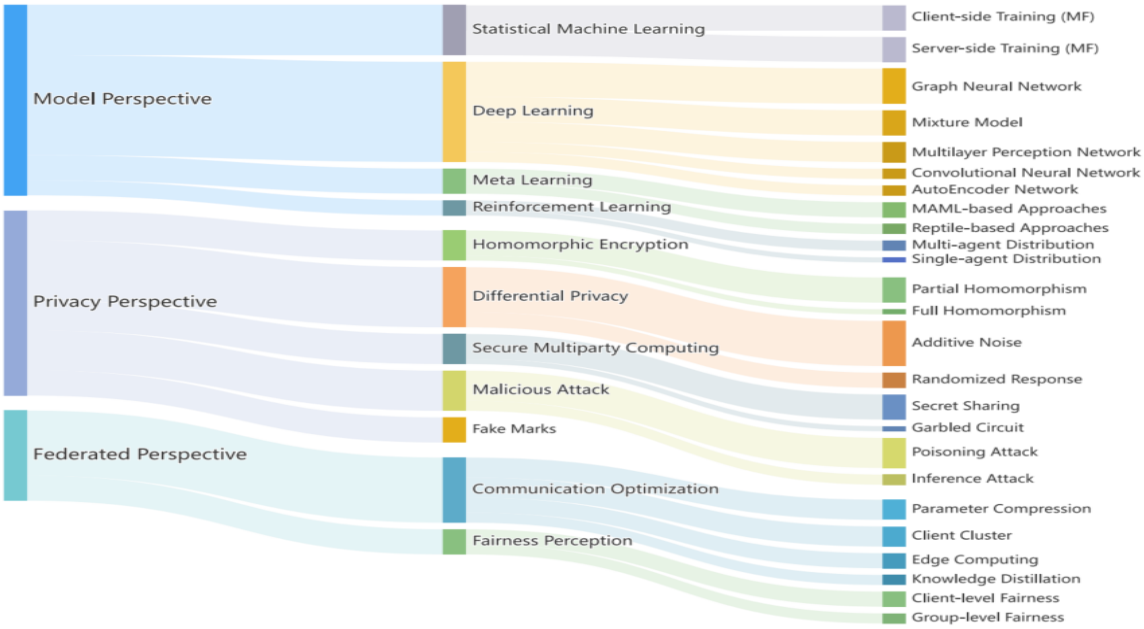


Figure 2.7.1 Proposed HFedRec taxonomy.

This taxonomy highlights the multifaceted nature of HFedRec research. Common examples fitting this taxonomy include Federated Matrix Factorization (FedMF) and Federated Neural Collaborative Filtering (FedNCF).

2.7.2 Common challenges in HFedRec (Non-IID data, communication)

While Horizontal Federated Recommender Systems offer significant privacy advantages over centralized approaches, they introduce their own set of unique challenges stemming from the distributed and heterogeneous nature of the learning process. Addressing these is crucial for the practical deployment and effectiveness of HFedRec.

1. **Statistical Heterogeneity (Non-IID Data):** A defining characteristic of federated learning, particularly in recommender systems, is that client data is typically not independent and identically distributed (non-IID). Users naturally have diverse preferences, different rating scales or frequencies, varying amounts of interaction data, and interests that cover different subsets of items. This statistical heterogeneity can significantly impede the convergence of the global model, potentially cause it to drift away from a good general solution and degrade overall recommendation performance compared to training on IID data (McMahan et al., 2017; Li et al., 2020). Aggregation algorithms like FedProx as mentioned earlier in 2.6.2 are specifically designed, to mitigate the negative impacts of this non-IID data distribution.

2. **Communication Overhead:** As Federated learning inherently involves repeated communication between the central server and participating clients to exchange model parameters or updates. For complex recommendation models like those extending deep learning models with millions of parameters, transmitting these updates can become a major bottleneck (Saif et al., 2025). This consumes significant network bandwidth and requires energy, which is particularly critical for mobile devices, and can increase the overall training time (McMahan et al., 2017). Consequently, developing communication-efficient strategies, such as model compression, update sparsification, or less frequent communication rounds, is a primary area of research in FL and HFedRec (Wang et al., 2024;).

3. **Systems Heterogeneity:** Beyond statistical data differences, the clients participating in HFedRec often exhibit significant systems heterogeneity (Saif et al., 2025). Devices vary greatly in terms of computational power CPU/GPU, memory, network connection stability and speed Wi-Fi, 3G/4G/5G, and power availability battery level. This variability makes strictly synchronous update protocols, where the server waits for all selected clients to report back, impractical. Slower devices or those with poor connections become stragglers, potentially delaying entire training rounds, or being dropped altogether, which can introduce bias. Frameworks like FedProx explicitly aim to address systems heterogeneity by allowing clients to perform variable amounts of local work and incorporating these partial results, rather than simply discarding contributions from stragglers (Li et al., 2020).

4. **Privacy Concerns:** Although FL's core design prevents raw data sharing, the transmission of model updates is not inherently immune to privacy leakage (Asad et al., 2023;). Sophisticated adversaries might potentially infer sensitive information about a client's local data by analysing these shared updates through techniques like model inversion or membership inference attacks. Techniques such as Differential Privacy (DP) applied to the updates, Secure Multiparty Computation (SMC) for secure aggregation, or Homomorphic Encryption (HE) are often employed to provide stronger, mathematically rigorous privacy guarantees.

5. **Cold-Start Problem:** The cold-start problem, a persistent challenge in traditional recommender systems, also affects HfedRec. It refers to the difficulty of providing meaningful recommendations for new users who have little to no interaction history, or for newly introduced items that have not yet been rated or interacted with by users. In the

federated setting, gathering sufficient initial data locally for a new user can be slow, and aggregating information about new items across clients presents unique challenges. While FL's collaborative nature might eventually help gather diverse opinions on new items, specific federated strategies, sometimes involving trust mechanisms or leveraging item attributes when available, are actively being researched to explicitly address both user and item cold-start scenarios (Wang et al., 2024).

2.8 State-of-the-Art in Horizontal Federated Book Recommendations

Building upon the principles and challenges of Horizontal Federated Learning, this section delves into the current state-of-the-art specifically concerning its application to book recommender systems, often utilizing datasets like Goodreads.

2.8.1 Overview of HFedRec Research

The domain of Federated Recommender Systems has experienced a surge in research interest, marked by a significant increase in publications from 2019 to mid-2023. Within this rapidly expanding field, HFedRec have emerged as the most dominant research subject, attracting considerably more attention than Vertical or Transfer FedRec approaches (Wang et al., 2024, Fig. 1). This focus stems largely from HFedRec's natural applicability to user-centric scenarios where data resides locally on individual devices, making user-level privacy inherently feasible.

Current state-of-the-art efforts within HFedRec primarily revolve around adapting various recommendation algorithms for the federated setting. While foundational techniques like Matrix Factorization (MF), the core algorithm for this project, remain frequently explored (e.g., FedMF), there is a clear trend towards integrating more complex Deep Learning (DL) models such as Neural Collaborative Filtering (NCF) and Graph Neural Networks (GNNs). Alongside model adaptation, integrating robust privacy enhancements and developing strategies to mitigate core federated challenges such as communication overhead and non-IID data are central themes in contemporary HFedRec research.

This focus on adapting models, enhancing privacy, and overcoming federated obstacles within the HFedRec paradigm defines the current research frontier. The following section will examine how these advancements are being applied specifically to book recommendation systems using datasets like Goodreads.

2.8.2 State of Art Book Recommendation Systems (e.g., using Goodreads)

While the general principles and advancements in HFedRec explored in the previous section are often demonstrated on standard datasets like MovieLens or Yelp, dedicated research applying these techniques specifically to the unique characteristics and scale of book recommendation datasets, such as Goodreads, appears less prevalent in readily available published literature. This relative scarcity provides a crucial context for the necessity and focus of this project.

Existing work within the book recommendation domain, even if centralized, offers valuable background. For instance, Ghugare (2024) explored hybrid approaches on Goodreads data, combining sentiment analysis from reviews with collaborative filtering methods like SVD and KNN, demonstrating techniques relevant to the data type, albeit without the federated learning layer.

Nevertheless, some recent work has begun to bridge this gap by directly exploring the application of FL to book recommendations using Goodreads. Malempati et al. (2024), for example, specifically benchmarked the performance of standard recommendation algorithms (including SVD, NMF, and NCF) in both non-federated and federated settings on this dataset. Their evaluation provided initial insights into the feasibility and the performance trade-offs (e.g., observing differences in F1 and RMSE metrics) encountered when applying basic FL to book data.

This emerging focus aligns with broader surveys in the field. Asad et al. (2023), in their comprehensive review of privacy-preserving techniques in Federated Recommendation Systems (FRSs), explicitly include Goodreads among datasets considered relevant for FRS research, underscoring the perceived importance and suitability of this data for investigating federated approaches.

Therefore, while the Goodreads dataset is recognized as important for FRS research and initial evaluations exist, the body of published work focusing specifically on the challenges and opportunities of implementing and optimizing *Horizontal Federated Learning* for book recommendations remains relatively limited. This observation paves the way for identifying specific research gaps addressed by this study in the following section.

2.9 Synthesis and Research Gap Identification

Synthesizing the findings from this literature review reveals significant progress in leveraging Federated Learning (FL) for privacy-preserving recommendations. Horizontal Federated Recommender Systems (HFedRec) represents a rapidly growing field where Matrix Factorization (MF) remains a relevant and widely studied model, while FedAvg and FedProx have become established as key aggregation strategies. Furthermore, datasets such as Goodreads are recognized as pertinent testbeds for federated recommender systems research, offering valuable insights into real-world application dynamics and user behavior patterns.

However, consolidating the literature reveals a distinct empirical research gap: while Federated Learning is a promising paradigm, there remains insufficient empirical analysis that systematically quantifies its performance trade-offs when applied to large-scale, highly sparse datasets. Specifically, the practical impact of decentralization on the interplay between model accuracy and system scalability—particularly when implementing a standard Matrix Factorization model with the Federated Averaging (FedAvg) algorithm—remains an open question. This gap impedes a comprehensive understanding of whether users can achieve superior recommendations while maintaining data locality, a core challenge that this project aims to address by testing the hypothesis that a Horizontal Federated Recommender System for book recommendations can achieve comparable accuracy to a centralized approach while enhancing privacy and exhibiting predictable scalability. This project will therefore use the Goodreads dataset as a representative case study to evaluate these dynamics empirically.

Concurrently, a significant application gap exists in the form of a lack of interactive visualization tools specifically designed to analyze the complex performance dynamics of federated learning simulations. While federated systems generate rich data on accuracy, scalability, and convergence across numerous clients and communication rounds, these insights often remain hidden within log files or static reports. This absence of dedicated dashboards hinders the practical interpretation of the inherent trade-offs between federated and centralized approaches, making it difficult for researchers and developers to intuitively grasp the nuanced relationships between

decentralization, efficiency, and model performance. Consequently, a tool that translates dense simulation output into accessible, comparable visualizations is crucial for enabling more informed, evidence-based system design and evaluation.

This project aims to directly address both identified gaps. It will systematically evaluate the performance of a federated, MF-based book recommender system, comparing its accuracy and scalability trade-offs against a traditional centralized baseline. Simultaneously, it will develop an interactive dashboard to visualize these findings, bridging the application gap and offering a valuable tool for federated learning analysis. This dual focus validates the project's necessity and contribution towards answering the central research question regarding privacy-preserving, high-quality book recommendations in federated environments.

2.10 Summary

This chapter has provided a comprehensive review of the literature relevant to federated recommender systems, tracing the evolution from traditional centralized approaches to privacy-preserving federated paradigms. The review began by examining the fundamental privacy and scalability challenges inherent in centralized recommender systems, establishing the motivation for exploring alternative architectures. Federated Learning emerged as a promising solution, offering a decentralized training paradigm that inherently enhances privacy by keeping user data local while enabling collaborative model training.

The chapter delved into the core principles of Federated Learning, differentiating it from traditional distributed learning approaches and outlining the Horizontal Federated Learning (HFL) setting most relevant for user-based recommendations. Key aggregation algorithms central to HFL—namely FedAvg and FedProx—were discussed in detail, with particular attention to their respective strengths, computational characteristics, and robustness under heterogeneous data conditions. The application of HFL to recommender systems (HFedRec) was then explored, highlighting common challenges such as non-IID data distribution and communication overhead that arise in federated settings.

The state-of-the-art in federated recommender systems was examined, revealing the continued prevalence of Matrix Factorization alongside emerging deep learning trends. Particular attention was given to book recommendation systems and the use of datasets such as Goodreads, which provide realistic testbeds for evaluating federated approaches. Through this comprehensive review, the chapter established both the theoretical foundations and practical considerations necessary for developing and evaluating federated recommender systems, while identifying critical gaps that motivate the current project's empirical and application-focused contributions.

Chapter 3: Requirements Elicitation and Analysis

3.1 Introduction

This chapter details the requirements elicitation and analysis process for the Dashboard. Its primary objective is to define the scope, functionalities, and operational characteristics of the proposed system. Through a systematic approach, this chapter will identify the key actors interacting with the system, enumerate the functional and non-functional requirements, and comprehensively model the system. This modelling encompasses its interactive behaviour from a user's perspective, Use Cases, the dynamic flow of operations and state transitions, Sequence, Activity, and State Diagrams, and its underlying static and data structures, Class and Entity-Relationship Diagrams.

The requirements and models presented herein, including these behavioral and structural diagrams, will serve as a foundational blueprint for the subsequent design and implementation phases of the project. By clearly articulating what the system must do and how it should perform, this chapter aims to ensure that the developed dashboard effectively meets the needs of its users and fulfills the project's overarching goals of providing a robust tool for evaluating federated learning in book recommender systems.

3.2 System Actors

Actors represent entities, human or system, that interact with the dashboard. These roles help define how different users and components engage with the system's functionality. The main actors are:

1. **System:** The core technical component representing the dashboard application. It is responsible for managing federated learning simulations, processing data, visualizing results, and responding to user inputs.

2. **User:** This human actor represents any individual who interacts with the dashboard. The User engages with the system to achieve a spectrum of objectives, which can range from highly technical investigations to high-level strategic understanding. Their interactions may involve configuring, running, and meticulously analysing federated learning (FL) experiments in detail, as well as evaluating system performance, optimizing models, and exploring trade-offs such as accuracy versus privacy. Furthermore, the User may consume summarized insights, high-level reports, and visual dashboards to understand trends and

support strategic or operational decision-making. The User leverages the dashboard's capabilities to evaluate the effectiveness, scalability, and trade-offs of federated learning in book recommender systems, adapting their level of interaction and focus based on their specific goals and expertise.

In essence, the System provides the tools and data, while the User interacts with it to gain insights and make evaluations, tailoring their engagement according to their specific analytical or strategic needs.

3.3 Functional Requirements

To delineate the specific functionalities required by the system and the user, a structured approach has been adopted to define the Functional Requirements (FRs). These requirements are categorized based on the primary actor they concern: the System itself, and the User interacting with it. The subsequent tables provide a detailed breakdown of these requirements, outlining the essential actions and capabilities that the dashboard must possess to effectively support the project's goals.

To ensure the dashboard operates effectively and meets the project's objectives, specific functionalities are mandated for the system itself. These Functional Requirements (FRs) detail the core operations the system must perform, from data handling and simulation execution to metric logging and result visualization. The following table outlines these essential system-level functionalities that form the backbone of the dashboard's capabilities.

Table 3.3.1 Functional Requirements for the System Actor

Req ID	Description
FR-SYS-01	The system shall load and preprocess the Goodreads dataset by filtering out interactions with zero ratings.
FR-SYS-02	The system shall implement Matrix Factorization (MF) as the core recommendation algorithm for the centralized recommendation environment.
FR-SYS-03	The system shall execute the training of the centralised MF model.
FR-SYS-04	The system shall record the finals accuracy (RMSE) and total training time for the centralised model
FR-SYS-05	The system shall simulate a virtual federated learning environment using the loaded Goodreads dataset.
FR-SYS-06	The system shall partition the preprocessed dataset into local user-specific subsets, where each user is an individual client to simulate the Horizontal Federated Learning (HFL) environment..
FR-SYS-07	The system shall implement Matrix Factorization (MF) as the core recommendation algorithm within a federated learning environment.
FR-SYS-08	The system shall implement FedAvg as the default aggregation algorithm for federated model training.
FR-SYS-10	The system shall record the following performance metrics after each global federated learning round: global weighted average RMSE, and time taken for the round.
FR-SYS-11	The system shall display convergence curves visualizing global weighted average RMSE across federated training rounds for selected simulations.
FR-SYS-12	The system shall support the execution of federated learning simulations with a configurable number of participating clients
FR-SYS-13	The system shall visualize a comparison of training time versus the number of participating clients for federated learning simulations
FR-SYS-14	The system shall evaluate global model performance on each client's local test set after each training round
FR-SYS-15	The system shall visualize a comparison of model accuracy (global weighted average RMSE) versus the number of participating clients for federated learning simulations
FR-SYS-16	The system shall display a comparison of final model accuracy (RMSE) and total training time between the centralized and federated versions of the Matrix Factorization model using the same dataset.
FR-SYS-17	The system shall allow export of generate visualizations in a common image format (e.g. PNG)

Following the definition of the system's functional requirements, the following table outlines the essential functionalities from the perspective of the User interacting with the dashboard. These requirements detail how the User will engage with the system to configure experiments, initiate simulations, and interpret the results, ensuring a user-centric and effective experience.

Table 3.3.2 Functional Requirements for the User Actor

Req ID	Description
FR-USR-01	The user shall be able to initiate the training process for the centralized baseline model and the federated learning model.
FR-USR-02	The user shall be able to configure hyperparameters (e.g., learning rate, regularization, number of latent factors) for the matrix factorization model used in training.
FR-USR-03	The user shall be able to configure the number of virtual clients for a federated learning simulation
FR-USR-04	The user shall be able to modify the number of training rounds in the federated setup.
FR-USR-05	The user shall be able to view scalability trends showing training time and accuracy versus increasing number of clients.
FR-USR-06	The user shall be able to view visualizations of model performance (e.g., RMSE trends, convergence graphs) via the interactive dashboard.
FR-USR-07	The user shall access side-by-side comparisons of centralized vs. federated models.

With the functional requirements clearly established for both the System and the User, the pathway is now set for addressing the crucial non-functional aspects that will define the dashboard's quality and operational characteristics.

3.4 Non-functional Requirements

Beyond the specific functions detailed above, Non-Functional Requirements (NFRs) define the quality attributes and operational characteristics of the dashboard. These ensure the system is usable, reliable, performs efficiently, and adheres to other critical constraints. The following table outlines the non-functional requirements of the dashboard.

Table 3.4.1 Key NFRs for this project.

Req ID	Category	Description
NFR-01	Usability	The dashboard shall be intuitive and accessible to users with varying technical backgrounds.
NFR-02	Usability	All visualizations (charts, graphs) must be clearly labelled with appropriate titles, axis labels, and legends, ensuring that data insights are easily interpretable.
NFR-03	Usability	Users shall be able to easily configure experiment parameters through the dashboard interface.
NFR-04	Reliability	The calculated and displayed performance metrics (e.g., RMSE, MAE, training time) must be accurate and consistent with the underlying model computations for the given dataset and configuration.
NFR-05	Reliability	The system shall provide informative error messages if a simulation fails or if invalid configuration parameters are entered by the user, guiding them towards correction.

NFR-06	Scalability	The system shall handle sparse datasets efficiently using CSR (Compressed Sparse Row) matrix representation to reduce memory usage and improve computational efficiency.
NFR-07	Maintainability	The system's codebase shall be modular, allowing for future enhancements such as the addition of new federated aggregation algorithms or evaluation metrics with minimal impact on existing components.
NFR-08	Privacy	The system shall ensure that user data remains on local devices during training by implementing Federated Learning, minimizing exposure of raw data.
NFR-09	Performance	The dashboard interface shall remain responsive during user interactions (e.g., configuring parameters, navigating views). For longer operations, visual feedback (e.g., loading indicators) shall be provided to the user.
NFR-10	Performance	The system shall execute federated learning simulations efficiently, allowing for timely evaluation. Execution time should scale in a reasonably predictable manner with the number of clients and rounds.

3.5 Use Case Modelling

To further articulate the system's capabilities and how the User actor will interact with them to achieve specific objectives, this section employs Use Case Modelling. This approach provides a crucial user-centric view of the system's functional requirements, visualizing how the dashboard responds to user actions and fulfills their goals. It serves as a bridge between the abstract requirements identified earlier and a more concrete understanding of the system's intended behaviour.

We begin with a Use Case Diagram to offer a high-level visual overview of the primary interactions between the User and the dashboard. This visual summary is then complemented by detailed textual descriptions for each identified use case, elaborating on their specific triggers, steps, preconditions, and postconditions.

3.5.1 Use Case Diagram

The Use Case Diagram provides a high-level visual representation of the key interactions (use cases) between the User and the dashboard system. It serves as an initial blueprint to understand the scope of functionalities offered by the dashboard and how these functionalities are interconnected from the User's perspective.

Figure 3.5.1 below depicts these primary use cases and their relationships, illustrating the main services the dashboard offers and how they relate to each other through <<include>> and <<extend>> relationships.

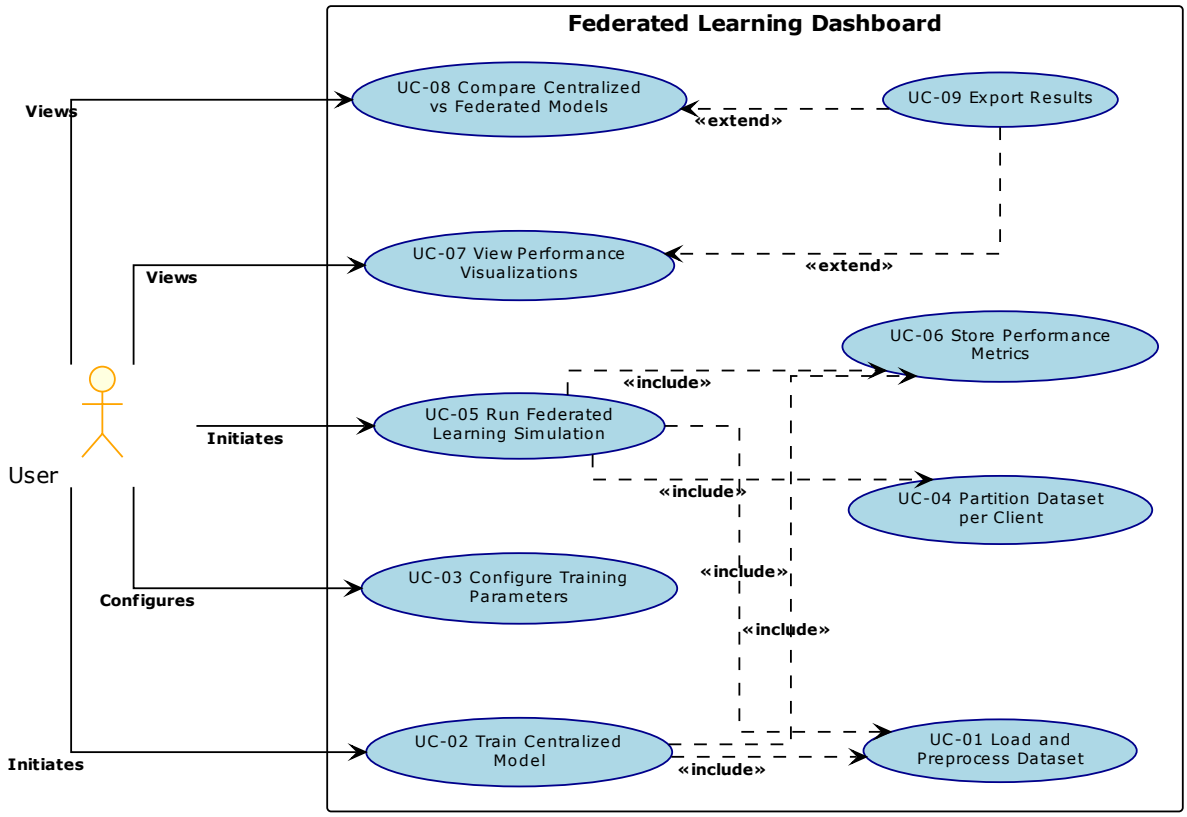


Figure 3.5.1 Use Case Diagram for the Federated Learning Dashboard

The diagram in Figure 3.5.1 illustrates that the User directly initiates core processes such as UC-02 Train Centralized Model, UC-05 Run Federated Learning Simulation, and UC-03 Configure Training Parameters. The User also Views the outcomes of these processes through UC-07View Performance Visualizations and UC-08 Compare Centralized vs Federated Models.

Several functionalities are fundamental to the core operations, as indicated by <<include>> relationships. Both the centralized training process (UC-02) and the federated learning simulation (UC-05) necessarily include UC-01 Load and Preprocess dataset and UC-06 Store Performance Metrics. This signifies that loading and preprocessing the data, as well as storing the resulting performance metrics, are integral sub-processes automatically invoked during these training operations. Furthermore, UC-05 Run Federated Learning Simulation specifically includes UC-04 Partition Dataset per Client, highlighting that data partitioning is an essential preparatory step for conducting federated simulations.

The diagram also showcases optional functionalities through <<extend>> relationships. For instance, UC-09 Export Results extends both UC-07 View Performance Visualizations and UC-08 Compare Centralized vs Federated Models. This indicates that exporting results is an

additional, optional action the User can choose to perform when viewing performance visualizations or model comparisons.

This diagram effectively outlines the main user interactions and the system's responses, providing a clear roadmap of the dashboard's capabilities from an external viewpoint. The following section provides detailed descriptions for each of these identified use cases.

3.5.2 Use Case Descriptions

Following the visual overview provided by the Use Case Diagram (Figure 3.5.1), this section provides detailed descriptions for each identified use case. Each description outlines the purpose of the use case, the involved actor, its preconditions, postconditions, and a representative scenario to illustrate its practical application within the dashboard. These descriptions serve to clarify the specific interactions and functionalities encompassed by each use case.

Table 3.5.1 UC-01 Load and Preprocess Dataset.

ATTRIBUTE	VALUE
Use Case ID	UC-01
Actor	System
Title	Load and Preprocess Dataset
Description	Loads the Goodreads dataset, filters out zero ratings, and maps raw user/item IDs to contiguous integer indices.
Precondition(s)	Raw dataset is available Required libraries (e.g., Pandas) are installed
Postcondition(s)	Dataset filtered (zero ratings removed) User/book IDs mapped to integers
Main Flow of events	1. User initiates an experiment via the Dashboard UI. 2. The System automatically triggers the "Load and Preprocess Dataset" use case. 3. The Dataset Loader component loads the raw Goodreads dataset. 4. The raw data is passed to the Data Filter, which removes entries with a rating of 0. 5. The filtered data is sent to the ID Mapper for user and book ID mapping to integers. 6. The mapped data is returned to the Dataset Loader. 7. The Dataset Loader finalizes pre-processing and returns the cleaned dataset. 8. The System confirms the dataset is loaded and preprocessed, ready for the next stage.

Table 3.5.2 UC-02 Train Centralized Model

ATTRIBUTE	VALUE
Use Case ID	UC-02
Actor	User
Title	Train Centralized Model
Description	Trains a baseline MF model on the full preprocessed dataset for performance comparison.
Precondition(s)	UC-01 completed.

	UC-03 completed
Postcondition(s)	A trained centralized model exists. Performance metrics (RMSE) logged.
Main Flow of events	1. User navigates to the "Centralized Training" section of the Dashboard UI. 2. User optionally configures training parameters (e.g., learning rate, number of latent factors) via the "Configure Training Parameters" use case (UC-03). 3. User clicks the "Start Centralized Training" button. 4. The Dashboard UI sends a trigger to the Training Manager. 5. The Training Manager loads the pre-processed dataset and initializes the MF Trainer. 6. The MF Trainer trains the model using the configured parameters. 7. Upon completion, the MF Trainer computes the final RMSE and training time.

Table 3.5.3 UC-03 Configure Training Parameters

ATTRIBUTE	VALUE
Use Case ID	UC-03
Actor	User
Title	Configure Training Parameters
Description	Allows the user to set hyperparameters such as learning rate, regularization, number of latent factors, and local epochs for both centralized and federated models.
Precondition(s)	Dashboard interface is accessible
Postcondition(s)	Hyperparameters updated for both training modes
Main flow of events	1. User accesses the configure training area within the Dashboard UI. 2. The Dashboard UI displays the current configuration settings. 3. User inputs new parameters (e.g., adjusts learning rate, number of latent factors, number of local epochs). 4. User clicks the "Submit Configuration" or "Save Changes" button. 5. The Dashboard UI sends the configuration to the Configuration Manager. 6. The Configuration Manager saves the new parameters

Table 3.5.4 UC-04: Partition Dataset per Client

ATTRIBUTE	VALUE
Use Case ID	UC-04
Actor	System
Title	Partition Dataset per Client
Description	Partitions the dataset into client-specific subsets based on user index for Horizontal Federated Learning simulation.
Precondition(s)	Dataset has been preprocessed (UC-01 completed)
Postcondition(s)	Each client has its own local dataset for FL simulation
Main flow of events	1. The "Run Federated Learning Simulation" use case (UC-05) is initiated. 2. The System automatically triggers the "Partition Dataset per Client" use case. 3. The Dataset Handler sends a "Request partitioning" message to the Partitioner. 4. The Partitioner splits the preprocessed dataset based on unique user indices. 5. The Partitioner returns the client datasets to the Dataset Handler. 6. The Dataset Handler sends a "Notify completion" message to the System, indicating successful partitioning.

	7. Each client is now ready to receive its specific data partition.

Table 3.5.5 UC-05: Run Federated Learning Simulation

ATTRIBUTE	VALUE
Use Case ID	UC-05
Actor	User
Title	Run Federated Learning Simulation
Description	Simulates federated learning using Flower framework with FedAvg across multiple clients.
Precondition(s)	- Datasets are partitioned per client (UC-04 completed)
Postcondition(s)	- Federated learning rounds are executed. - Global model is updated after each round.
Main Flow of events	1. User navigates to the "Federated Learning" section of the Dashboard UI. 2. User optionally configures federated learning parameters (e.g., number of clients, communication rounds) via UC-03. 3. User clicks the "Start Federated Learning Simulation" button. 4. The Dashboard UI sends a "Trigger FL run" message to the FL Coordinator. 5. The FL Coordinator initializes the global model and sends it to the FL Server. 6. The FL Server distributes the initial global model parameters to selected clients via the Client Manager. 7. For each communication round: a) Client Execution: Clients receive the global model, train locally, and upload their model updates. b) Server Aggregation: The FL Server receives updates and aggregates them to produce a new global model. c) Metric Logging: The FL Coordinator logs metrics (e.g., RMSE, round time) and sends the updated model back to the FL Server.

Table 3.5.6 UC-06: Store Performance Metrics

ATTRIBUTE	VALUE
Use Case ID	UC-06
Actor	System
Title	Store Performance Metrics
Description	Logs key metrics like RMSE, MAE, convergence speed, and training time after each FL round.
Precondition(s)	FL simulation is running or completed (UC-05 completed) or UC-02 completed
Postcondition(s)	Metrics stored for visualization and analysis
Main flow of events	1. Following the completion of a training phase (either centralized via UC-02 or a federated round via UC-05) 2. If Centralized Training: The Training Manager sends a LogMetrics(RMSE, training_time) message to the Metrics Logger. 3. If Federated Learning Round: The FL Coordinator sends a LogMetrics(global_RMSE, round_time) message to the Metrics Logger. 4. The Metrics Logger performs an AppendToLogFile() operation to store the received metrics. 5. Upon successful logging, the Metrics Logger sends a "Confirm storage" message back to the initiating component (Training Manager or FL Coordinator).

	6. The stored metrics are now available for visualization and analysis.

Table 3.5.7 UC-07: View Performance Visualizations

ATTRIBUTE	VALUE
Use Case ID	UC-07
Actor	User
Title	View Performance Visualizations
Description	Displays interactive graphs showing RMSE trends, convergence curves, and comparisons between centralized and federated models.
Precondition(s)	- Dashboard is accessible - Metrics have been logged (UC-06 completed)
Postcondition(s)	User views dynamic visualizations of model performance
Main flow of evetns	1. Following the completion of a training phase (either centralized via UC-02 or a federated round via UC-05) 2. User Navigates to results in the Dashboard UI 3. The Visualization Engine requests logged data from the Metrics Logger using a "RetrieveLoggedData()" message. 4. The Metrics Logger returns the relevant logged data (e.g., rounds vs. RMSE, client count vs. training time). 5. The Visualization Engine generates the chart using the provided data. 6. The Visualization Engine sends the rendered chart back to the Dashboard UI. 7. The Dashboard UI displays the visualization to the User.

Table 3.5.8 UC-08 Compare Centralized vs Federated Models

ATTRIBUTE	VALUE
Use Case ID	UC-08
Actor	User
Title	Compare Centralized vs Federated Models
Description	Enables side-by-side comparisons of accuracy, training time, and scalability between centralized and federated approaches.
Precondition(s)	- Both models trained (UC-02 and UC-05 completed). - Metrics logged (UC-06 completed)
Postcondition(s)	User views comparative insights and trade-offs between models
Main Flow of Events	1. User navigates to the "Results" section of the Dashboard UI. 2. User selects a specific centralized training run (e.g., from UC-02) for comparison. 3. User selects a federated learning simulation (e.g., from UC-05) for comparison. 4. The Dashboard UI sends a "Request model comparison" message to the Comparison Engine. 5. The Comparison Engine requests metrics for both models from the Metrics Logger. 6. The Comparison Engine performs calculations to determine differences (e.g., RMSE difference, training time ratio). 7. The Comparison Engine formats the comparison data for presentation. 8. The Comparison Engine sends the comparison results to the Dashboard UI.

	9. The Dashboard UI displays the comparative insights, potentially as charts or tables, to the User.

Table 3.5.9 UC-09 Export Results

ATTRIBUTE	VALUE
Use Case ID	UC-09
Actor	User
Title	Export Results
Description	Allows user to export graphs, logs, and summary reports in common formats. (e.g., PNG, CSV, PDF).
Precondition(s)	Visualizations and metrics are available (UC-07 and UC-08 completed)
Postcondition(s)	Files exported and made available for download
Scenario	1. User is viewing results or comparison data (UC-07 or UC-08). 2. User clicks the "Export Results" button or option. 3. The Dashboard UI sends an "Open export options" message to the Export Module. 4. The Export Module interacts with the Visualization Engine and Metrics Logger to capture the necessary data/visualizations. 5. User selects the desired export format(s) (e.g., CSV, PNG, PDF). 6. The Export Module sends a "GenerateFiles" message with the selected formats to the File Generator. 7. The File Generator creates the output files. 8. The File Generator sends a "FileGenerationComplete" message with the file paths back to the Export Module. 9. The Export Module provides download links or prompts for download to the Dashboard UI. 10. The Dashboard UI allows the User to download the exported files.

The detailed use case descriptions outlined the primary flows and provided a clear and granular understanding of the dashboard's interactive capabilities. These descriptions, derived from the high-level use case diagram, serve to solidify the system's functional requirements by illustrating concrete user interactions and system responses. This detailed exploration of the system's behavior is now established, setting the stage for the subsequent phase of dynamic modeling, which will further elaborate on the system's operational workflows and state transitions.

3.6 Dynamic Modelling

While Use Case diagrams provide a high-level view of user-system interactions, dynamic models delve deeper into the behaviour of the system over time and the sequence of operations. This section presents various UML dynamic models, including Sequence Diagrams, Activity Diagrams, and State Diagrams, to illustrate the flow of control, object interactions, and state transitions within key processes of the dashboard. These models are

crucial for understanding the system's runtime behaviour and ensuring that the designed interactions fulfill the specified functional requirements.

3.6.1 Sequence Diagrams

Sequence Diagrams are used to model the interactions between objects in a sequential order, emphasizing the time-ordering of messages. They are particularly useful for visualizing how different components of the system collaborate to achieve a specific use case or operation. The following diagrams illustrate the sequence of interactions for critical functionalities of the dashboard.

1. SD1: Load and Preprocess Dataset

This sequence diagram (Figure 3.6.1.1) illustrates the interactions involved when the system loads and preprocesses the Goodreads dataset. The process is initiated by the System actor and involves several key components: the Dataset Loader, Data Filter, ID Mapper, and Preprocessor.

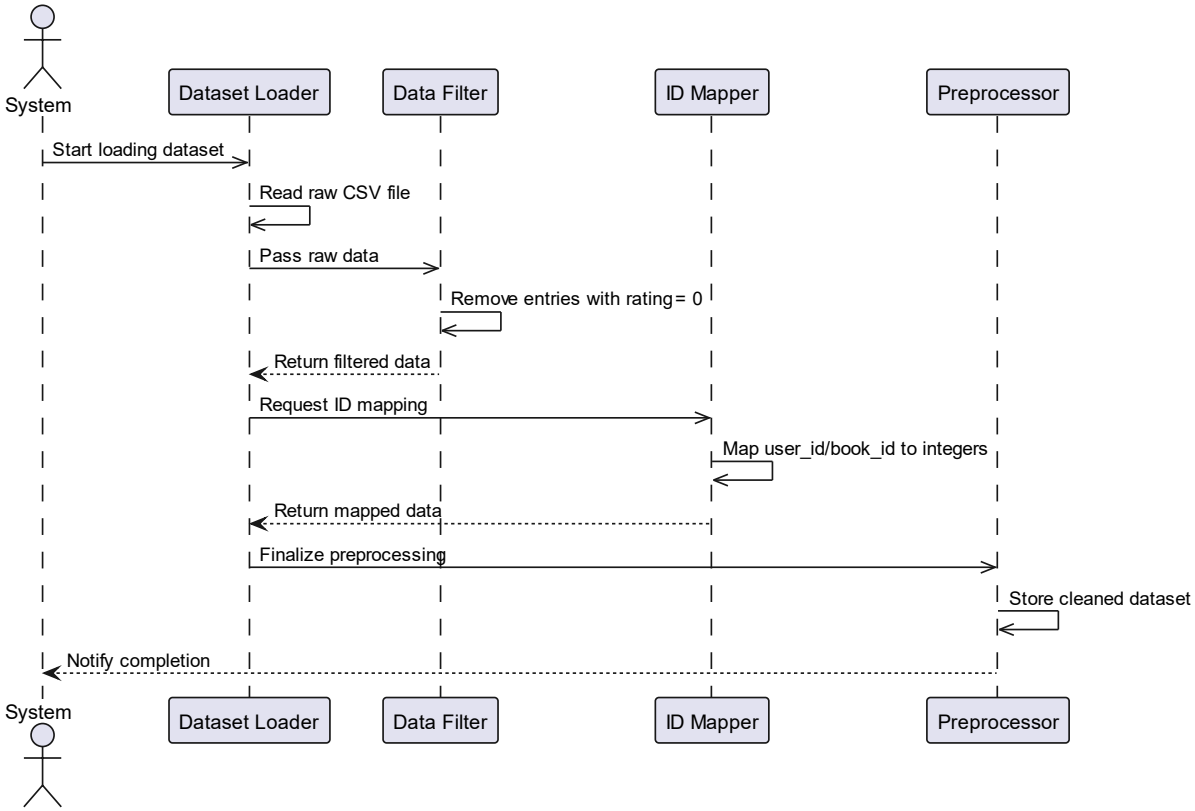


Figure 3.6.1 Sequence Diagram for Load and Preprocess Dataset

As depicted in Figure 3.6.1.1, the process begins when the System sends a Start loading dataset message to the Dataset Loader. The Dataset Loader then reads the raw CSV file and passes the raw data to the Data Filter. The Data Filter is responsible for Remove entries with rating = 0 and subsequently returns the filtered data to the Dataset Loader. Following this, the Dataset Loader sends a Request ID mapping message to the ID Mapper. The ID Mapper performs the Map user_id/book_id to integers operation and returns the mapped data to the Dataset Loader. Finally, the Dataset Loader sends a Finalize preprocessing message to the Preprocessor, which then Store cleaned dataset. Upon completion of these steps, the Preprocessor sends a Notify completion message back to the System, indicating that the dataset is loaded and pre-processed.

2. SD2: Train Centralized Model

The sequence diagram in Figure 3.6.1.2 details the process of training a centralized recommendation model. This interaction is initiated by the User through the Dashboard UI and involves the Training Manager, MF Trainer (Matrix Factorization Trainer), and Metrics Logger components.

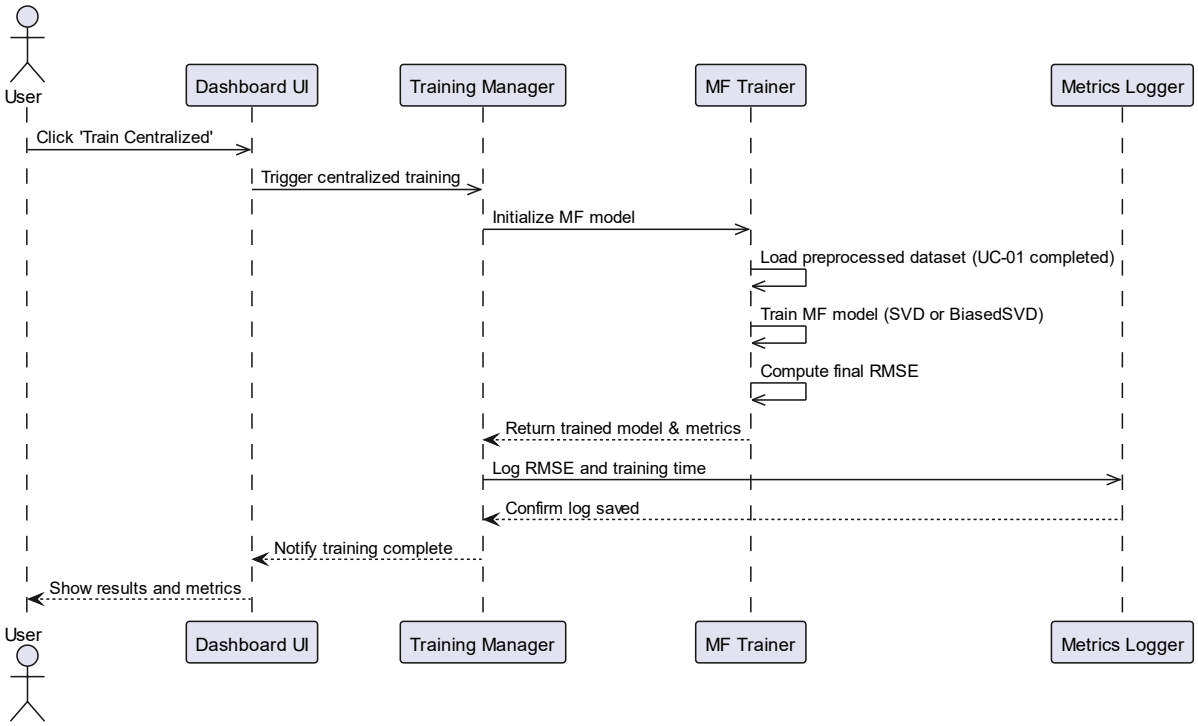


Figure 3.6.2 Sequence Diagram for Train Centralized Model

As shown in Figure 3.6.1.2, the User initiates the process by sending a Click 'Train Centralized' message to the Dashboard UI. The Dashboard UI then sends a Trigger centralized training message to the Training Manager. The Training Manager invokes the TrainModel() method on the MF Trainer. The MF Trainer first Load preprocessed dataset (UC-01 completed), then proceeds to Train MF model (SVD or BiasedSVD), and finally Compute final RMSE. Once training is complete, the MF Trainer sends a Return metrics message back to the Training Manager. The Training Manager then sends a Log RMSE and training time message to the Metrics Logger, which responds with a Confirm log saved message. Subsequently, the Training Manager sends a Notify training complete message to the Dashboard UI, which in turn Show results and metrics to the User.

3. SD3: Configure Parameters

Figure 3.6.1.3 illustrates the sequence of interactions when the User configures training parameters via the Dashboard UI. This process primarily involves the Dashboard UI and the Configuration Manager and includes an alt fragment to manage both successful and failed validation scenarios.

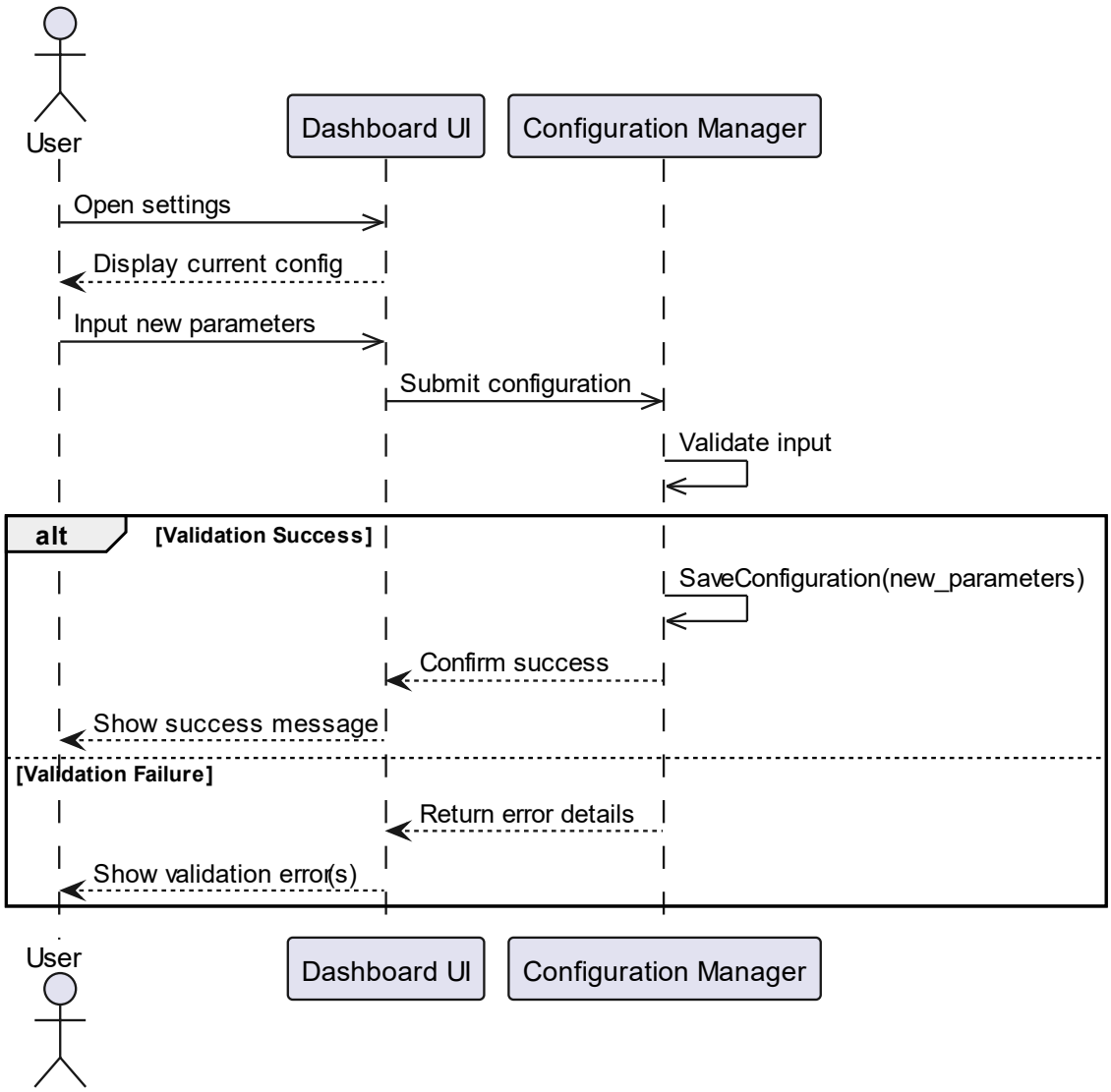


Figure 3.6.3 Sequence Diagram for Configure Parameters

The interaction begins when the User sends an Open settings message to the Dashboard UI, as depicted in Figure 3.6.1.3. The Dashboard UI then Display current config and allows the User to Input new parameters. Once the new parameters are entered, the Dashboard UI sends a Submit configuration message to the Configuration Manager. The Configuration Manager then performs a Validate input operation.

The alt fragment shows two possible outcomes. If the validation is successful ([Validation Success]), the Configuration Manager executes SaveConfiguration(new_parameters) and sends a Confirm success message back to the Dashboard UI, which then Show success message to the User. Conversely, if the validation fails ([Validation Failure]),

the Configuration Manager sends a Return error details message to the Dashboard UI, which then Show validation error(s) to the User.

4. SD4: Partition Dataset per Client

The sequence diagram in Figure 3.6.1.4 outlines the process of partitioning the preprocessed dataset for individual clients, a crucial step for federated learning simulations. This operation is initiated by the System and involves the Dataset Handler and the Partitioner components.

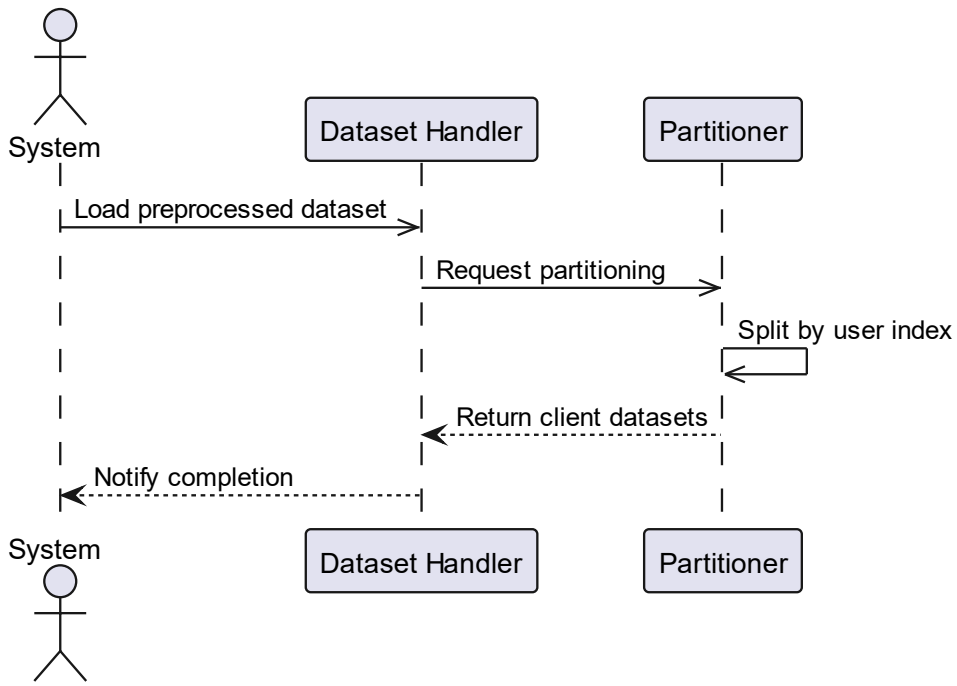


Figure 3.6.4 Sequence Diagram for Partition Dataset per Client

As shown in Figure 3.6.1.4, the process starts when the System sends a Load preprocessed dataset message to the Dataset Handler. Once the dataset is loaded, the dataset handler sends a Request partitioning message to the Partitioner. The Partitioner then performs the Split by user index operation on the dataset. After partitioning, the Partitioner sends a Return client datasets message back to the Dataset Handler. Finally, the Dataset Handler sends a Notify completion message to the System, indicating that the dataset has been successfully partitioned for client use.

5. SD5: Run Federated Learning Simulation

Figure 3.6.1.5 provides a detailed view of the sequence of interactions during a federated learning simulation. The process is initiated by the User via the Dashboard UI and is orchestrated by the FL Coordinator. It involves multiple components including the FL

Server, Client Manager, a representative Client 1 (along with its Local Trainer), and the Metrics Logger. The diagram features nested loop fragments to represent multiple communication rounds and operations across multiple clients.

The simulation begins when the User sends a Start FL Simulation message to the Dashboard UI, which in turn sends a Trigger FL run message to the FL Coordinator (as depicted in Figure 3.6.1.5). The FL Coordinator first sends and initialize global model message to the FL Server, followed by Send initial params. The outer loop fragment signifies the execution of multiple communication rounds ([for each round]). Within each round, the FL Coordinator sends a StartTrainingRound(round_num) message to the FL Server. The FL Server then sends DistributeParameters(round_params) to the client Manager. The inner loop fragment ([for each client]) shows the Client Manager sending a Send parameters & train command message to Client 1. Client 1 then interacts with its Local Trainer to Train locally and Return updated parameters. Client 1 then interacts with its Local Trainer to train locally and return updated parameter.

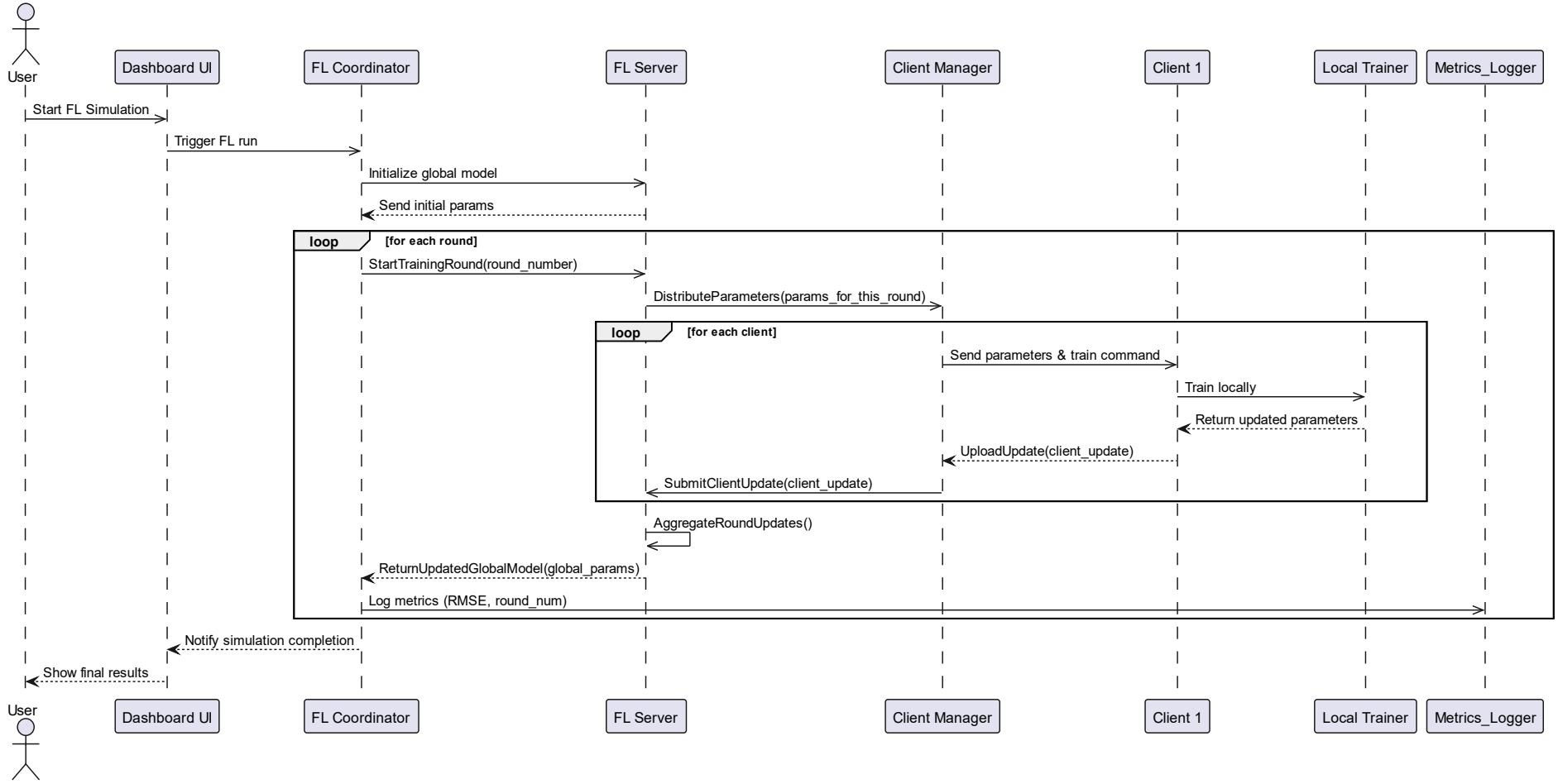


Figure 3.6.5 Sequence Diagram for Run Federated Learning Simulation

6. SD6: Store Performance Metrics

The sequence diagram in Figure 3.6.1.6 illustrates how performance metrics are logged by the system. This process can be triggered by either a centralized training completion or after each federated learning round. It involves the Training Manager (for centralized metrics) or the FL Coordinator (for federated metrics) interacting with the Metrics Logger. The diagram utilizes alt (alternative) fragments to show these distinct scenarios.

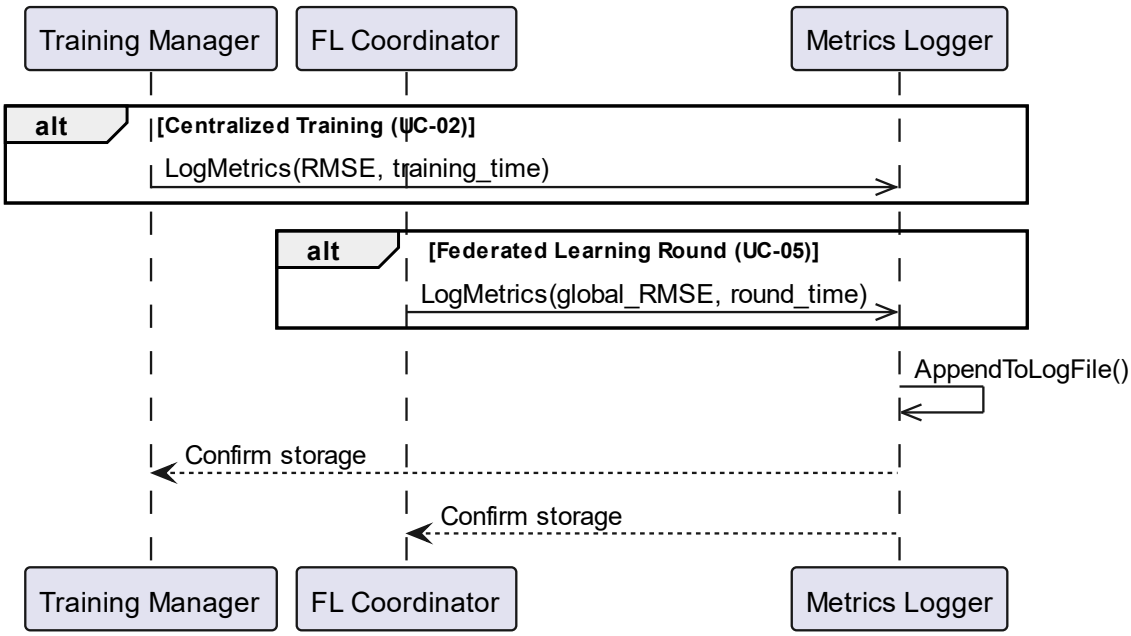


Figure 3.6.6 Sequence Diagram for Store Performance Metrics

As Figure 3.6.1.6 illustrates, two alternative paths exist for logging metrics. In the first alternative fragment, Centralized Training (UC-02), the Training Manager sends a LogMetrics(RMSE, training_time) message to the Metrics Logger. Conversely, in the second alternative fragment, Federated Learning Round (UC-05), the FL Coordinator sends a LogMetrics(global_RMSE, round_time) message to the Metrics Logger. In both scenarios, upon receiving the metrics, the Metrics Logger performs an AppendToLogFile() operation. Subsequently, after successfully appending the data, the Metrics Logger sends a Confirm storage message back to either the Training Manager or the FL Coordinator, depending on the initiator.

7. SD7: View Performance Visualizations

Figure 3.6.1.7 details the sequence of interactions when the User requests to view performance visualizations through the Dashboard UI. This process involves the Dashboard UI, Visualization Engine, and Metrics Logger.

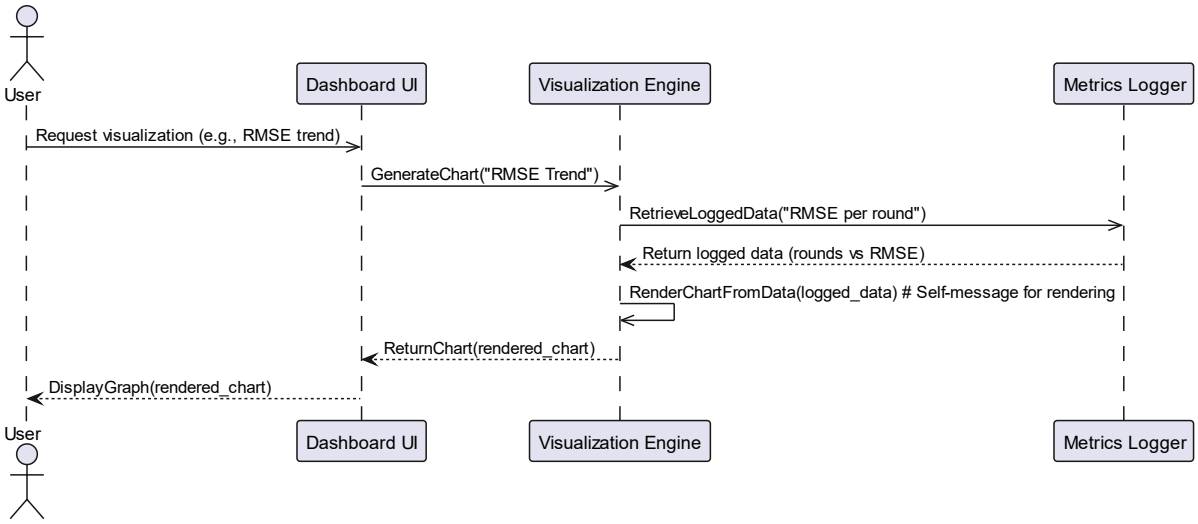


Figure 3.6.7 Sequence Diagram for View Performance Visualizations

The interaction, depicted in Figure 3.6.1.7, starts with the User sending a Request visualization (e.g., RMSE trend) message to the Dashboard UI. The Dashboard UI then sends a GenerateChart("RMSE Trend") message to the Visualization Engine, which subsequently sends a RetrieveLoggedData() message to the Metrics Logger. After processing this request, the Metrics Logger sends a Return logged data (rounds vs RMSE) message back to the Visualization Engine. The Visualization Engine then performs a self-message RenderChartFromData(logged_data) to generate the visualization. Finally, the Visualization Engine sends a ReturnChart(rendered_chart) message to the Dashboard UI, which then displays DisplayGraph(rendered_chart) to the user.

8. SD8: Compare Centralized vs Federated Models

The sequence diagram in Figure 3.6.1.8 illustrates the process initiated by the User to compare the performance of centralized and federated models. This interaction involves the Dashboard UI, the Comparison Engine, and the Metrics Logger.

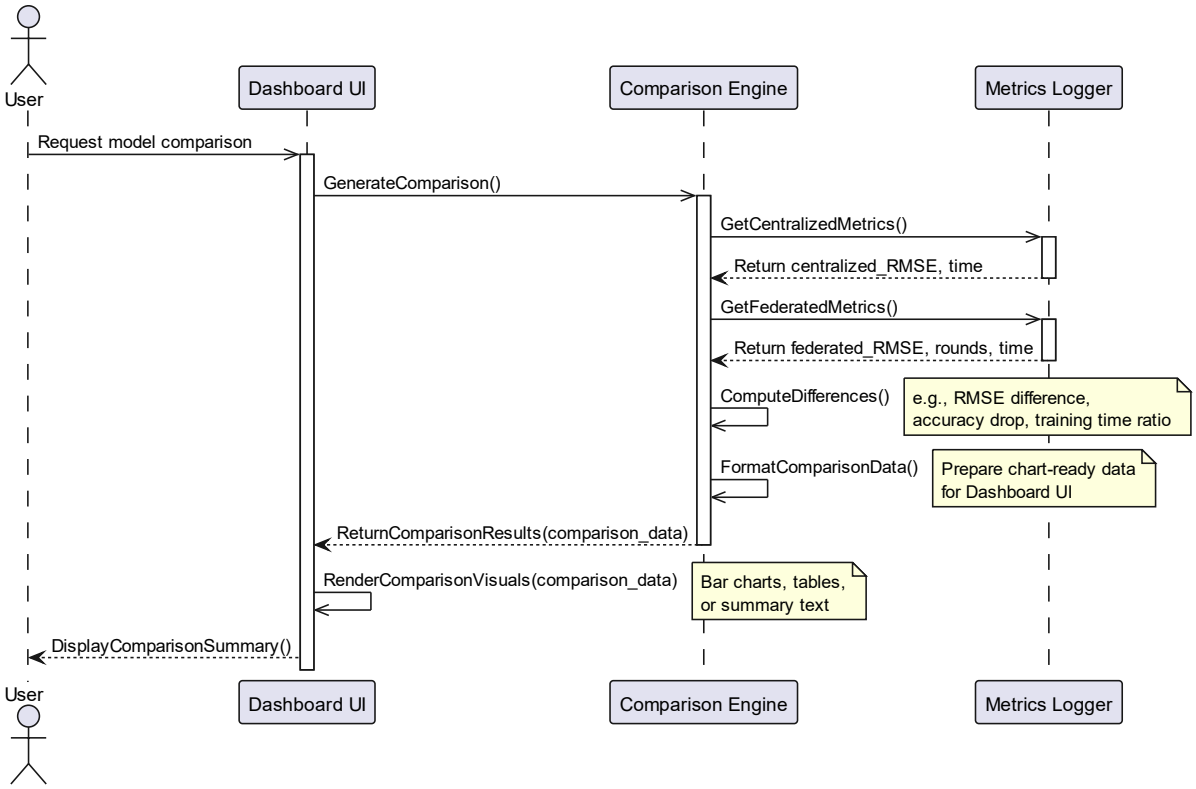


Figure 3.6.8 Sequence Diagram for Compare Centralized vs Federated Models

As depicted in Figure 3.6.1.8, the User starts by sending a Request model comparison message to the Dashboard UI. The Dashboard UI then sends a GenerateComparison() message to the Comparison Engine. The Comparison Engine first sends a GetCentralizedMetrics() message to the Metrics Logger, which responds with Return centralized_RMSE, time. Next, the Comparison Engine sends a GetFederatedMetrics() message to the Metrics Logger, which returns Return federated_RMSE, rounds, time. With both sets of metrics, the Comparison Engine performs ComputeDifferences() (e.g., RMSE difference, accuracy drop, training time ratio) and then FormatComparisonData() to prepare chart-ready data for the Dashboard UI. Once the data is formatted, the Comparison Engine sends a ReturnComparisonResults(comparison_data) message to the Dashboard UI. The Dashboard UI then performs a self-message RenderComparisonVisuals(comparison_data) to generate bar charts, tables, or summary text, and finally DisplayComparisonSummary() to the User.

9. SD9: Export Results

Figure 3.6.1.9 details the sequence of interactions when the User opts to export results, such as visualizations or metric logs. This process involves the Dashboard UI, Export module, Visualization Engine, Metrics Logger, and File Generator.

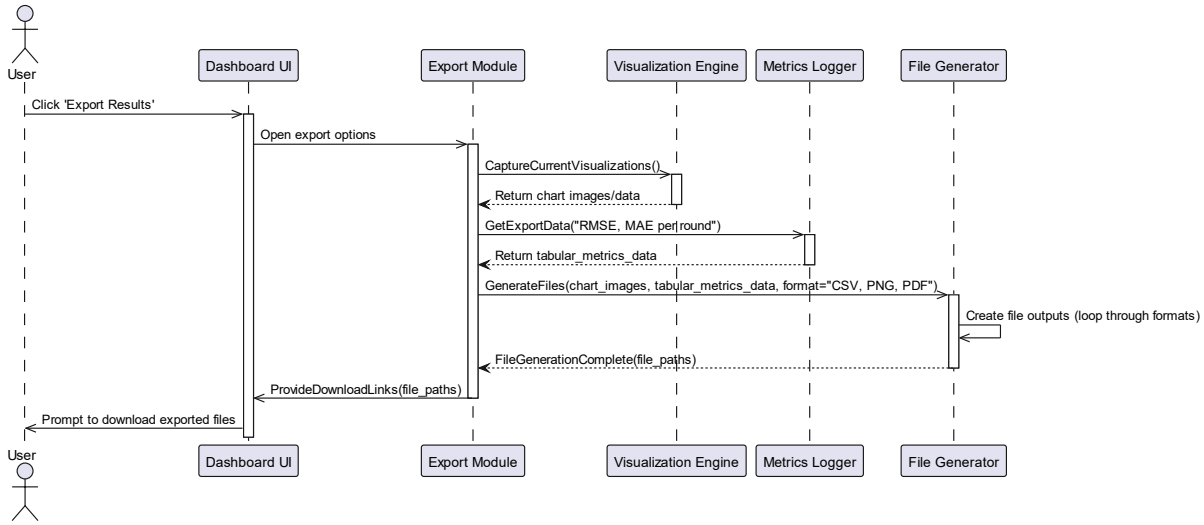


Figure 3.6.9 Sequence Diagram for Export Results

The interaction begins when the User sends a Click 'Export Results' message to the Dashboard UI, as shown in Figure 3.6.1.9. The Dashboard UI then sends an Open export options message to the Export Module. The Export Module first sends a CaptureCurrentVis() message to the Visualization Engine (or the UI itself if it holds the rendered charts), which returns Return chart images/data. Concurrently, the Export module sends GetExportData("RMSE, MAE per round") to the Metrics Logger, which responds with Return tabular_metrics_data. With both visual and tabular data, the Export Module sends a GenerateFiles(chart_images, tabular_metrics_data, format="CSV, PNG, PDF") message to the File Generator. The File Generator then Create file outputs (loop through formats). Upon successful file creation, the File Generator sends a FileGenerationComplete(file_paths) message back to the Export Module. The Export Module then sends ProvideDownloadLinks to the Dashboard UI, which finally Prompt to download exported files to the User.

These sequence diagrams have illustrated the precise, time-ordered interactions between various system components for key dashboard functionalities, clarifying the message flows and collaborations. They effectively map out the dynamic execution of tasks from data handling to results presentation. To complement this view of inter-object communication, the following section introduces Activity Diagrams, which will focus on the broader workflow and control logic within these processes.

3.6.2 Activity Diagrams

Activity Diagrams provide a graphical representation of workflows of stepwise activities and actions with support for choice, iteration, and concurrency. They are used to describe the flow of control from one activity to another, making them ideal for modelling the operational flow of complex processes within the dashboard. This subsection details the key activities involved in major system operations.

Here are the activity diagrams for the critical use cases:

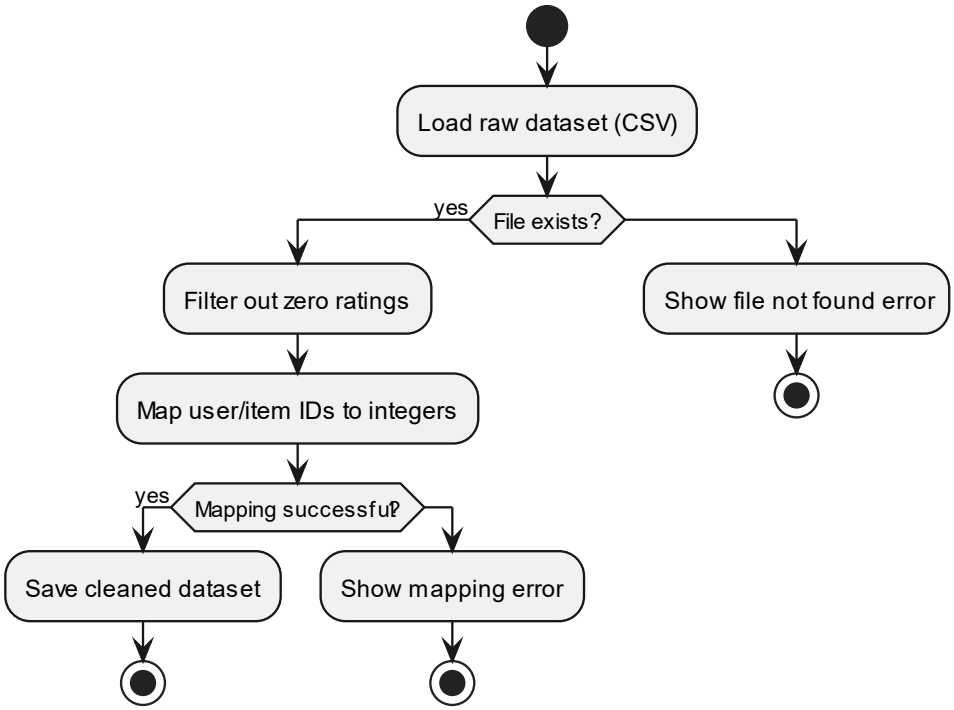


Figure 3.6.10 Activity Diagram for UC-01: Load and preprocess Dataset

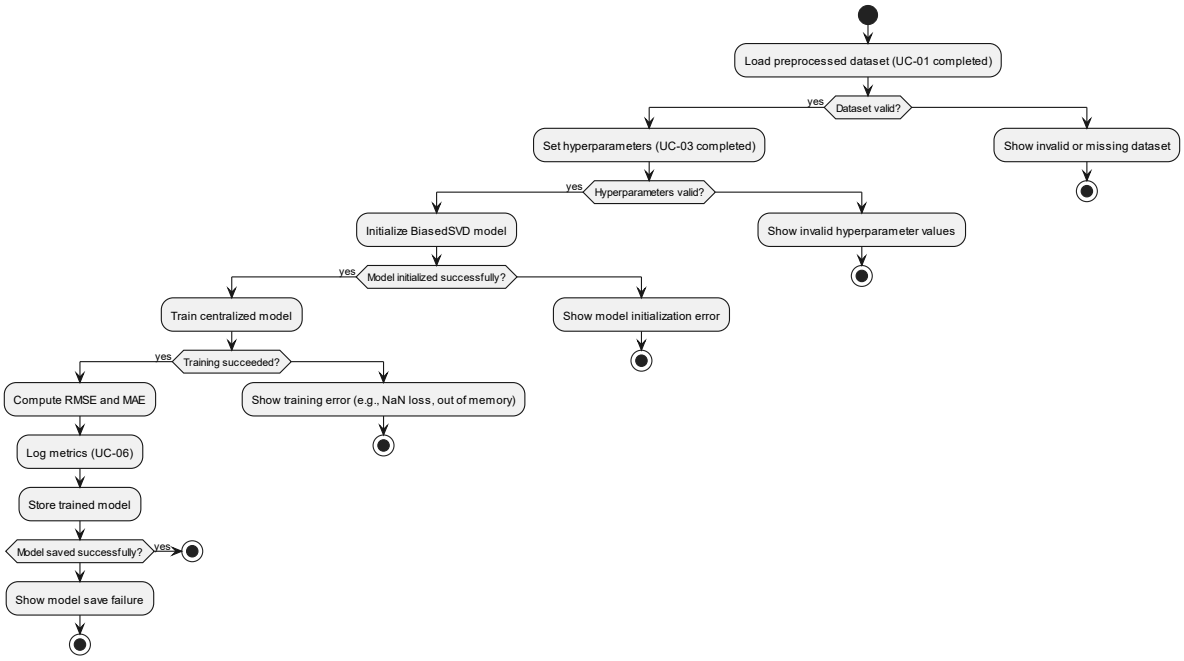


Figure 3.6.11 Activity diagram for UC-02: Train Centralized Model

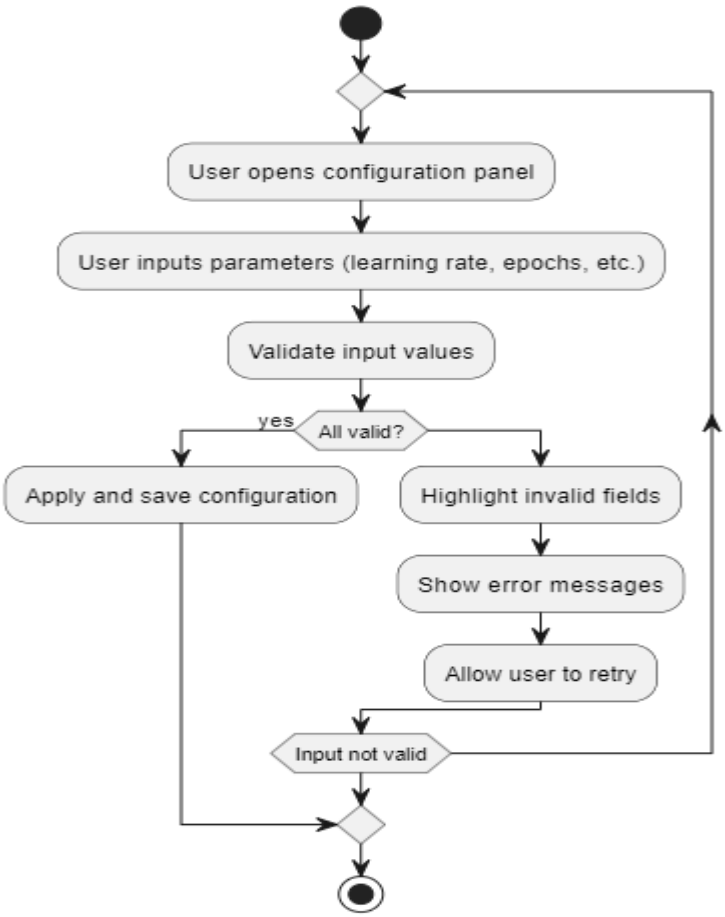


Figure 3.6.12 Activity Diagram for UC-03: Configure Training Parameters

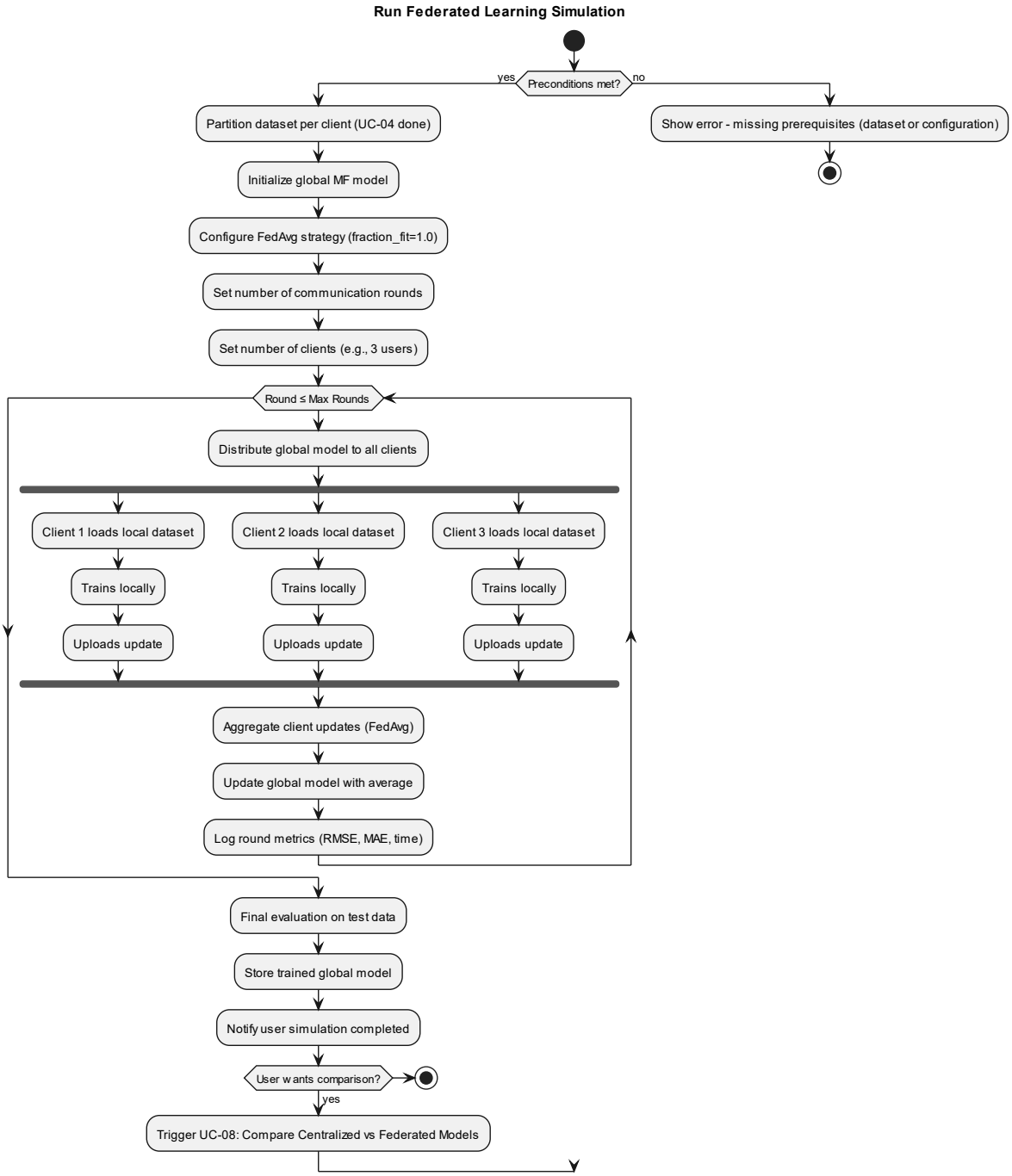


Figure 3.6.13 Activity Diagram for UC-05 Run Federated Learning Simulation

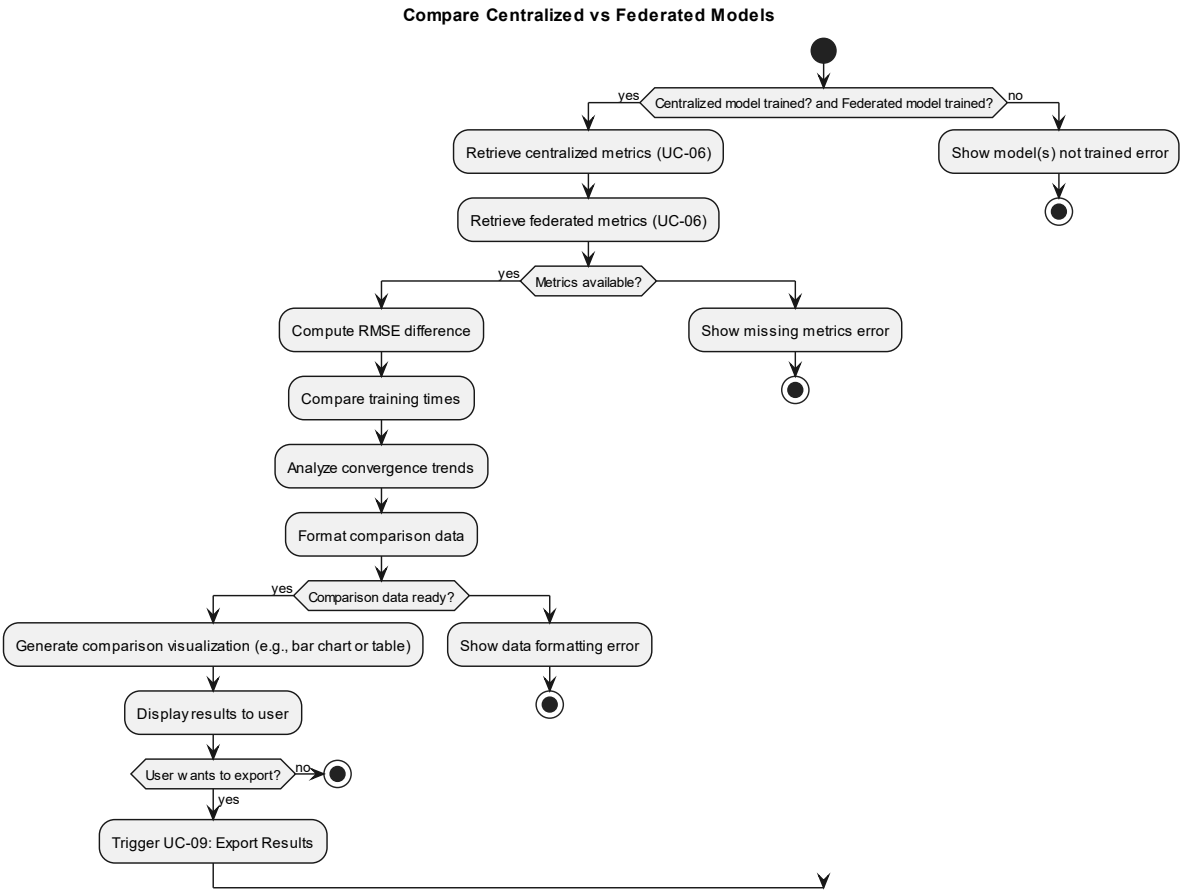


Figure 3.6.14 Activity Diagram for UC-08: Compare Centralized vs Federated Models

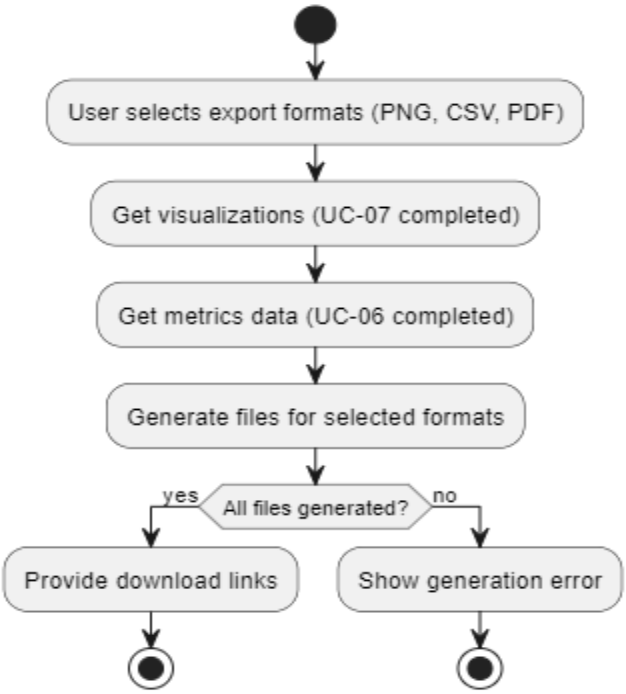


Figure 3.6.15 Activity Diagram for UC-09: Export Results

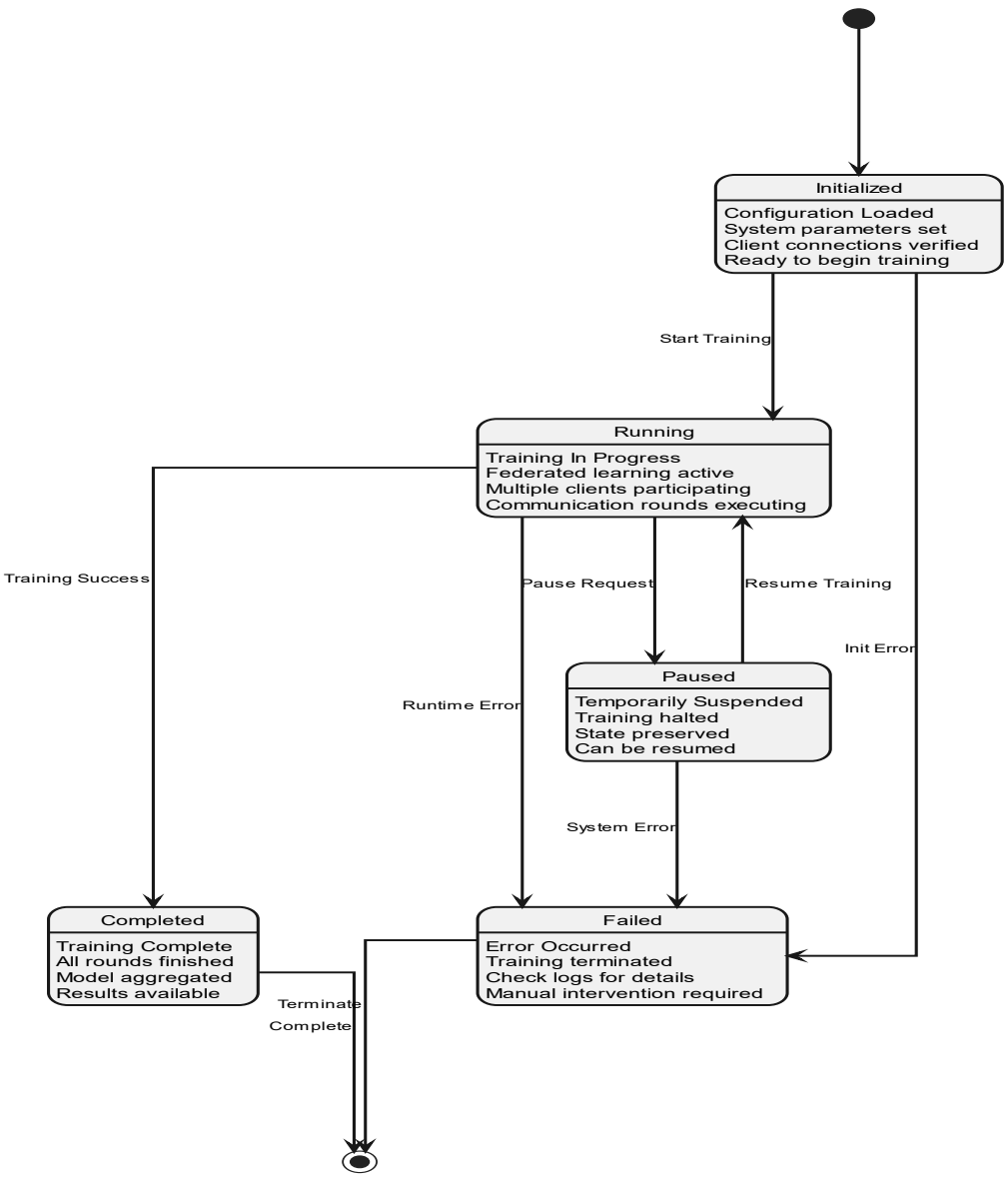
The preceding activity diagrams have offered a visual depiction of the operational workflows for key system processes, illustrating the sequence of activities, decision points, and flow of control from start to completion. They provide a clear understanding of how various tasks within the dashboard are orchestrated to achieve specific outcomes. While activity diagrams focus on the flow of actions, the next section will delve into State Diagrams, which model the behavior of individual objects or components by showing how they transition between different states in response to events.

3.6.3 State Diagrams

State Diagrams are used to model the behavior of a single object, specifying the sequence of states an object goes through during its lifetime in response to events, along with its responses and actions. They are essential for understanding and designing systems with complex, event-driven behavior. The following diagrams illustrate the states and transitions for key components or processes within the federated learning dashboard.

1- Federated Learning Simulation State Diagram.

Figure 3.6.3.1 depicts the various states and transitions for the Federated Learning Simulation process.



The diagram illustrates that the simulation begins in an Initialized state, where configuration is loaded, system parameters are set, client connections are verified, and it's ready to begin training. Upon a Start Training event, it transitions to the Running state. While Running, federated learning is active, multiple clients participate, and communication rounds execute; this state is concurrently monitored for client health, progress, and performance metrics. From the Running state, a Pause Request (by User/System) can move it to a Paused state, where training is temporarily halted but the state is preserved for resumption. A Resume Training event (User resumes or Init Error/Config Error is resolved) transitions it back to Running.

If all rounds complete successfully (Training Success), the simulation moves from Running to the Completed state, where training is finished, all rounds are treated, the model is

aggregated, and results are available. From Completed, a Terminate event leads to the final state. However, if a Runtime Error occurs during the Running or Paused (System Error/Resource State issue) states, or if there's an initialization/configuration error from the Initialized state, the simulation transitions to a Failed state, indicating an error occurred, training was terminated, and manual intervention might be required. Common failure causes include network issues, client resource exhaustion, data validation errors, or configuration conflicts.

2- Centralized Training State Diagram

Figure 3.6.3.2 illustrates the states and transitions involved in the Centralized Training Process within the dashboard.

The Centralized Training Process begins in an Idle state, where the system is waiting for a training task. When the userStartsCentralizedTraining event occurs, the process transitions to the Configuring state, during which the user sets hyperparameters and selects the dataset. Upon configurationComplete, it moves to LoadingData to get the preprocessed dataset. Once dataLoaded, it enters the InitializingModel state to set up the model structure with the chosen hyperparameters.

With modelInitialized, the process transitions to the Training state, where model training iterations occur. After trainingEpochsComplete, it moves to Evaluating to compute metrics like RMSE and MAE. Following evaluationComplete, it enters the LoggingMetrics state to record these computed metrics. Once metricsLogged, it transitions to StoringModel to save the trained model artifact. Finally, after modelStored, the process reaches the Completed state, signifying all steps have finished successfully.

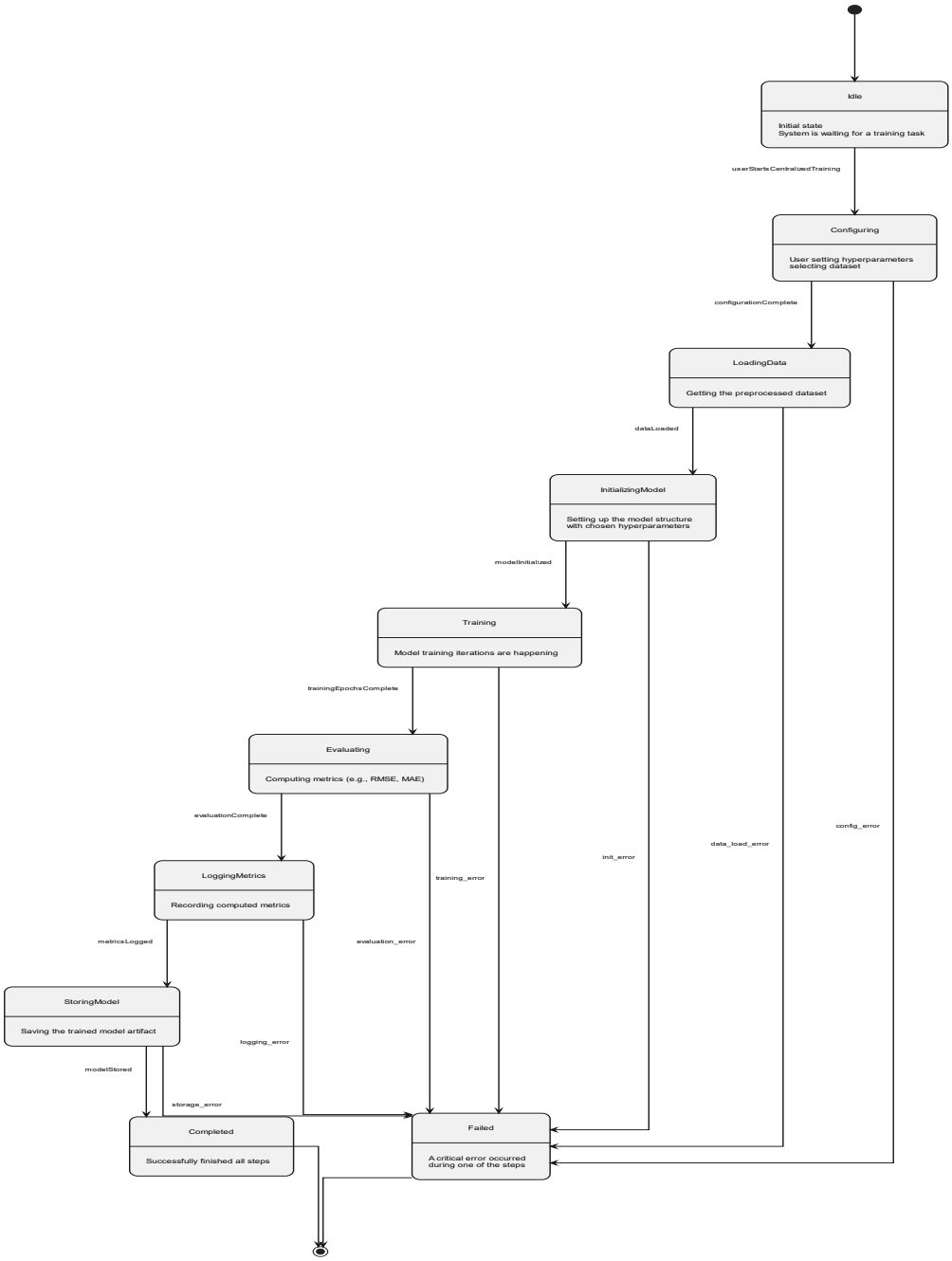


Figure 3.6.16 State Diagram for Centralized Training Process

At any point from configuring, LoadingData, InintalizingModel, Training, Evaluating, LoggingMetrics, or StoringModel, if an error occurs (e.g., errorOccurred(config_error) or errorOccurred(data_load_error)) the process transitions to a Failed state, indicating a critical error occurred during one of the steps.

3- Client State Diagram (Within One FL Round)

Figure 3.6.3.3 models the lifecycle of a client participating within a single round of Federated Learning.

The client begins in an IdleInRound state (AwaitingParameters), waiting for the server to send the global model for the current round. Upon the receiveGlobalParameters event, it transitions to ReceivingParameters, where it actively receives the global model and training instructions.

If a parameterError occurs, the client returns to the IdleInRound state; otherwise, if parametersReceived successfully, it transitions to ParametersReceived (also known as ReadyToTrain). From ParametersReceived, a startLocalTraining event moves the client to TrainingLocalModel to perform local training on its dataset. Should a trainingError arise, it shifts to LocalTrainingFailed.

If training completes successfully (trainingComplete), it transitions to LocalTrainingComplete (also known as PreparingUpdate), where it prepares the local model update. An initiateUpload event then moves the client to UploadingUpdate to send the local model update to the server. If an uploadError occurs, it transitions to UpdateUploadFailed. If the uploadSuccessful event occurs, the client reaches the UpdateSent state (also known as RoundParticipationComplete), indicating its local model update was successfully sent and its participation in this round is complete. Both LocalTrainingFailed and UpdateUploadFailed states lead to a final error state for this client's participation in the round, as does successful completion in UpdateSent.

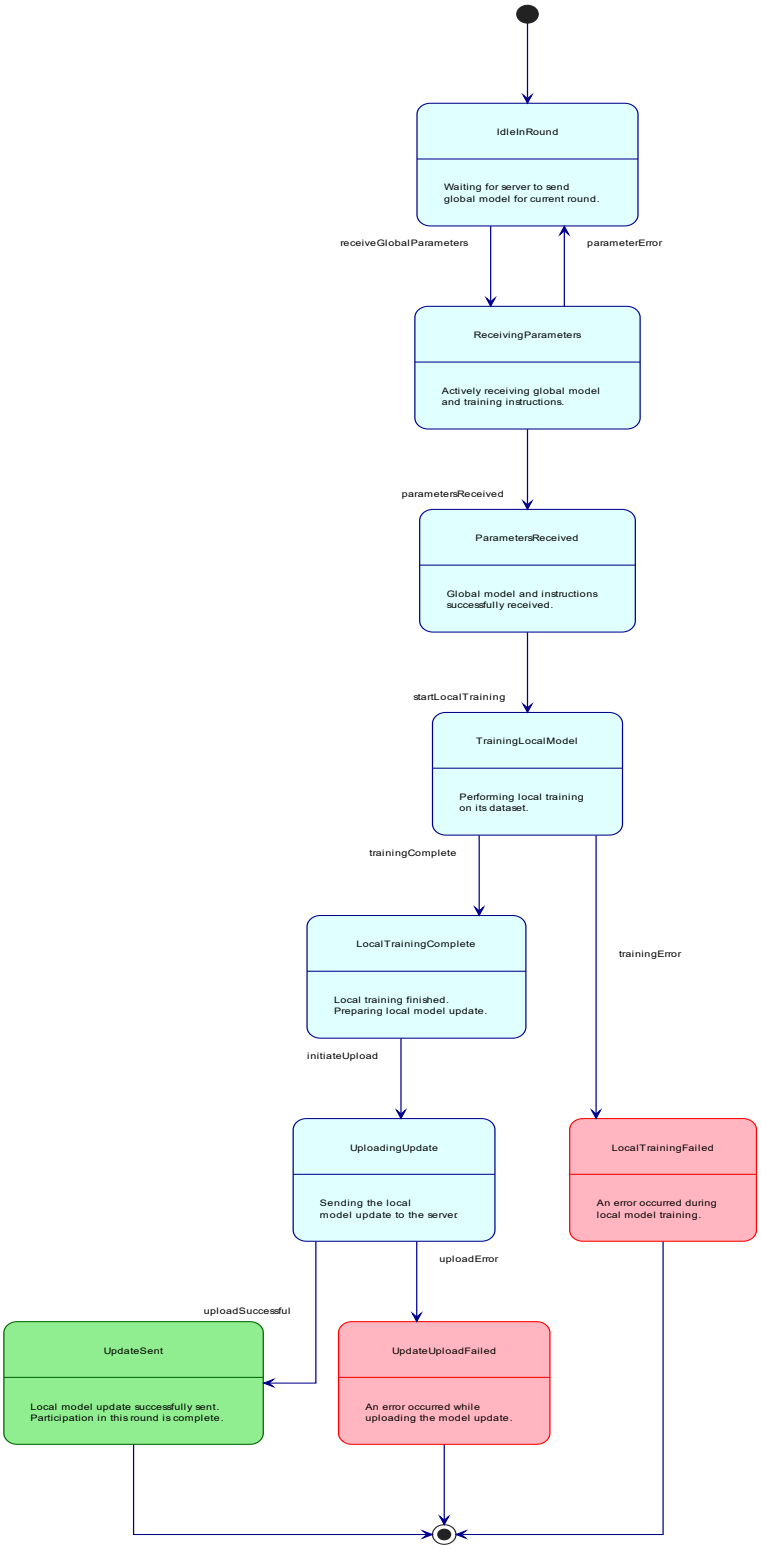


Figure 3.6.17 State Diagram for a Client within one FL Round

4- Dashboard View State Diagram

Figure 3.6.3.4 presents the state diagram for the overall Dashboard View, illustrating how the user interface transitions between different views and operational states based on user interactions and system events.

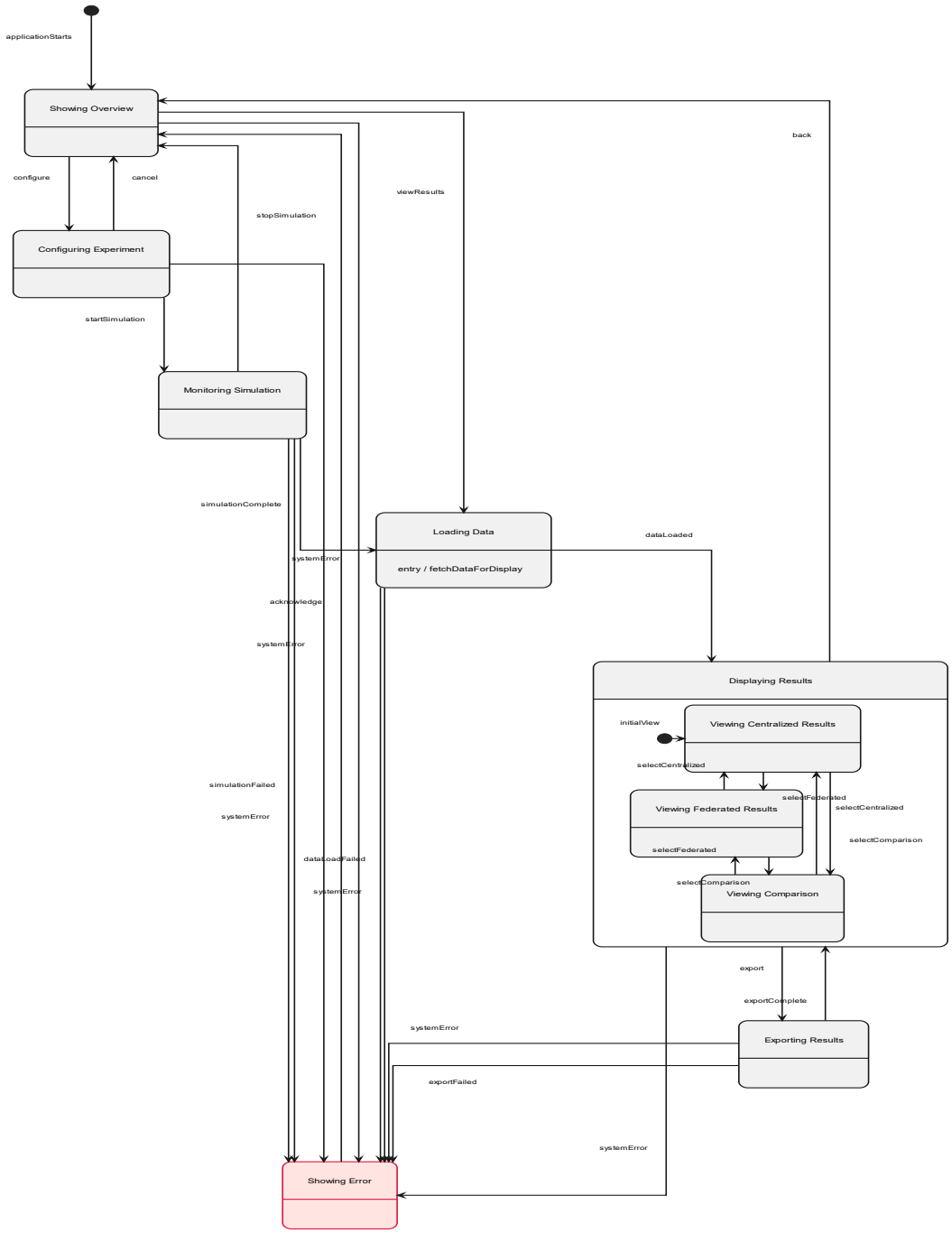


Figure 3.6.18 State Diagram for Dashboard View

The dashboard starts in the Showing Overview state upon applicationStarts. From here, a configure event transitions it to Configuring Experiment, and a cancel event returns it to Showing Overview. A startSimulation event from Configuring Experiment moves it to Monitoring Simulation. If the simulationComplete event occurs, or if the user triggers viewResults from Showing Overview, the dashboard enters the Loading Data state (entry action: fetchDataForDisplay). If dataLoaded successfully, it transitions to the Displaying Results superstate. A stopSimulation event from Monitoring Simulation or a clearResults event from Displaying Results can return the view to Showing Overview.

Within the Displaying Results superstate, the initial view is Viewing Centralized Results. The user can then navigate between selectFederated to Viewing Federated Results, selectComparison to Viewing Comparison, or selectCentralized to return to centralized results. From any of these sub-states within Displaying Results, an export event transitions to the Exporting Results state. Once exportComplete, it returns to the previous viewing sub-state. A back navigation from Displaying Results leads back to Showing Overview.

Throughout these states, various error events such as exportFailed, simulationFailed, dataLoadFailed can transition the dashboard to a general Showing Error state. An acknowledge event from showing Error typically returns the view to showing overview.

3.7 Structural Modelling

Following the exploration of the system's dynamic behavior through sequence, activity, and state diagrams, this section shifts focus to its static structure. To provide a comprehensive architectural understanding, we will now detail the system's components and their inherent relationships using established modelling techniques.

3.7.1 Class Identification

The following tables list the major classes anticipated for the system, categorized into Entity, Boundary, and Control classes. This categorization helps in understanding the role of each class within the system's architecture and bridges the gap from requirements to a more concrete design structure.

1- **Entity Classes:** Classes that represent the persistent information and domain-specific data managed by the system.

Class Name	Description
Dataset	Represents the collection of user-item interactions (e.g., Goodreads data), including raw and preprocessed forms.
Rating	Represents a single user-item interaction, typically a rating, with attributes like user ID, item ID, and rating value.
Book	Represents a book entity (could be extended with metadata beyond just an ID if needed).
LocalUserData	Represents a user whose data is part of the system, particularly their local data partition in federated simulations.
RecommenderModel	Abstract or concrete class (e.g., BiasedSVDModel) encapsulating the state and structure of the recommendation model being trained.
PerformanceMetric	Represents a single performance measurement (e.g., RMSE, MAE, training time) with its value and associated context (round, client).
SimulationRun	Represents a single execution of a training process (centralized or federated) with its configuration and collected results.
Configuration	Stores the settings and parameters for a model or a simulation run (e.g., learning rate, number of clients, federated strategy).

2- Control Classes: Classes encapsulate the application's business logic, manage workflows, and coordinate interactions between Entity and Boundary classes.

Class Name	Description
SimulationManager	Manages the overall lifecycle of simulations, initiates training, configures parameters, collects results, and coordinates other controllers.
DataHandler	Responsible for loading, preprocessing (filtering, ID mapping), and partitioning the dataset for clients in federated simulations.
TrainingManager	Orchestrates the training process, delegating to specific trainer classes for centralized or federated approaches.
CentralizedTrainer	Implements the training logic for the centralized recommendation model (e.g., the baseline SVD model).
FederatedLearningServer	Central coordinator of the federated learning process (akin to Flower's Server). Manages training rounds, selects clients (via strategy), distributes global model parameters, and directs aggregation of client updates using a FederationStrategy.
FederationStrategy	Implements the specific algorithm for aggregating model updates from clients (e.g., FedAvg), used by the FederatedLearningServer.
FederatedLearningClient	Represents a participant in the FL process (akin to Flower's Client). Manages its local data partition, holds/trains its local RecommenderModel instance, and communicates model parameters/updates with the FederatedLearningServer.
MetricsCalculator	Calculates performance metrics (RMSE, MAE, etc.) for both centralized and federated models, covering global and local client evaluations.
MetricsLogger	Centralizes the recording and storage of PerformanceMetric objects generated throughout various stages of simulation runs.

ConfigurationManager	Handles the loading, saving, and validation of Configuration objects based on user input from the dashboard interface.
VisualizationEngine	Generates data structures and prepares visualizations (charts, graphs) based on processed metrics and simulation data.
ExportManager	Manages the logic for generating file content (e.g., image bytes for charts, CSV data for logs) for export.

3- **Boundary Classes:** Classes that facilitate communication between the system and its external actors, primarily the User through the graphical user interface.

Class Name	Description
DashboardUI	The main graphical interface through which the User interacts with the system. Presents controls, displays information, and visualizations.
ConfigurationView	A specific UI component (part of DashboardUI) allowing the user to input and modify experiment configurations and hyperparameters.
ResultsView	A specific UI component (part of DashboardUI) responsible for displaying charts, graphs, and tables of simulation results.
ExportView	A UI component (part of DashboardUI) allowing the user to select formats and initiate the export of reports or visualizations.
SystemNotificationArea	A UI component (part of DashboardUI) for displaying status messages, progress updates, or error notifications to the user.

3.7.2 Class Diagram

Following the identification of key system classes in Section 3.7.1, this section presents the UML Class Diagram. This diagram, shown in Figure 3.7.1, visually models the static structure of the "Dashboard for Evaluating Federated Learning in Book Recommender Systems." It illustrates the principal classes, their core attributes and methods, and the significant relationships that define their interactions and dependencies. The class diagram serves as an architectural blueprint, clarifying the responsibilities of each component and how they collectively form the system.

The classes within the diagram are visually distinguished by color to represent their architectural roles: Entity classes (blue) manage the system's data and domain concepts; Control classes (orange) encapsulate the business logic and orchestrate workflows; and Boundary classes (green) facilitate user interaction through the graphical interface.

The diagram employs various lines and arrows to represent the different types of relationships between classes, indicating how they are connected, influence each other, and collaborate to fulfill the system's objectives. Some of the key relationships depicted are:

- **Inheritance (Specialization/Implementation):** This "is-a-kind-of" link enables abstraction and polymorphism. A general class or interface defines a contract that specialized classes then fulfill. For instance, BiasedSVDModel is a specific RecommenderModel, and FedAvgStrategy implements FederationStrategy, promoting flexibility for future additions.

- **Aggregation/Composition (Containment/Ownership):** This "has-a" or "part-of" relationship shows that instances of one class are composed of or own instances of others. DashboardUI is composed of ConfigurationView and ResultsView. Similarly, a SimulationRun contains its Configuration and PerformanceMetrics.

- **Association (Collaboration/Usage):** This relationship signifies ongoing collaboration, where one class holds a reference to another. SimulationManager is associated with DataHandler and TrainingManager. In the federated context, FederatedLearningServer associates with multiple FederatedLearningClient instances.

- **Dependency (Temporary Usage):** This represents a transient reliance, often through method parameters or local instantiation, without a persistent link. ResultsView depends on VisualizationEngine for data formatting. ConfigurationView relies on ConfigurationManager for saving/loading. CentralizedTrainer and FederatedLearningClient depend on MetricsCalculator for performance scores.

The below Class Diagram in the figure serves as a critical architectural guide for the development of the dashboard, clarifying the responsibilities of each component and the static relationships between them, thereby promoting a modular and maintainable system design.

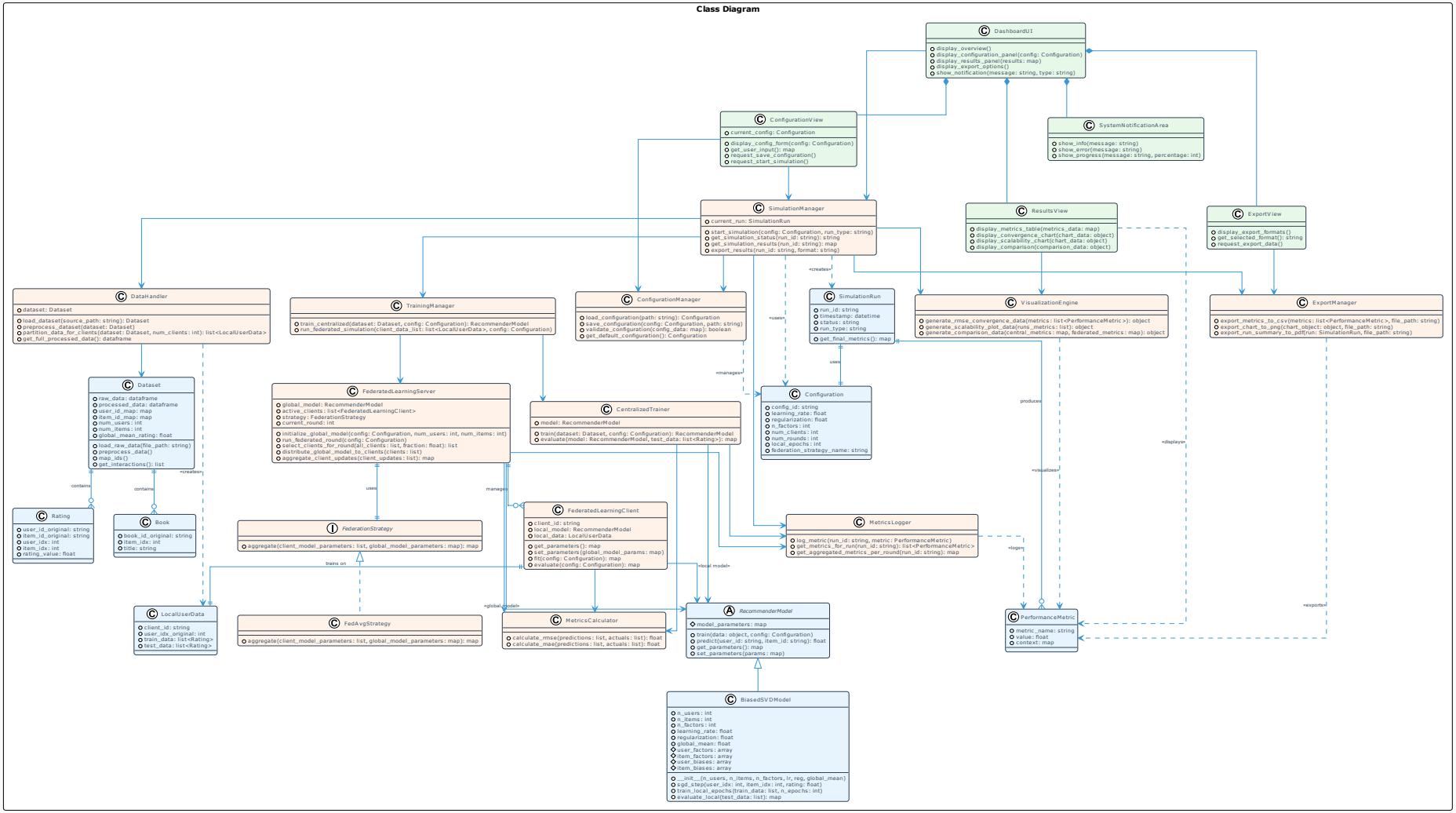


Figure 3.7.1 Class Diagram

3.7.3 ER Diagram

While the Class Diagram in Section 3.7.2 details the object-oriented structure and behavioral aspects of the system, this section presents the Entity-Relationship Diagram (ERD). The ERD offers a data-centric perspective, focusing on the fundamental data entities the system needs to manage for persistence, tracking, and evaluation, along with their key attributes and inter-relationships.

The ERD (Figure 3.7.3.1 below) illustrates these persisted entities and their connections.

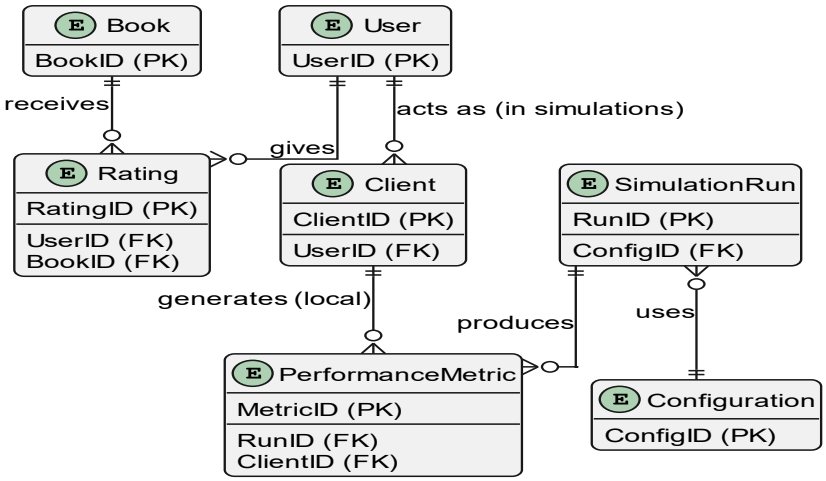


Figure 3.7.2 System Entity-Relationship Diagram

The ERD highlights the following core data entities necessary for the dashboard's functionality, namely:

1- The User entity represents an individual from the original Goodreads dataset, serving as the foundational element for recommendations. Its primary purpose is to uniquely identify each user and link them to their interactions, with the UserID acting as its key attribute.

2- The Book entity represents a specific book within the dataset, crucial for identifying and connecting books to user interactions. Its purpose is to uniquely identify each book, and its key attribute is BookID, potentially supplemented by its Title.

3- The Rating entity acts as an associative entity, capturing the explicit feedback users provide for books. Its purpose is to store these ratings, which are the primary input for recommender models, and its key attributes include a RatingID or a composite of UserID and BookID, along with UserID, BookID, RatingValue, and Timestamp.

4- The SimulationRun entity tracks each complete execution of a training process, whether centralized or federated. Its purpose is to record the context of each experimental run, including its type, status, and timing, and to link these runs to their specific configurations and results, with RunID as its primary key and ConfigID as a foreign key, along with TimestampStart, TimestampEnd, RunType, and Status.

5- The Configuration entity stores the precise set of parameters and model choices used for a particular SimulationRun. Its purpose is to ensure reproducibility and enable analysis of how different settings affect performance, with ConfigID as its primary key, and attributes like ModelAlgorithmName, common hyperparameters, federated-specific parameters, and a flexible ModelHyperparameters field for algorithm-specific settings.

6- The PerformanceMetric entity records specific quantitative outcomes of simulations. Its purpose is to store measurements like RMSE, MAE, or training time per round for analysis and visualization, with MetricID as its primary key, and foreign keys to RunID and potentially ClientID, along with MetricName, Value, and Context_Round.

7- The Client entity represents a distinct participant in a federated learning SimulationRun. Its purpose in federated simulations is to identify individual participants, link them to their original UserID, and associate them with any client-specific performance metrics or contributions, with ClientID as its primary key and UserID as a foreign key.

Chapter 4: Design

4.1 Introduction

The design phase for the Dashboard is pivotal in translating the identified requirements into a functional, insightful, and user-friendly system. This chapter delves into the architectural framework, interfaces, and data flow necessary to meet the specific needs outlined in previous chapters, particularly concerning the simulation, evaluation, and visualization of federated learning processes. The design ensures that the dashboard is not only robust in its analytical capabilities and maintainable in its structure but also accessible, empowering users with varying levels of technical expertise to explore and understand the nuances of federated learning.

System design for this dashboard involves defining the architecture, interfaces, and data flow to meet the specific requirements of orchestrating federated learning experiments and presenting their outcomes. The proposed system will utilize a structured architectural approach, allowing for a clear separation of concerns and facilitating the seamless integration of simulation components and visualization tools. This approach supports scalability for evaluating diverse federated scenarios (e.g., varying client numbers, communication rounds) and flexibility for future enhancements, such as incorporating new federated aggregation algorithms or advanced performance metrics. Instead of managing raw user datasets directly in a traditional database sense (which would contradict FL principles), the system will focus on managing experiment configurations, aggregated model updates/parameters, and performance logs, potentially using lightweight data structures or fit-for-purpose logging mechanisms.

Key elements of the system design include the relationships between core modules for simulation control, metrics aggregation, and user interface presentation, the overall architectural style guiding their interaction, and the potential integration of third-party libraries (e.g., for federated learning simulation like Flower, or for charting). To articulate this, an enhanced class diagram, building upon the initial model from Chapter 4 and incorporating appropriate packaging to reflect the architectural structure, will be detailed. Furthermore, a deployment diagram will be presented to illustrate the physical or logical deployment of the system components, considering its typical use as a research or analytical tool. This comprehensive approach aims to create a cohesive system that effectively bridges experiment execution, data analysis, and user-driven exploration.

By focusing on these design principles, this project aims to address the challenges in understanding and comparing decentralized learning paradigms. It seeks to empower researchers and developers to systematically analyze the effectiveness, scalability, and performance dynamics of various federated learning approaches within the specific domain of book recommender systems, all through an intuitive and interactive visual platform.

4.2 Overview of Software Architectures

The design of the "Dashboard for Evaluating Federated Learning in Book Recommender Systems" requires an architectural approach that balances robust analytical capabilities, maintainability, and accessibility for a diverse user base, including technically proficient Researchers and less-technical Stakeholders. This section explores three potential software architectures, evaluating their strengths and weaknesses in the context of the project's objectives and non-functional requirements. These architectures are presented in descending order of suitability.

4.2.1 Architecture Option 1: Client-Server Web Application (Monolithic Backend)

This architecture proposes a two-tiered system consisting of a web-based frontend and a Python-based backend server. Figure 4.2.1 below represents a generic structure of the architecture.

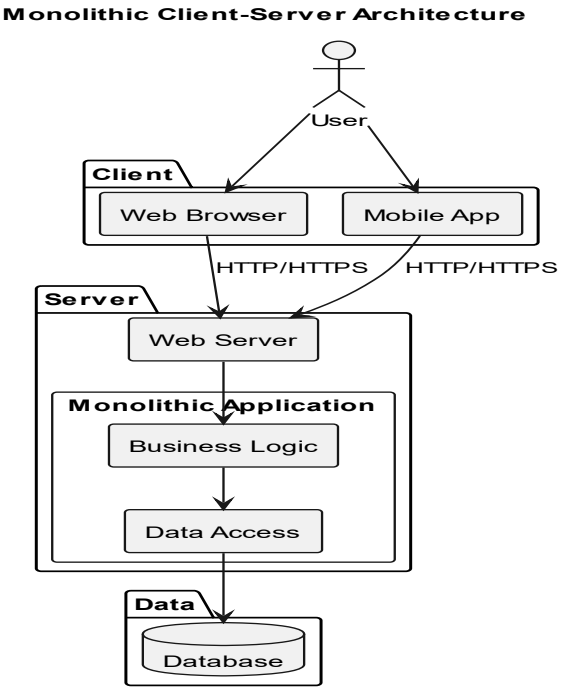


Figure 4.2.1 Architecture Design Option 1

As inferred from the diagram this model fundamentally separates the system's responsibilities into two main components: a client-side frontend, which is concerned with presentation and user interaction, and a server-side backend, which handles all business logic, data processing, and storage. The two tiers communicate over a network, allowing the computationally intensive tasks to be centralized on the server while providing a responsive user experience in the client's web browser.

This architecture is highly suitable for the project because it can deliver distinct user experiences; non-technical users access an intuitive browser-based dashboard with no local installation, while technical users can be given advanced UI controls and API access for deeper analysis. The clear separation between the frontend's presentation logic and the backend's complex simulation logic enhances maintainability (NFR-07) and ensures the UI remains responsive (NFR-09) by offloading computationally intensive tasks to the server. While monolithic, the backend's layered structure promotes modularity, and Python's robust ecosystem for data science and federated learning is fully leveraged.

However, this approach introduces potential challenges. It has a higher initial development complexity than a desktop application, requiring skills in both frontend and backend technologies. State management and communication between the two tiers via the API must also be carefully designed to ensure reliability.

4.2.2 Architecture Option 2: Layered Monolithic Desktop Application

This architecture envisions a single, cohesive desktop application. Figure 4.2.2 represents a generic structure of the proposed architecture.

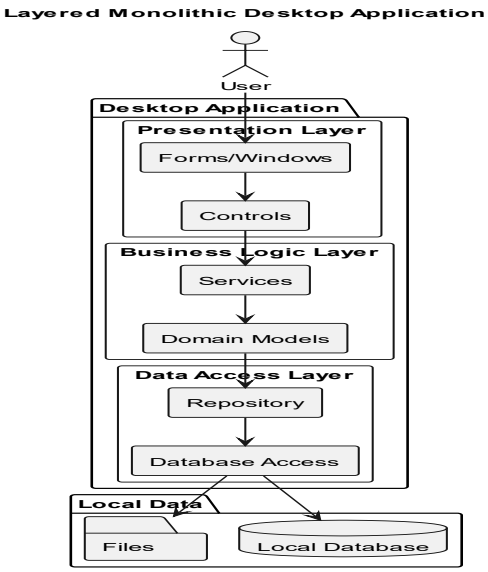


Figure 4.2.2 Architecture Design option 2

As shown in the image, the architecture is characterized by a single, cohesive desktop application. It typically comprises a Presentation Layer for user interaction, a Business Logic Layer for core functionalities, and a Data Access Layer for data management, all interacting with local data storage.

This architecture is moderately suitable and presents a strong alternative, particularly if development speed and simplicity are paramount. The entire system can be built within a unified Python environment, simplifying the development workflow and facilitating seamless integration of data science libraries and the Flower framework. For the Researcher persona, who is likely comfortable running Python applications, this architecture is highly effective. However, catering to non-technical users becomes more challenging with this approach. While frameworks like Streamlit or Dash can create multi-page applications with different views, designing a single UI that is simultaneously simple enough for a non-technical user and powerful enough for a technical user without compromising usability for either group is difficult. Furthermore, accessibility for non-technical users might be a concern if they are not comfortable running local Python applications; this would necessitate deploying the dash or streamlit application on an accessible server. UI responsiveness (NFR-09) during long simulations would need careful management using threading or asynchronous operations within the Python application.

The primary challenge lies in achieving an optimal user experience for both distinct user personas within a single UI framework. Ensuring the application remains responsive during intensive computations requires explicit multi-threading/asynchronous programming, which can add complexity. Deployment for easy stakeholder access might also require additional server setup.

4.2.3 Architecture Option 3: Layered Architecture with Modular Microservices (and a Web Frontend)

This architecture decomposes backend functionality into multiple, independently deployable microservices. The diagram 4.2.3 depicts a generic representation of the architecture.

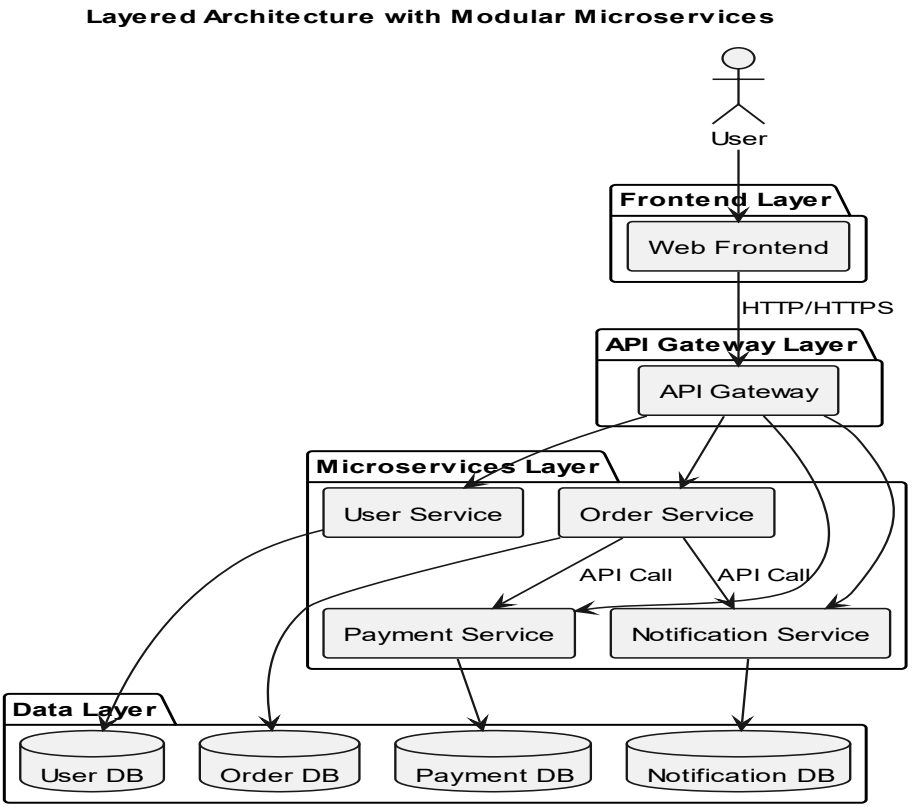


Figure 4.2.3 Architecture Design option 3

In this architecture, a Frontend Web Application handles user interaction, while an API Gateway serves as the single entry point for all incoming requests. The system is composed of multiple Backend Microservices, including a Simulation Orchestrator, Data Processing, Model Training, and Metrics & Results services. These microservices communicate through well-defined APIs or an event bus, depending on the use case. Each service may use its own dedicated datastore or share data storage with others, depending on its responsibilities and data access patterns.

This architecture is considered less suitable for the current project scope, despite its powerful advantages in other contexts. Its strengths lie in high scalability of individual components, strong fault isolation, and technological flexibility, which are typically critical for large-scale, production-grade applications. For this evaluation dashboard, the benefits of such extensive decomposition are likely outweighed by the significant increase in operational and development complexity. Managing inter-service communication, distributed data, service discovery, and deployment of multiple services introduces substantial overhead. While the frontend benefits for user personas are similar to Architecture Option 1, the added backend complexity does not proportionally enhance the core project objectives of simulation, evaluation, and insight presentation for a research-oriented tool. The project's NFRs, such as

maintainability (NFR-07) and scalability (NFR-06), can be adequately addressed by the more straightforward monolithic backend of Architecture Option 1, especially when that monolith is well-structured internally.

The primary challenges include a significant increase in development and operational complexity due to managing a distributed system. There are also challenges in debugging across services, ensuring data consistency, and a potential for increased resource consumption. This approach is likely an over-architecture for the defined project goals.

The detailed evaluation of these architectural options provides a foundation for selecting the most effective approach to realize the dashboard's functionalities while adhering to its quality attributes. The subsequent section will delve into the rationale for the chosen architecture and elaborate on its specific design components.

4.3 Selected Software Architecture

The selected software architecture for the dashboard is the Client-Server Web Application with a Monolithic Backend, as detailed in Section 4.2.1. This choice best balances the project's need for tailored user experiences for both technical and non-technical users, robust federated learning simulation capabilities, and manageable development complexity.

This architecture distinctly separates the frontend user interface from the backend Python application that handles all core logic. The frontend will be a web application providing an accessible and intuitive experience, particularly for non-technical users to gain insights. The backend will manage data processing, the execution of centralized and federated learning simulations using Flower, metrics calculation, and will expose a well-defined API for frontend communication. While the backend is deployed as a single unit (monolithic), it will be internally structured into layers (API, Service/Control, Domain/FL Core, Data Access) to ensure modularity and maintainability.

The primary components and their interactions are visualized in the component diagram below (Figure 4.3.1). This architectural framework enables independent evolution of the user interface and the core simulation engine, ensuring the system is both functional and maintainable. The subsequent section, 4.4 Design Model, will further detail the refined class structures and relationships within this chosen architecture.

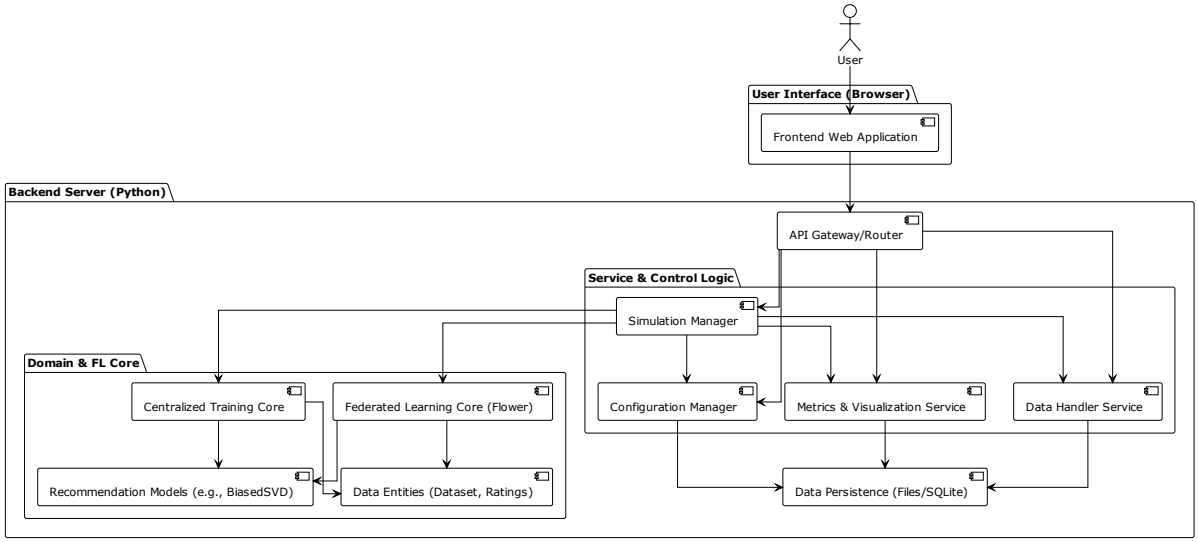


Figure 4.3.1 Architecture Design option 3

4.4 Design Model

Following the the selection of the the Client-Server Web Application architecture, this section details the refined design model of the system. The primary artifact of this model is an enhanced class diagram (Figure 4.4.1), which builds upon the initial class identification (Section 4.7.1) and incorporates principles of SOLID design, packaging for modularity, and relevant design patterns to ensure a robust, maintainable and extensible system. This detailed model will serve as the blueprint for the implementation phase.

The design model illustrated in Figure 4.4.1 is organized into three primary layers, further subdivided into packages to enhance modularity and clarity.

1- The Core Domain Layer encapsulates the fundamental data structures and entities of the system. It is composed of several packages: core.model contains the IRecommenderModel interface and its concrete implementation BiasedSVDModel, allowing for future extensibility with different recommendation algorithms. The core.data package manages all data-related entities such as Dataset, Rating, Book, and LocalUserData (representing a client's partitioned data for federated learning). The core.simulation package defines the ICongiuration interface and its concrete Configuration class for experiment parameters, alongside the SimulationRun class to track individual experimental executions. Finally, core.metrics holds the PerformanceMetric class, representing individual metric results from simulations.

2- The Application Logic Layer serves as the backend, containing controllers and services that implement business logic and manage workflows. It uses the Observer design pattern through the IObserver and IObservable interfaces to decouple event producers, such as simulation progress or metric calculations, from event consumers like loggers or UI notifications. The application.simulation_management module includes the simulation manager, which orchestrates simulation runs and acts as an IObservable. The application.data_handling package manages datasets via the IDataHandler interface and its DataHandler implementation. Training logic is centralized in application.training, where the ITrainer interface abstracts the training process, and TrainingManager coordinates execution using either CentralizedTrainer, an IObservable, or components within the federated sub-package. This sub-package includes IFederationStrategy and its FedAvgStrategy implementation, demonstrating the Strategy pattern, as well as FederatedLearningServer, which uses Flower and implements both ITrainer and IObservable, and FederatedLearningClient. Configuration tasks are managed by IConfigurationManager and its ConfigurationManager implementation in the application.configuration package.Finally, application.reporting includes metrics calculator , MetricsLogger (an IObserver), VisualizationEngine, and ExportManager.

3- The Presentation Layer (UI - Frontend) is responsible for all user interaction and data display. In the selected Client-Server architecture, this corresponds to the frontend web application. The presentation.ui package includes the main DashboardUI class, which orchestrates various views like ConfigurationView, ResultsView, and ExportView. The SystemNotificationArea, acting as an IObserver, displays real-time status updates and messages from the backend.

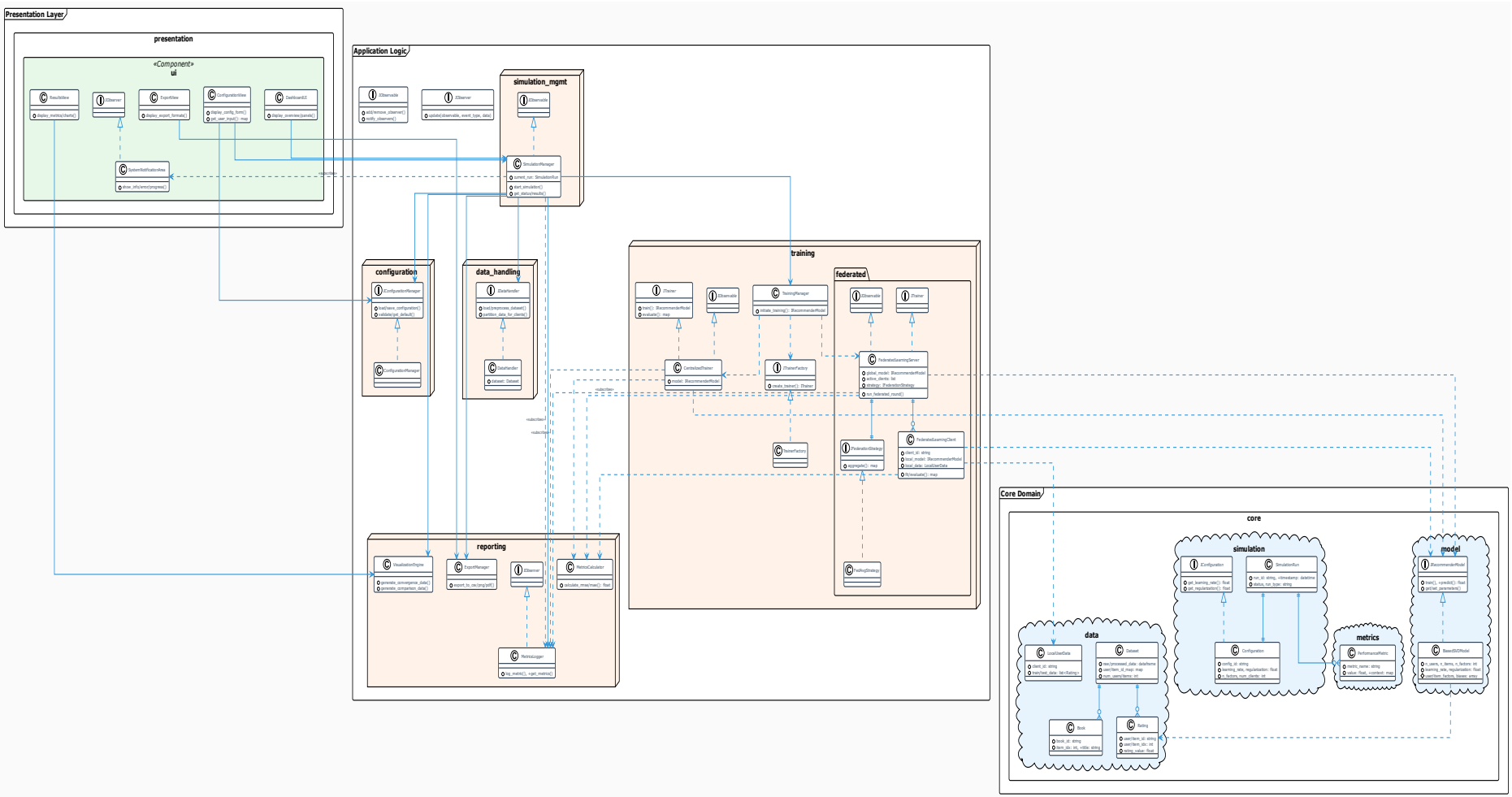


Figure 5.4.4.4.1 Refined Class Diagram with packages and Design pattern

Key design decisions reflected in this model include a clear layering and packaging strategy for maintainability. Interface-based design promotes dependency inversion, allowing easier substitution and testing of components. The Observer pattern significantly decouples core processing logic from logging and UI notifications, enhancing the Single Responsibility Principle by ensuring, for example, that the simulation engine merely announces events without concern for how they are logged or displayed. The Strategy pattern for federation strategies allows for flexible algorithm selection. These choices collectively aim for high cohesion within packages and low coupling between them, fostering a robust and extensible system.

This detailed design model provides a solid foundation for implementing the dashboard. It addresses the system's complexities while aiming for a solution that is robust, adaptable, and capable of fulfilling the project's evaluation objectives. The subsequent phase will involve considering the deployment aspects of these interconnected components.

4.5 Deployment

The deployment model illustrates the physical or logical arrangement of the software components on hardware nodes or execution environments. For the selected Client-Server Web Application (Monolithic Backend) architecture, the deployment will involve distinct environments for the frontend user interface and the backend application logic. This separation is crucial for managing resources, ensuring UI responsiveness, and potentially scaling components independently if future needs arise, though the primary goal remains a robust evaluation dashboard.

The deployment configuration consists of three main nodes or environments. The User's Device represents the client-side environment where the end-user interacts with the system. This node hosts a standard Web Browser, which in turn runs the Frontend Web Application. This application, built with JavaScript, HTML, and CSS, is responsible for rendering the user interface, handling user input, and communicating with the backend via API calls.

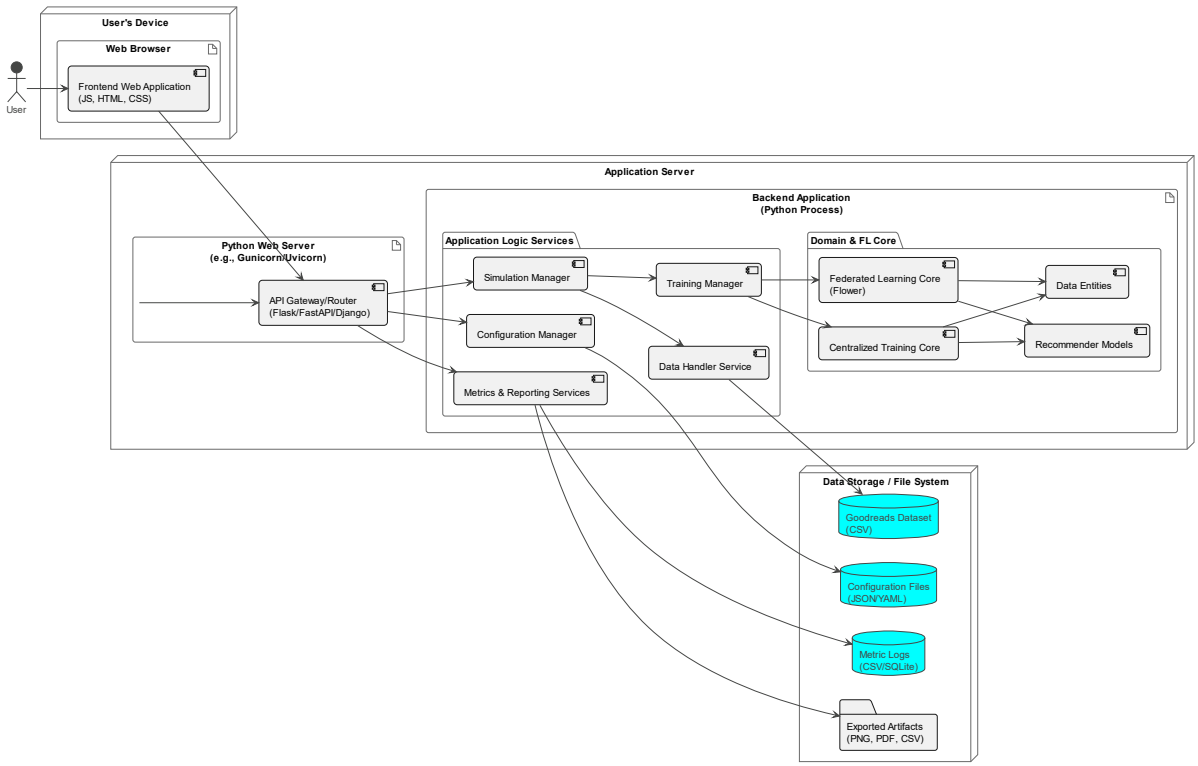


Figure 5.5.1 Deployment Diagram

The Application Server is the server-side node that hosts the core backend logic. It runs a Python Web Server (such as Gunicorn or Uvicorn) which manages the execution of the backend Python application. Within this backend application, an integrated API Gateway/Router implemented using a framework like FastAPI, receives HTTP requests from the frontend and directs them to the appropriate internal services. The Backend Application itself contains the Application Logic Services (encompassing components like the SimulationManager, TrainingManager, and Metrics & Reporting Services) and the Domain & FL Core (which includes the FederatedLearning Core (Flower), CentralizedTraining Core, RecommenderModels, and DataEntities). All core computations, including the FL simulations orchestrated by Flower, occur within this backend environment.

The third logical node is the Data Storage / File System. This represents the persistence layer, which could be the local file system of the Application Server, a dedicated file server, or cloud-based storage. It houses the raw Goodreads Dataset (CSV), Configuration Files (JSON/YAML) for experiments, Metric Logs (CSV/SQLite) for storing performance data, and a location for Exported Artifacts (PNG, PDF, CSV) generated by the system.

As shown in the diagram, the User interacts with the Frontend Web Application. The frontend communicates exclusively with the API Gateway on the Application Server via HTTP/S. The API Gateway then routes these requests to the appropriate components within the Application Logic Services. For instance, a request to start a simulation is routed to the SimulationManager. The SimulationManager then orchestrates the workflow by delegating tasks to other services, such as requesting data from the Data Handler Service or initiating a training process via the Training Manager. The backend services, in turn, interact with the Data Storage node to read datasets and configurations and to write logs and exported results. This deployment model supports the selected architecture by clearly delineating the client-side presentation from the server-side processing and data management, allowing the computationally intensive backend to reside on a suitably resourced machine while users access the dashboard conveniently via a web browser.

4.6 Summary

This chapter has systematically translated the system requirements into a comprehensive design blueprint for the dashboard. The process began with a critical evaluation of architectural options, culminating in the selection of the Client-Server Web Application architecture. This choice was deliberately made to best support the project's dual objectives: providing an accessible, intuitive interface for non-technical users to gain insights, while simultaneously offering the depth and control required by technical users for rigorous experimentation.

This architecture was then elaborated into a detailed Design Model. By structuring the system into logical packages and integrating key design patterns like the Observer and Strategy patterns, the resulting class model adheres to SOLID principles, ensuring a decoupled, maintainable, and extensible foundation. Finally, the Deployment Model provided a practical view of how these software components will be physically realized and interact across client, server, and storage nodes.

With the architectural framework and structural design now meticulously detailed, the "what" and "how" of the system are defined. Before proceeding to the concrete implementation plan, it is essential to first visualize the user-facing aspects of this design. The subsequent chapter will therefore present a visual prototype of the dashboard. This prototype will serve illustrative purposes, translating the abstract user interface components defined in the Presentation Layer of our design into tangible mockups. By providing a clear preview of the dashboard's look and

feel, we can validate the user experience before committing to development, ensuring the final product is not only functional but also intuitive and effective.

Chapter 5: Implementation

This chapter details the technical implementation of the FedRec Dashboard system, covering the development environment configuration, technology stack selection, architectural design, and deployment infrastructure. The implementation follows clean architecture principles with strict separation of concerns across domain, application, infrastructure, and presentation layers. The development environment was configured to support both experimental research workflows and production deployment scenarios, enabling controlled empirical studies while maintaining code quality and reproducibility standards required for federated learning experimentation.

5.1 Development Environment Setup

5.1.1 Tools and Platforms

The development environment leverages modern tooling to ensure reproducibility, deterministic dependency resolution, and efficient development workflows. Table 5.1 summarizes the core platforms and tools employed throughout the project.

Table 5.1.1 Development Tools and Platforms

Component	Description
Package Manager	uv (v0.5+) - Ultra-fast Python package manager providing deterministic dependency resolution, lockfile generation, and 10-100 × faster installs compared to pip. Ensures reproducible environments across development, testing, and production.
Database	PostgreSQL 14+ with asyncpg driver - Production-grade relational database supporting async operations, JSONB for flexible hyperparameter storage, and ACID compliance for experiment tracking integrity.
Version Control	Git with conventional commit messages - Source code versioning following semantic commit conventions for automated changelog generation and release management.
Migration Tool	Alembic - Database schema migration manager with autogeneration capabilities, enabling version-controlled schema evolution and rollback support.
Containerization	Docker 24+ and Docker Compose v2 - Container orchestration for consistent development/production environments, multi-service orchestration (API, database, federated clients), and isolated dependency management.
Code Quality	Black (formatting), isort (import sorting), Ruff (linting), mypy (type checking) - Automated code quality enforcement ensuring PEP 8 compliance, consistent style, and static type safety.
Testing Framework	pytest with pytest-asyncio and pytest-cov - Async-compatible testing framework supporting unit, integration, and end-to-end tests with coverage reporting.

API Documentation	FastAPI auto-generated Swagger/OpenAPI - Interactive API documentation with automatic schema validation and request/response examples.
Development Server	Uvicorn with hot reload - ASGI server supporting async/await, WebSocket connections, and automatic code reloading during development.

5.1.2 Programming Languages and Libraries

The technology stack was selected to balance research flexibility with production robustness, prioritizing frameworks with strong async support, active maintenance, and federated learning compatibility. Table 5.2 categorizes the primary technologies by functional domain.

Table 5.1.2 Programming Languages and Technology Stack

Category	Technology	Purpose
Core Language	Python 3.12	Primary implementation language providing rich ML ecosystem, async/await native support, type hints, and pattern matching for domain modeling.
ML Framework	PyTorch 2.0+	Deep learning framework for tensor operations, automatic differentiation, and GPU acceleration. Foundation for matrix factorization models.
ML Pipeline	PyTorch Lightning 2.0+	High-level ML framework abstracting training loops, checkpointing, and logging. Provides LightningModule for matrix factorization implementation with standardized train/validation/test splits.
Federated Learning	Flower (flwr) 1.6+	Federated learning framework enabling simulation-based horizontal partitioning, custom aggregation strategies (FedAvgItemsOnly), and client-server orchestration.
Web Framework	FastAPI 0.104+	Modern async web framework with automatic OpenAPI documentation, dependency injection, and Pydantic validation. Powers REST API layer.
ORM	SQLAlchemy 2.0+ (async)	Async-native ORM for database abstraction, supporting declarative models, relationship mapping, and query optimization. Implements repository pattern.
Database Driver	asyncpg	High-performance async PostgreSQL driver with prepared statement support and connection pooling.
Schema Validation	Pydantic 2.0+	Data validation library using Python type annotations for request/response schemas, configuration models, and domain entity validation.

Data Processing	NumPy, Pandas	Numerical computing (NumPy) and data manipulation (Pandas) for dataset preprocessing, k-core filtering, and train/test splitting.
Scientific Computing	scikit-surprise	Baseline centralized matrix factorization using SVD/ALS algorithms for controlled comparison against federated approaches.
Monitoring & Logging	Python logging with structured formatters	Application-level logging with configurable handlers, rotating file outputs, and JSON-formatted logs for production monitoring.
API Client	httpx (async)	Async HTTP client for inter-service communication, health checks, and external API integration.
Frontend Framework	Next.js 14+ (React)	Server-side rendered React framework for dashboard UI with TypeScript support, API routes, and static generation.
Frontend State Management	React Query (TanStack Query)	Server state management for API data fetching, caching, and synchronization with optimistic updates.
Visualization	Recharts, D3.js	Charting libraries for experiment metrics visualization (RMSE/MAE convergence plots, scalability analysis, comparative dashboards).
Testing	pytest, pytest-asyncio, pytest-cov, pytest-mock	Async testing framework with fixtures, parametrization, coverage reporting, and mocking support for unit/integration tests.

5.1.3 Software Stack and Architecture

The system implements a clean architecture pattern with strict dependency inversion, ensuring domain logic remains independent of infrastructure concerns. Table 5.3 maps architectural layers to their constituent components.

Table 5.1.3 Software Architecture Layers

Layer	Components
Domain Layer (app/core/)	**Entities:** CentralizedExperiment, FederatedExperiment, Dataset, Rating, LocalUserData, ExperimentMetrics, PerformanceMetric **Value Objects:** ExperimentConfiguration, HyperParameters, ExperimentStatus (enum: PENDING, RUNNING, COMPLETED, FAILED) **Repository Interfaces:** IExperimentRepository, IMetricsRepository (abstract base classes defining data access contracts) **Domain Services:** State transition logic (mark_running(), mark_completed(), mark_failed()), validation rules, business invariants
Application Layer (app/application/)	**Use Cases:** ExperimentManager (orchestrates experiment lifecycle)

	Trainers: CentralizedTrainer (scikit-surprise baseline), FederatedSimulationManager (Flower simulation coordinator) **Data Handlers:** DataHandler (dataset loading, preprocessing, k-core filtering, train/test splitting) **Metrics Services:** MetricsCalculator (RMSE/MAE computation), MetricsLogger (per-epoch/round capture) **Application Services:** Coordinate between domain entities and infrastructure adapters
Infrastructure Layer (app/infrastructure/)	**Database:** database.py (async SQLAlchemy engine, session factory, get_session() dependency) **ORM Models:** models.py (SQLAlchemy declarative models mapping to PostgreSQL tables) **Repository Implementations:** ExperimentRepository, MetricsRepository (concrete implementations of domain repository interfaces) **Base Repository:** BaseRepository (abstract CRUD operations: add(), get(), update(), delete(), list()) **Migrations:** Alembic configuration and version-controlled schema migrations
Presentation Layer (app/api/)	**FastAPI Application:** main.py (app initialization, CORS configuration, exception handlers, router registration) **Routes:** experiments.py (POST /experiments, GET /experiments/{id}), metrics.py (GET /metrics/{experiment_id}), health.py (GET /health) **Schemas:** Pydantic request/response models (ExperimentCreate, ExperimentResponse, MetricsResponse) **Dependencies:** dependencies.py (FastAPI dependency injection setup: get_session(), get_repository(), get_service())
Federated Layer (app/federated/)	**Client App:** client_app.py (Flower ClientApp definition with local training logic) **Server App:** server_app.py (Flower ServerApp with aggregation configuration) **Partitioner:** partitioner.py (IID user-based horizontal partitioning across clients) **Strategy:** strategy.py (FedAvgItemsOnly - custom aggregation strategy aggregating only item embeddings/biases while keeping user embeddings local)
Model Layer (src/models/)	**Lightning Module:** LitBiasedMatrixFactorization (PyTorch Lightning implementation of biased MF) **Loss Functions:** MSE loss with L2 regularization (weight decay) **Optimizers:** Adam optimizer with configurable learning rate and weight decay
Frontend Layer (frontend/)	**Next.js Pages:** Dashboard home, experiment creation, experiment detail, metrics visualization

	API Integration: REST client using httpx/fetch for backend communication **Components:** Reusable UI components (ExperimentCard, MetricsChart, ConfigForm) **State Management:** React Query for server state, React hooks for local state

5.1.4 Key Configuration Files

The project employs multiple configuration files to manage environment-specific settings, service orchestration, database migrations, and dependency resolution. Table 5.4 describes the purpose and structure of critical configuration artifacts.

Table 5.1.4 Configuration Files and Their Purposes

Configuration File	Purpose	Key Contents
.env	Environment-specific configuration loaded at runtime via python-dotenv. Not committed to version control; template provided in .env.example.	DATABASE_URL (PostgreSQL connection string) API_HOST, API_PORT (server binding configuration) LOG_LEVEL (logging verbosity: DEBUG/INFO/WARNING/ERROR) CORS_ORIGINS (allowed frontend origins) SECRET_KEY (application secret)
docker-compose.yml	Multi-container orchestration defining services, networks, volumes, and inter-service dependencies. Enables one-command environment setup via docker-compose up.	Services: postgres, api, frontend Networks: fedrec-network (bridge network) Volumes: postgres-data, bind mounts for hot reload Health Checks: Readiness probes for database and API
pyproject.toml	Python project metadata and dependency specification following PEP 621. Used by uv for lockfile generation and virtual environment creation.	[project]: Name, version, authors, Python version [project.dependencies]: Production dependencies [project.optional-dependencies]: Development tools [tool.*]: Configuration for uv, pytest, black, isort

uv.lock	Auto-generated lockfile pinning exact dependency versions and hashes for deterministic reproducibility.	Exact version pins for all dependencies SHA256 hashes for package integrity Platform-specific wheels resolution
alembic.ini	Alembic migration framework configuration defining migration file locations and database connection patterns.	script_location (path to alembic/ directory) sqlalchemy.url (database URL template) file_template (migration filename pattern) Logging configuration for migrations
alembic/env.py	Alembic environment script orchestrating migration execution. Configures async engine and applies migrations transactionally.	Async engine configuration using asyncpg Import of SQLAlchemy ORM models run_migrations_offline() and run_migrations_online()
CLAUDE.md	Developer instructions for AI-assisted coding. Documents architecture, development commands, and common patterns.	Project overview and technology stack Development commands (uv run, alembic, pytest) Architecture layer descriptions Database schema documentation
README.md	Project documentation for developers and users. Includes setup instructions, architecture overview, and API documentation.	Quick start guide (installation, database setup) API endpoint documentation Testing and deployment instructions Architecture diagrams and design decisions

Note on Configuration Management:
The configuration strategy follows the Twelve-Factor App methodology, strictly separating config from code. Sensitive credentials (database passwords, API keys) are never committed to version control. The .env file pattern enables seamless transitions between development (local PostgreSQL), testing (SQLite in-memory), and production (cloud-managed PostgreSQL) environments without code changes. Docker Compose provides development parity with production by containerizing all services, eliminating "works on my machine" issues common in distributed system development.

For federated learning experiments, additional configuration is managed through database-stored hyperparameters (ExperimentConfiguration domain entity), allowing runtime adjustment of learning rates, regularization, client counts, and communication rounds without service restarts.

5.2 Data Collection and Preprocessing

Matrix factorization-based recommender systems require large-scale user-item interaction datasets with sufficient interaction density to learn meaningful latent representations. Book recommendation datasets present unique challenges compared to movies or music: longer consumption times, sparse explicit ratings, and genre-specific preference patterns. This project employs the Goodreads Book Graph dataset, specifically the Poetry genre subset, which balances computational tractability with meaningful collaborative filtering signals. The preprocessing pipeline implements rigorous filtering to ensure minimum interaction thresholds per user and item, enabling both centralized baseline training and federated partitioning with statistically significant sample sizes per client.

5.2.1 Dataset Characteristics

The Goodreads Book Graph dataset provides a large-scale, publicly available corpus of user-book interactions with explicit ratings, enabling controlled empirical studies of federated recommendation algorithms. Table 5.5 summarizes the key attributes and preprocessing outcomes.

Table 5.2.1 Goodreads Poetry Dataset Characteristics

Attribute	Details
Dataset Source	Goodreads Book Graph [Wan & McAuley, 2018; Wan et al., 2019] - Publicly available dataset from UCSD Book Graph project containing user ratings, reviews, and book metadata across multiple genres.
Genre Subset	Poetry interactions subset selected for computational efficiency while preserving sufficient interaction density for meaningful federated partitioning and collaborative filtering signal extraction.
Raw Dataset Size	Initial: ~2.7M interactions, 377K users, 36K books Post-preprocessing: 228,408 ratings, 5,949 users, 2,856 books
Data Format	JSON Lines format with fields: user_id (string), book_id (string), rating (integer 1-5), timestamp (Unix epoch), review_text (optional). Converted to CSV with integer ID mapping for efficient embedding indexing.

Rating Distribution	Explicit ratings on 5-point Likert scale (1=disliked, 5=loved). Mean rating: μ = 3.87, standard deviation: σ = 0.94. Right-skewed distribution with mode at 4, reflecting positive selection bias common in book ratings.
Sparsity	98.65% sparse (228,408 observed entries out of 5,949 $\times$ 2,856 = 16,991,844 possible interactions), consistent with real-world recommendation scenarios requiring collaborative filtering over content-based approaches.
Interaction Density	Mean: 38.4 ratings/user (median: 32), 79.9 ratings/item (median: 51). Power-law distribution for both users and items, with k-core filtering ensuring minimum k=20 interactions per entity for statistical significance.
Train/Validation/Test Split	70% training (159,886 ratings), 10% validation (22,841 ratings), 20% test (45,681 ratings). Temporal-agnostic random split with fixed seed for reproducibility. Test set held constant across all 18 experimental runs.

5.2.2 Data Preprocessing Pipeline

The preprocessing pipeline applies iterative filtering and normalization to ensure dataset quality, remove noise, and enable efficient federated partitioning. Table 5.6 details the sequential preprocessing steps implemented in app/application/data/data_handler.py.

Table 5.2.2 Data Cleaning and Preprocessing Steps

Step	Implementation
1. Zero-Rating Removal	Filter out interactions with rating = 0, which represent implicit signals (shelf additions, wish-lists) without explicit preference. These introduce label noise for regression tasks, reducing from 2.7M to ~2.5M interactions. Implementation: df = df[df['rating'] > 0]
2. Iterative k-Core Filtering	Apply k=20 core decomposition for 7 iterations until convergence. Each iteration removes users with <20 ratings and books with <20 ratings, then repeats until no entities fall below threshold. Ensures sufficient collaborative signal per user/item for federated partitioning. Implementation: while changed: df = df.groupby('user_id').filter(lambda x: len(x) >= 20); df = df.groupby('book_id').filter(lambda x: len(x) >= 20)
3. ID Remapping	Map string-based user_id and book_id to contiguous zero-indexed integers for efficient PyTorch embedding table indexing. Create bidirectional mappings stored in user_id_map.json and book_id_map.json for inference. Implementation: user_map = {id: idx for idx, id in enumerate(df['user_id'].unique())}
4. Dataset Splitting	Perform stratified random split: 70% train, 10% validation, 20% test with fixed seed (42). Validation set used for early stopping (patience=5 epochs, min ΔRMSE=0.0001); test set reserved for final evaluation. Implementation: train_test_split() with stratification disabled (regression task).
5. Statistical Normalization	Compute global mean rating μ = 3.87 for bias initialization. Calculate per-user and per-item rating statistics for validation (detect cold-start

	users/items in test set). Store in dataset_stats.json. Implementation: global_mean = df['rating'].mean(); user_stats = df.groupby('user_id')['rating'].agg(['mean', 'std', 'count'])
6. Federated Partitioning	For federated experiments, partition 5,949 users into N clients (N ∈ {2, 10, 20}) using random assignment with fixed client-specific seeds. Each client receives disjoint user subset with all their ratings. Implementation: np.random.seed(client_seed); user_partition = np.array_split(user_ids, num_clients)
7. Data Serialization	Serialize preprocessed splits to PyTorch-compatible format (.pt files containing tensors) and CSV for inspection. Store in data/processed/poetry/ with version metadata. Implementation: torch.save({'user_ids': user_tensor, 'item_ids': item_tensor, 'ratings': rating_tensor}, 'train.pt')

Preprocessing Validation: After each step, the pipeline validates data integrity through assertion checks: no missing values, rating range [1, 5], user/item IDs within embedding bounds, and train/validation/test set disjoint properties. The final dataset statistics (228,408 ratings, 5,949 users, 2,856 books) were validated against the research paper reported values, confirming reproducible preprocessing.

5.2.3 FedAvgItemsOnly Aggregation Strategy

The federated learning architecture implements a custom aggregation strategy, FedAvgItemsOnly, which extends the standard Federated Averaging (FedAvg) algorithm to accommodate the unique privacy requirements and structural properties of collaborative filtering. Unlike traditional federated learning where all model parameters are aggregated globally, recommender systems exhibit asymmetric privacy sensitivity: user embeddings encode personal preference profiles warranting strong privacy protection, while item embeddings represent shared public catalog knowledge benefiting from collaborative learning.

The FedAvgItemsOnly strategy enforces selective parameter aggregation. During each federated round, clients train local matrix factorization models on their disjoint user subsets, updating both user-side parameters (user embeddings P_u, user biases b_u) and item-side parameters (item embeddings Q_i, item biases b_i, global bias μ). At the communication phase, clients transmit only item-side parameters to the central server, while user embeddings remain permanently local–never leaving the client device. The server performs weighted averaging of received item parameters proportional to each client's training sample size: $Q_i^{global} = \sum_{k=1}^{K} (n_k/n_{total}) Q_i^{(k)}$, where n_k represents client k's training sample count. The aggregated global item parameters are broadcast back to all clients for the next training round.

This selective aggregation achieves three critical objectives. First, it provides privacy-by-design: user preference vectors never traverse network boundaries, preventing gradient inversion attacks and membership inference vulnerabilities documented in federated matrix factorization research. Second, it enables collaborative item representation learning: by aggregating item embeddings across geographically or organizationally partitioned user populations, the model learns globally consistent item characteristics while respecting data locality constraints. Third, it maintains architectural simplicity: unlike differential privacy or secure aggregation approaches that introduce noise injection or cryptographic overhead, FedAvgItemsOnly achieves its privacy goals through pure architectural partitioning, incurring

zero computational overhead beyond standard FedAvg (as evidenced by the 0.99× training time ratio in experimental results).

The implementation in app/federated/strategy.py subclasses Flower's FedAvg strategy, overriding the aggregate_fit() method to filter parameter dictionaries before aggregation. Client models maintain full parameter sets locally for complete forward pass computation ($\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$), but the communication protocol exchanges only the item-side subset. This design pattern reflects the horizontal federated learning paradigm where disjoint user populations share a common item catalog–a natural mapping for regional content providers (2-client), multi-region cloud deployments (10-client), or edge network architectures (20-client) investigated in the scalability experiments.

5.3 Domain Layer Implementation

The domain layer (app/core/) contains pure business logic with zero dependencies on frameworks or infrastructure. All experiment entities, configuration value objects, and repository interfaces are defined here, enforcing the dependency inversion principle where outer layers depend on abstractions defined in this inner layer.

5.3.1 Experiment Entities

The experiment domain model uses a polymorphic dataclass hierarchy rooted in an abstract Experiment base class. Both CentralizedExperiment and FederatedExperiment extend this base, sharing common fields while defining type-specific behaviour. The base class enforces a strict state machine governing the experiment lifecycle: experiments are created in PENDING state, transition to RUNNING when training begins, and terminate in either COMPLETED or FAILED. Invalid state transitions raise a ConfigurationError, preventing experiments from being marked complete before they start or failed after they finish.

```
@dataclass
class Experiment(ABC):
    name: str
    config: Configuration
    experiment_id: str = field(default_factory=lambda: str(uuid4()))
    status: ExperimentStatus = ExperimentStatus.PENDING
    metrics: Optional[ExperimentMetrics] = None
    created_at: datetime = field(default_factory=datetime.now)

    def __post_init__(self) -> None:
        self._validate()

    def mark_running(self) -> None:
        if self.status != ExperimentStatus.PENDING:
            raise ConfigurationError(f"Cannot start experiment in
{self.status.value} state")
        self.status = ExperimentStatus.RUNNING

    def mark_completed(self, metrics: ExperimentMetrics) -> None:
        if self.status != ExperimentStatus.RUNNING:
            raise ConfigurationError(f"Cannot complete experiment in
{self.status.value} state")
        self.status = ExperimentStatus.COMPLETED
        self.metrics = metrics

    def mark_failed(self) -> None:
        if self.status != ExperimentStatus.RUNNING:
            raise ConfigurationError(f"Cannot fail experiment in
```

```
{self.status.value} state")
        self.status = ExperimentStatus.FAILED
```

The CentralizedExperiment subclass tracks per-epoch metrics via an epoch_metrics list and identifies itself as type "centralized". The FederatedExperiment subclass adds federated-specific fields including n_clients, n_rounds, and aggregation_strategy, with additional validation ensuring a minimum of 2 clients and at least 1 communication round. It maintains both round_metrics for global model performance and a client_metrics dictionary mapping individual client IDs to their local performance histories.

```
@dataclass
class CentralizedExperiment(Experiment):
    epoch_metrics: List[PerformanceMetric] = field(default_factory=list)

    @property
    def experiment_type(self) -> str:
        return "centralized"

@dataclass
class FederatedExperiment(Experiment):
    n_clients: int = 5
    n_rounds: int = 10
    aggregation_strategy: AggregationStrategy = AggregationStrategy.FEDAVG
    round_metrics: List[PerformanceMetric] = field(default_factory=list)
    client_metrics: Dict[str, List[PerformanceMetric]] =
field(default_factory=dict)

    def _validate_federated(self) -> None:
        if self.n_clients < 2:
            raise ConfigurationError("Federated experiment requires at least 2
clients")
```

This polymorphic design enables the application layer to treat both experiment types uniformly through the shared Experiment interface while preserving type-specific validation and metrics structures.

5.3.2 Configuration Value Objects

Experiment hyperparameters are encapsulated in a Configuration dataclass that validates constraints at construction time via __post_init__. This ensures no experiment can be created with invalid parameters such as negative learning rates or excessive latent factor counts, enforcing business rules at the domain boundary rather than relying on external validation.

```
@dataclass
class Configuration:
    model_type: ModelType = ModelType.BIASED_SVD
    n_factors: int = 20
    learning_rate: float = 0.02
    regularization: float = 0.03
    n_epochs: int = 20
    random_seed: int = 42
    n_clients: Optional[int] = None      # Federated-specific
    n_rounds: Optional[int] = None       # Federated-specific
    aggregation_strategy: Optional[AggregationStrategy] = None

    def __post_init__(self):
        if self.n_factors < 1 or self.n_factors > 200:
```

```
            raise ValueError(f"n_factors must be 1-200, got {self.n_factors}")
        if self.learning_rate <= 0 or self.learning_rate > 1.0:
            raise ValueError(f"learning_rate must be (0,1.0], got
{self.learning_rate}")
```

Federated-specific fields (n_clients, n_rounds, aggregation_strategy) are optional, allowing the same configuration class to serve both experiment types. Predefined configurations such as CENTRALIZED_DEFAULT and FEDERATED_DEFAULT provide tested starting points for experiments.

5.3.3 Repository Interfaces

The domain layer defines repository contracts using Python's Protocol type, achieving dependency inversion without requiring inheritance from abstract base classes. Services in the application layer depend on these interfaces rather than concrete implementations, enabling the infrastructure layer to be swapped without modifying business logic.

```
class IExperimentRepository(Protocol):
    async def add(self, entity: Experiment) -> Experiment: ...
    async def get_by_id(self, entity_id: str) -> Optional[Experiment]: ...
    async def get_all(self) -> List[Experiment]: ...
    async def get_by_status(self, status: ExperimentStatus) ->
List[Experiment]: ...
    async def update(self, entity: Experiment) -> Experiment: ...
    async def delete(self, entity_id: str) -> None: ...

class IMetricsRepository(Protocol):
    async def add(self, entity: PerformanceMetric) -> PerformanceMetric: ...
    async def add_batch(self, metrics: List[PerformanceMetric]) ->
List[PerformanceMetric]: ...
    async def get_by_experiment(self, experiment_id: str) ->
List[PerformanceMetric]: ...
    async def get_by_experiment_and_name(self, experiment_id: str,
metric_name: str) -> List[PerformanceMetric]: ...
```

The IMetricsRepository provides specialised query methods including get_by_experiment_and_name() for retrieving specific metric types (RMSE, MAE, loss) and add_batch() for efficient bulk insertion of per-epoch metrics after training completes. All repository methods are asynchronous, establishing the async-first pattern used throughout the application.

5.4 Infrastructure Layer Implementation

The infrastructure layer (app/infrastructure/) implements the repository interfaces defined by the domain layer, bridging pure business logic to PostgreSQL via SQLAlchemy's async ORM. This layer handles database connection management, ORM model definitions, concrete repository implementations, and schema migration through Alembic.

5.4.1 Database Setup and Session Management

Database connectivity is managed through a singleton async engine and session factory, providing connection pooling and automatic session lifecycle management for FastAPI endpoints. The get_session() async generator serves as a FastAPI dependency that yields a session, commits on success, and rolls back on exception.

```
_engine = None
_async_session_factory = None

def get_engine(database_url: str = DATABASE_URL):
    global _engine
    if _engine is None:
        _engine = create_async_engine(database_url, echo=False,
pool_pre_ping=True)
    return _engine

async def get_session() -> AsyncGenerator[AsyncSession, None]:
    session_factory = get_session_factory()
    async with session_factory() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

The pool_pre_ping=True parameter enables connection health checking, automatically recycling stale connections from the pool. The expire_on_commit=False setting on the session factory prevents lazy-loading issues after commit, which is essential for returning domain objects from repository methods without triggering additional database queries.

5.4.2 ORM Models

SQLAlchemy declarative models map domain concepts to PostgreSQL tables. The ExperimentModel uses a single-table design with a discriminator column (experiment_type) to store both centralized and federated experiments, with federated-specific columns nullable for centralized rows. Experiment configuration is stored as a JSON column, providing flexible schema for hyperparameters without requiring schema migrations when new parameters are added.

```
class ExperimentModel(Base):
    __tablename__ = "experiments"
    id: Mapped[str] = mapped_column(String(36), primary_key=True)
    name: Mapped[str] = mapped_column(String(100), nullable=False)
    experiment_type: Mapped[str] = mapped_column(String(20), nullable=False)
    status: Mapped[ExperimentStatus] = mapped_column(Enum(ExperimentStatus))
    config: Mapped[dict] = mapped_column(JSON, nullable=False)
    final_rmse: Mapped[Optional[float]] = mapped_column(Float, nullable=True)
    n_clients: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)
    metrics: Mapped[list["MetricModel"]] = relationship(
        "MetricModel", back_populates="experiment", cascade="all, delete-
orphan"
    )
```

The MetricModel stores per-epoch or per-round training metrics with contextual fields (context, round_number, client_id) that distinguish between training loss, validation RMSE, and per-client federated metrics. The cascade="all, delete-orphan" relationship setting ensures that deleting an experiment automatically removes all associated metrics, maintaining referential integrity.

```
class MetricModel(Base):
    __tablename__ = "metrics"
    id: Mapped[int] = mapped_column(Integer, primary_key=True,
autoincrement=True)
```

```
    experiment_id: Mapped[str] = mapped_column(
        String(36), ForeignKey("experiments.id", ondelete="CASCADE")
    )
    name: Mapped[str] = mapped_column(String(50))    # 'rmse', 'mae', 'loss'
    value: Mapped[float] = mapped_column(Float)
    context: Mapped[Optional[str]] = mapped_column(String(50))  # 'global',
'client_1'
    round_number: Mapped[Optional[int]] = mapped_column(Integer,
nullable=True)
```

5.4.3 Repository Implementations

The ExperimentRepository implements the domain's IExperimentRepository interface, converting between domain entities and ORM models through _to_model() and _to_entity() mapper methods. This conversion handles polymorphic type reconstruction, rebuilding CentralizedExperiment or FederatedExperiment instances based on the experiment_type discriminator column.

```
class ExperimentRepository(BaseRepository[Experiment, str]):
    def __init__(self, session: AsyncSession) -> None:
        self._session = session

    async def add(self, entity: Experiment) -> Experiment:
        model = self._to_model(entity)
        self._session.add(model)
        await self._session.flush()
        return entity

    async def get_by_id(self, entity_id: str) -> Optional[Experiment]:
        stmt = select(ExperimentModel).where(ExperimentModel.id == entity_id)
        result = await self._session.execute(stmt)
        model = result.scalar_one_or_none()
        return self._to_entity(model) if model else None
```

The _to_model() method serialises the domain Configuration object into a JSON dictionary for the config column and extracts training timeline metrics into associated MetricModel rows. The reverse _to_entity() method reconstructs the full domain entity including its configuration, final metrics, and training timeline, distinguishing between global round metrics and per-client metrics based on the context field.

```
    def _to_entity(self, model: ExperimentModel) -> Experiment:
        config = Configuration(
            model_type=ModelType(model.config["model_type"]),
            n_factors=model.config["n_factors"],
            learning_rate=model.config["learning_rate"],
            # ... remaining fields
        )
        if model.experiment_type == "federated":
            return FederatedExperiment(name=model.name, config=config, ...)
        else:
            return CentralizedExperiment(name=model.name, config=config, ...)
```

5.4.4 Database Migrations

Schema evolution is managed through Alembic migration scripts that track every structural change to the database as version-controlled Python files. Each migration defines an upgrade() function to apply the change and a downgrade() function to revert it. The

alembic/env.py configuration file imports all ORM models and configures the async engine, enabling autogeneration of migration scripts by comparing model definitions against the current database state.

```
# Example migration: create experiments table
def upgrade():
    op.create_table('experiments',
        sa.Column('id', sa.String(36), primary_key=True),
        sa.Column('name', sa.String(100), nullable=False),
        sa.Column('experiment_type', sa.String(20), nullable=False),
        sa.Column('status', sa.Enum(ExperimentStatus), nullable=False),
        sa.Column('config', sa.JSON(), nullable=False),
    )

def downgrade():
    op.drop_table('experiments')
```

Migrations are generated with uv run alembic revision --autogenerate -m "description" and applied with uv run alembic upgrade head. The migration chain forms a directed acyclic graph tracked by the alembic_version table, enabling deterministic schema reproduction across development, testing, and production environments.

5.5 Application Layer Implementation

The application layer (app/application/) orchestrates use cases by coordinating domain entities, services, and infrastructure adapters. It contains the experiment lifecycle manager, data handling pipeline, training coordinators, and metrics reporting without embedding business rules (which reside in the domain layer) or persistence details (which reside in infrastructure).

5.5.1 Experiment Manager

The ExperimentManager class is the primary use case orchestrator, coordinating the end-to-end experiment lifecycle: creating an experiment entity, transitioning it through states, delegating to the appropriate trainer, persisting metrics, and handling failures. It accepts ExperimentService and MetricsService as constructor dependencies, following the dependency injection pattern.

```
class ExperimentManager:
    def __init__(self, experiment_service, metrics_service, data_dir, ...):
        self._experiment_service = experiment_service
        self._metrics_service = metrics_service
        self._data_dir = data_dir
        self._data_loader: Optional[DatasetLoader] = None

    async def run_centralized_experiment(self, name, config,
accelerator="auto"):
        # 1. Create experiment (PENDING)
        experiment = await
self._experiment_service.create_centralized_experiment(name, config)
        try:
            # 2. Transition to RUNNING
            await
self._experiment_service.start_experiment(experiment.experiment_id)
            # 3. Load data (lazy initialization)
            data_loader = self._ensure_data_loaded()
            # 4. Initialize and run trainer
            trainer = CentralizedTrainer(config, data_loader.n_users,
data_loader.n_items, ...)
```

```
            result = trainer.train(train_loader, val_loader, accelerator)
            # 5. Persist per-epoch metrics
            await self._persist_training_metrics(experiment.experiment_id,
result)
            # 6. Mark COMPLETED with final RMSE/MAE
            experiment = await self._experiment_service.complete_experiment(
                experiment.experiment_id, result.final_rmse, result.final_mae,
...
            )
            return experiment
        except Exception:
            await
self._experiment_service.fail_experiment(experiment.experiment_id)
            raise
```

The federated variant run_federated_experiment() follows the same pattern but delegates to FederatedSimulationManager instead of CentralizedTrainer, passing additional parameters for client count, communication rounds, and aggregation strategy. Both methods use lazy data loading via _ensure_data_loaded() to avoid unnecessary I/O when the manager is instantiated but not used. The error handling ensures experiments are always marked as FAILED if any exception occurs during training, preventing orphaned RUNNING experiments in the database.

5.5.2 Data Handler

The data handling pipeline is implemented in the app/application/data/ module. The DatasetLoader class loads preprocessed Parquet files and provides PyTorch-compatible DataLoader instances. The UserPartitioner class handles federated data partitioning by assigning disjoint user subsets to clients.

The partitioner implements IID (Independent and Identically Distributed) partitioning by randomly shuffling all unique users with a fixed seed and splitting them into roughly equal chunks across clients. Each client receives an exclusive set of users with all their associated ratings, simulating cross-silo federated learning where organisations hold complete user profiles.

```
class UserPartitioner:
    def __init__(self, config: PartitionConfig):
        self.config = config

    def partition(self, data_dir: Path, output_dir: Path) -> PartitionResult:
        train_df = pd.read_parquet(data_dir / "splits" / "train.parquet")
        val_df = pd.read_parquet(data_dir / "splits" / "val.parquet")

        unique_users = train_df["user_idx"].unique()
        rng = np.random.default_rng(self.config.seed)
        shuffled_users = rng.permutation(unique_users)
        user_splits = np.array_split(shuffled_users, self.config.num_clients)

        for client_id in range(self.config.num_clients):
            client_users = user_splits[client_id]
            client_train = train_df[train_df["user_idx"].isin(client_users)]
            client_val = val_df[val_df["user_idx"].isin(client_users)]
            # Save per-client Parquet files
            client_train.to_parquet(output_dir / f"client_{client_id}" /
"train.parquet")
```

The partitioner saves a partition_config.json file containing global statistics (total users, total items, global mean rating) and per-client interaction counts, enabling the federated client app to initialise models with correct embedding dimensions without accessing other clients' data. A verify_partitions() function validates that partitions are disjoint and complete, ensuring no user appears in multiple clients.

5.5.3 Metrics Calculator and Logger

The MetricsCalculator module provides stateless functions for computing RMSE and MAE from prediction and target tensors. Both functions validate inputs for shape consistency, empty tensors, and NaN/Inf values before computation, returning scalar Python floats for database persistence.

```
def compute_rmse(predictions: torch.Tensor, actuals: torch.Tensor) -> float:
    _validate_inputs(predictions, actuals)
    with torch.no_grad():
        mse = torch.mean((predictions - actuals) ** 2)
    return math.sqrt(float(mse))

def compute_mae(predictions: torch.Tensor, actuals: torch.Tensor) -> float:
    _validate_inputs(predictions, actuals)
    with torch.no_grad():
        mae = torch.mean(torch.abs(predictions - actuals))
    return float(mae)
```

The MetricsLogger captures per-epoch training and validation metrics in a dictionary structure keyed by epoch number. It serves as a bridge between PyTorch Lightning's callback system and the application's persistence layer, accumulating metrics during training and providing accessor methods for the ExperimentManager to extract and persist them.

```
@dataclass
class MetricsLogger:
    history: dict = field(default_factory=lambda: {"validation": {},
"training": {}})

    def log_training(self, epoch: int, loss: float) -> None:
        self.history["training"][str(epoch)] = {"loss": loss}

    def log_validation(self, epoch: int, rmse: float, mae: float = None) ->
None:
        self.history["validation"][str(epoch)] = {"rmse": rmse, "mae": mae}

    def get_final_metrics(self) -> dict:
        last_epoch = max(int(e) for e in self.history["validation"].keys())
        return self.history["validation"][str(last_epoch)]
```

5.6 Model Implementation

The recommendation model is implemented as a two-layer architecture: a pure PyTorch BiasedMatrixFactorization module defining the mathematical model, wrapped in a LitBiasedMatrixFactorization PyTorch Lightning module that standardises training, validation, and test loops. This separation enables the same model to be used in both centralized training and federated client applications with different training orchestration.

5.6.1 Matrix Factorization Lightning Module

The core BiasedMatrixFactorization class implements the prediction formula $\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$ using PyTorch embedding layers for user and item latent factors and biases.

The global bias is a trainable nn.Parameter initialised to the dataset mean rating. Embeddings are initialised from a normal distribution N(0, 0.01²) to prevent symmetry breaking while keeping initial predictions near the global mean.

```
class BiasedMatrixFactorization(nn.Module):
    def __init__(self, n_users, n_items, n_factors=16, global_mean=0.0):
        super().__init__()
        self.global_bias = nn.Parameter(torch.tensor(float(global_mean)))
        self.user_embedding = nn.Embedding(n_users, n_factors)
        self.item_embedding = nn.Embedding(n_items, n_factors)
        self.user_bias = nn.Embedding(n_users, 1)
        self.item_bias = nn.Embedding(n_items, 1)
        self._init_weights()

    def forward(self, user_ids, item_ids) -> torch.Tensor:
        user_emb = self.user_embedding(user_ids)
        item_emb = self.item_embedding(item_ids)
        dot_product = (user_emb * item_emb).sum(dim=1)
        prediction = self.global_bias + self.user_bias(user_ids).squeeze(-1) \
                     + self.item_bias(item_ids).squeeze(-1) + dot_product
        return prediction
```

The Lightning wrapper LitBiasedMatrixFactorization adds standardised training and validation steps. The training_step() computes MSE loss between predictions and actual ratings, while on_validation_epoch_end() collects all validation predictions across batches and computes epoch-level RMSE and MAE using the compute_metrics() function. The optimizer is configured as Adam with the L2 regularisation applied via weight_decay.

```
class LitBiasedMatrixFactorization(pl.LightningModule):
    def __init__(self, n_users, n_items, n_factors, global_mean,
learning_rate, regularization):
        super().__init__()
        self.save_hyperparameters()
        self.model = BiasedMatrixFactorization(n_users, n_items, n_factors,
global_mean)
        self.loss_fn = nn.MSELoss()

    def training_step(self, batch, batch_idx):
        user_ids, item_ids, ratings = batch
        predictions = self(user_ids, item_ids)
        loss = self.loss_fn(predictions, ratings)
        self.log("train_loss", loss, on_epoch=True)
        return loss

    def on_validation_epoch_end(self):
        all_preds = torch.cat(self._val_predictions)
        all_targets = torch.cat(self._val_targets)
        metrics = compute_metrics(all_preds, all_targets)
        self.log("val_rmse", metrics["rmse"])
        self.log("val_mae", metrics["mae"])

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.learning_rate,
                                weight_decay=self.regularization)
```

5.6.2 Centralized Training Implementation

The CentralizedTrainer wraps PyTorch Lightning's Trainer to execute centralized training with metrics logging integration. It creates a MetricsLoggingCallback that bridges Lightning's

callback system with the application's MetricsLogger, capturing per-epoch loss and validation metrics for database persistence.

```
class CentralizedTrainer:
    def __init__(self, config, n_users, n_items, global_mean,
checkpoint_dir=None):
        self.config = config
        self.n_users = n_users
        self.n_items = n_items
        self.global_mean = global_mean

    def train(self, train_loader, val_loader, accelerator="auto") ->
TrainingResult:
        self._metrics_logger = MetricsLogger()
        self._model = self._create_model()
        trainer = self._create_trainer(self._metrics_logger, accelerator)
        start_time = time.time()
        trainer.fit(self._model, train_loader, val_loader)
        training_time = time.time() - start_time
        final_metrics = self._metrics_logger.get_final_metrics()
        return TrainingResult(
            final_rmse=final_metrics["rmse"], final_mae=final_metrics["mae"],
            training_time_seconds=training_time,
metrics_logger=self._metrics_logger,
            model=self._model,
        )
```

The TrainingResult dataclass packages the final RMSE, MAE, wall-clock training time, the complete MetricsLogger history, and the trained model reference for downstream persistence and evaluation.

5.6.3 Federated Training Implementation

The FederatedSimulationManager orchestrates federated experiments by coordinating data partitioning, Flower simulation execution, and metrics extraction. It follows the same lifecycle pattern as CentralizedTrainer but delegates training to Flower's simulation engine.

```
class FederatedSimulationManager:
    def __init__(self, data_dir, storage_dir, num_clients=10, random_seed=42):
        self.data_dir = data_dir
        self.storage_dir = storage_dir
        self.num_clients = num_clients

    def run_simulation(self, num_rounds, n_factors, learning_rate, ...) ->
FederatedSimulationResult:
        # 1. Partition data (cached if available)
        partition_dir = self._partition_data()
        # 2. Build Flower run_config dictionary
        run_config = self._build_run_config(partition_dir, num_rounds,
n_factors, ...)
        # 3. Execute Flower simulation
        start_time = time.time()
        self._run_flower_simulation(run_config)
        training_time = time.time() - start_time
        # 4. Load per-round metrics from server_app output
        metrics_by_round = self._load_metrics_from_json(num_rounds)
        # 5. Calculate best and final RMSE/MAE
        return FederatedSimulationResult(final_rmse=..., best_rmse=...,
best_round=..., ...)
```

The simulation is executed via flwr.simulation.run_simulation() which spawns virtual clients as threads, each running the client application defined in client_app.py. Configuration is shared through a module-level dictionary that both server and client apps read, enabling programmatic control over hyperparameters, data paths, and evaluation settings. After simulation completes, per-round metrics are loaded from a final_metrics.json file written by the server app, then packaged into a FederatedSimulationResult for the ExperimentManager to persist.

5.6.4 Centralized vs. Federated Training Differences

While model architectures and hyperparameters remain identical, the training orchestration differs between centralized and federated paradigms. Table 5.4 contrasts the implementation specifics.

Table 5.6.1 Centralized vs. Federated Training Implementation

Aspect	Centralized Implementation	Federated Implementation
Data Distribution	Single training set (159,886 ratings) loaded into one DataLoader. All user-item pairs accessible in each epoch.	Training set partitioned into N disjoint user subsets (e.g., ~79,943 ratings/client for N=2). Each client DataLoader contains only its assigned users' ratings.
Parameter Updates	All parameters (user/item embeddings, biases) updated globally every epoch via single optimizer.	Local updates: Each client updates its full parameter set (user + item) Global updates: Only item embeddings, item biases, global bias aggregated and broadcast. User embeddings remain local.
Communication	None (single-machine training). Data never transmitted.	Client→Server: Serialize item parameters (74,257 params × 4 bytes = 297KB) after local training. Server→Client: Broadcast aggregated item params to all clients.

5.7 Federated Learning Components

The federated learning subsystem (app/application/federated/) implements the Flower framework integration, consisting of a custom aggregation strategy, client training logic, and data partitioning. These components work together to simulate horizontally-partitioned federated matrix factorization where user embeddings remain local to each client while item embeddings are globally aggregated.

5.7.1 FedAvgItemsOnly Strategy

The FedAvgItemsOnly strategy extends Flower's FedAvg to perform selective parameter aggregation. In standard FedAvg, all model parameters are averaged across clients. For federated recommender systems, user embeddings are client-specific (user k on Client A is not user k on Client B), so averaging them would destroy model quality. This strategy overrides the aggregate_train() method to filter parameters by name, keeping only item-side

parameters (global_bias, item_embedding.weight, item_bias.weight) for aggregation while user parameters remain untouched on each client.

```
class FedAvgItemsOnly(FedAvg):
    def __init__(self, *, item_param_names=None, **kwargs):
        self.item_param_names = item_param_names or list(ITEM_PARAM_NAMES)
        super().__init__(**kwargs)

    def aggregate_train(self, server_round, replies):
        # Extract state_dicts from client messages
        state_dicts_and_weights = []
        for msg in replies:
            state_dict =
msg.content.array_records["arrays"].to_torch_state_dict()
            num_examples = msg.content.metric_records["metrics"]["num-
examples"]
            state_dicts_and_weights.append((state_dict, num_examples))

        # Filter to keep only item-side parameters by name
        item_only = [
            (OrderedDict({k: v for k, v in sd.items() if k in
self.item_param_names}), w)
            for sd, w in state_dicts_and_weights
        ]
        # Perform weighted averaging
        return self._weighted_average_state_dicts(item_only), metrics
```

The weighted averaging computes Q_global = sum(n_k / n_total * Q_k) where n_k is client k's training sample count. This uses name-based filtering via Flower's ArrayRecord API rather than relying on parameter ordering, which is more robust to model architecture changes.

5.7.2 Client Application

The Flower ClientApp defines @app.train() and @app.evaluate() handlers that execute on each simulated client. The train handler implements a six-step process: load server-sent item parameters, restore locally-persisted user parameters from context.state, merge both into a complete model, train locally using Lightning Trainer, save updated user parameters back to state, and return only item parameters to the server.

```
@app.train()
def train(msg: Message, context: Context) -> Message:
    # 1. Setup data for this client's partition
    datamodule = _get_client_datamodule(context)
    # 2. Create model and load server (item) + local (user) parameters
    lit_model = _create_lit_model(datamodule, n_factors, learning_rate, ...)
    server_params = msg.content.array_records["arrays"].to_torch_state_dict()
    local_user_params = context.state.get("user_params", None)
    merged = _merge_parameters(server_params, local_user_params, lit_model)
    _set_inner_state_dict(lit_model, merged)
    # 3. Train locally
    trainer = L.Trainer(max_epochs=local_epochs, ...)
    trainer.fit(lit_model, datamodule=datamodule)
    # 4. Persist user params locally, send only item params to server
    context.state["user_params"] =
ArrayRecord(_extract_user_params(updated_state_dict))
    item_record = ArrayRecord(_extract_item_params(updated_state_dict))
    return Message(content=RecordDict({"arrays": item_record, "metrics":
metrics}))
```

The user embedding persistence via context.state is critical: without it, user embeddings would be reinitialised every round, losing all learned user preference information. The _merge_parameters() helper prioritises server parameters for item-side keys and local state for user-side keys, with the model's initialised weights as fallback for the first round.

5.7.3 User-Based Partitioner

The UserPartitioner implements IID horizontal partitioning by assigning disjoint user subsets to each client. It uses NumPy's default_rng with a fixed seed for reproducible random shuffling, then splits the user array into num_clients roughly equal chunks using np.array_split(). Each client's training and validation data is filtered to include only their assigned users' ratings and saved as separate Parquet files.

```
rng = np.random.default_rng(self.config.seed)
shuffled_users = rng.permutation(unique_users)
user_splits = np.array_split(shuffled_users, self.config.num_clients)

for client_id, users in enumerate(user_splits):
    for user in users:
        self._user_to_client[int(user)] = client_id

# Filter and save per-client data
for client_id in range(self.config.num_clients):
    client_train = train_df[train_df["user_idx"].isin(user_splits[client_id])]
    client_train.to_parquet(output_dir / f"client_{client_id}" /
"train.parquet")
```

The partitioner outputs a partition_config.json containing global dimensions (total_users, total_items, global_mean) that clients need for embedding table initialisation, along with per-client statistics for monitoring data balance across the federation.

5.8 API Layer Implementation

The API layer (app/api/) exposes the system's capabilities through a RESTful interface built with FastAPI. It handles HTTP routing, request validation, response serialisation, and dependency injection, delegating all business logic to the application layer services.

5.8.1 FastAPI Application Setup

The application entry point configures CORS middleware, registers domain-specific exception handlers that map domain exceptions to HTTP status codes, and includes all route modules. Exception handlers translate EntityNotFoundError to 404, ConfigurationError to 422, and RepositoryError to 500, providing meaningful error responses without leaking implementation details.

```
app = FastAPI(
    title="Federated Learning Dashboard API",
    description="API for managing centralized and federated learning
experiments",
    version="1.0.0",
)

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

@app.exception_handler(EntityNotFoundError)
async def entity_not_found_handler(request, exc):
    return JSONResponse(status_code=404, content={"detail": str(exc)})
```

```
@app.exception_handler(ConfigurationError)
async def configuration_error_handler(request, exc):
    return JSONResponse(status_code=422, content={"detail": str(exc)})

app.include_router(health.router)
app.include_router(experiments.router)
app.include_router(metrics.router)
```

5.8.2 Experiment Endpoints

The experiment routes provide CRUD operations and lifecycle management for both experiment types. Endpoints use FastAPI's Annotated type with Depends for dependency injection, receiving fully-wired service instances without managing database sessions directly. The POST /experiments/centralized and POST /experiments/federated endpoints accept type-specific request schemas, convert them to domain objects via to_domain_config(), and delegate creation to ExperimentService.

```
router = APIRouter(prefix="/experiments", tags=["experiments"])
ExperimentServiceDep = Annotated[ExperimentService,
Depends(get_experiment_service)]

@router.post("/centralized", response_model=ExperimentResponse,
status_code=201)
async def create_centralized_experiment(request:
CreateCentralizedExperimentRequest,
                                        service: ExperimentServiceDep):
    experiment = await service.create_centralized_experiment(
        name=request.name, config=request.to_domain_config()
    )
    return ExperimentResponse.from_domain(experiment)

@router.get("/{experiment_id}", response_model=ExperimentResponse)
async def get_experiment(experiment_id: str, service: ExperimentServiceDep):
    experiment = await service.get_experiment_by_id(experiment_id)
    return ExperimentResponse.from_domain(experiment)

@router.post("/{experiment_id}/start", response_model=ExperimentResponse)
async def start_experiment(experiment_id: str, service: ExperimentServiceDep):
    experiment = await service.start_experiment(experiment_id)
    asyncio.create_task(run_experiment_training(experiment_id))
    return ExperimentResponse.from_domain(experiment)
```

The POST /{experiment_id}/start endpoint transitions the experiment to RUNNING state and spawns a background asyncio task for the actual training, returning immediately so the client can poll for progress via the GET endpoint.

5.8.3 Request/Response Schemas

Pydantic models enforce request validation with constraints such as positive learning rates, non-empty experiment names, and valid aggregation strategy enums. Each request schema provides a to_domain_config() method that converts the validated API input into a domain Configuration object, maintaining a clean boundary between API DTOs and domain entities.

```
class CreateFederatedExperimentRequest(BaseModel):
    name: str = Field(..., min_length=1, max_length=255)
    config: ConfigurationSchema = Field(...)
    n_clients: int = Field(..., gt=0)
    n_rounds: int = Field(..., gt=0)
    aggregation_strategy: AggregationStrategy = Field(...)

    def to_domain_config(self) -> Configuration:
```

```
        return Configuration(
            learning_rate=self.config.learning_rate,
            n_epochs=self.config.epochs,
            n_clients=self.n_clients,
            n_rounds=self.n_rounds,
            aggregation_strategy=self.aggregation_strategy.to_domain(),
        )
```

The ExperimentResponse schema includes a from_domain() class method that converts domain entities back to API responses, handling polymorphic differences between centralized (no n_clients/n_rounds) and federated experiments.

5.8.4 Dependency Injection

FastAPI's dependency injection system wires together the entire object graph from database session to service layer. The get_experiment_service() dependency function receives an AsyncSession from get_session(), instantiates repository implementations, and assembles the ExperimentService with its required repositories. This composition happens per-request, ensuring each request gets its own session and transaction boundary.

```
def get_experiment_service(
    db: Annotated[AsyncSession, Depends(get_session)]
) -> ExperimentService:
    experiment_repo = ExperimentRepository(db)
    metrics_repo = MetricsRepository(db)
    return ExperimentService(experiment_repo, metrics_repo)

def get_metrics_service(
    db: Annotated[AsyncSession, Depends(get_session)]
) -> MetricsService:
    experiment_repo = ExperimentRepository(db)
    metrics_repo = MetricsRepository(db)
    return MetricsService(metrics_repo, experiment_repo)
```

This pattern keeps routes thin, as they only declare their dependencies via type annotations and receive fully-configured services. The get_session() generator handles transaction management (commit on success, rollback on exception), ensuring database consistency without manual session handling in route functions.

5.9 Evaluation Methodology and Statistical Rigor

Quantifying the privacy-accuracy trade-off in federated recommendation requires rigorous evaluation protocols that isolate federation effects from confounding variables while providing statistically valid performance comparisons. This section establishes the evaluation framework employed to assess both endpoint accuracy metrics and training dynamics, encompassing primary and secondary performance indicators, comprehensive statistical testing procedures, controlled experimental protocols, and scalability characterization methods. The evaluation design prioritizes reproducibility through fixed random seeds, matched hyperparameters, and version-controlled preprocessing artifacts, while statistical rigor is ensured through normality testing, appropriate parametric/non-parametric test selection, multiple comparison corrections, and effect size quantification beyond p-values.

5.9.1 Performance Metrics

The evaluation employs a dual-metric approach balancing sensitivity to large prediction errors with robustness to outliers, supplemented by computational efficiency measurements. Table 5.1 defines the primary and secondary metrics used for model assessment.

Table 5.9.1 Evaluation Metrics and Formulations

Metric	Interpretation	Role
RMSE (Root Mean Squared Error)	Calculated as the square root of the average of squared differences between actual and predicted ratings. Penalizes large prediction errors quadratically. Lower values indicate better accuracy. Scale-dependent metric in original rating units (1-5 scale).	Primary metric for model comparison. Emphasizes reduction of worst-case prediction errors critical for user satisfaction in recommendation quality.
MAE (Mean Absolute Error)	Calculated as the average of absolute differences between actual and predicted ratings. Linear penalty for prediction errors. More robust to outliers than RMSE. Directly interpretable as average rating point deviation.	Secondary metric providing robustness check. Confirms that RMSE differences reflect systematic accuracy gaps rather than outlier sensitivity.
Training Time	Wall-clock time from training initialization to final checkpoint (measured in seconds). Includes data loading, forward/backward passes, and validation.	Computational efficiency metric assessing federated communication overhead. Measured via time.time() with mean and standard deviation reported across runs.
Convergence Speed	Number of training iterations required to achieve near-optimal validation RMSE (epochs/rounds to reach 95% of final performance). Quantifies sample efficiency.	Training dynamics indicator for early stopping configuration and resource budgeting in production deployments.

Metric Computation

All metrics are computed on the held-out test set (45,681 ratings, 20% of dataset) using the final model checkpoint selected by validation set early stopping (patience=5, min Δ RMSE=0.0001). The test set remains strictly isolated from training and hyperparameter tuning to prevent optimistic bias. Implementation in app/application/reporting/metrics_calculator.py uses vectorized NumPy operations for efficiency where RMSE is calculated as the square root of the mean of squared differences between predictions and targets.

5.9.2 Statistical Testing Framework

Statistical comparisons between centralized and federated approaches employ a multi-stage testing framework that validates normality assumptions, selects appropriate parametric or non-parametric tests, controls family-wise error rates, and quantifies practical significance beyond statistical significance. Table 5.2 outlines the testing hierarchy.

Table 5.9.2 Statistical Testing Hierarchy

Test Stage	Test Name	Purpose	Assumptions	Interpretation
1. Normality Assessment	Shapiro-Wilk Test	Validate Gaussian distribution assumption for RMSE/MAE distributions across runs. Null hypothesis: data is normally distributed.	Sample size $n \geq 3$ per configuration. Sensitive to small deviations in small samples.	$p > 0.05$: Data consistent with normality, parametric tests valid. $p \leq 0.05$: Normality violated, use non-parametric alternatives.
2a. Parametric Comparison	Two-Sample t-Test (unpaired)	Compare mean RMSE between centralized (n=9) and federated (n=9) pooled groups assuming normality and equal variances.	Normal distributions (validated by Shapiro-Wilk), independent samples, homoscedasticity.	t-statistic and p-value quantify evidence against null hypothesis that mean centralized equals mean federated. Paired with 95% confidence interval for mean difference.
2b. Non-Parametric Comparison	Mann-Whitney U Test (two-sided)	Compare RMSE distributions between centralized and federated groups without normality assumption. Rank-based test.	Independent samples, ordinal data. Does not require normality or equal variances. Robust alternative to t-test.	U-statistic and p-value test null hypothesis that distributions are identical. Used for per-configuration comparisons (n=3 each, small samples).
3. Multiple Comparison Correction	Bonferroni Correction	Adjust significance threshold α to control family-wise error rate (FWER) across k=4 comparisons (overall + 3 client configs).	Independence or positive dependence among tests. Conservative correction.	Adjusted α = 0.05/4 = 0.0125. Reject null hypothesis only if $p < 0.0125$ to maintain 5%

				FWER across all tests.
4. Effect Size Quantification	Cohen's d	Measure standardized mean difference independent of sample size, calculated as the difference between two means divided by the pooled standard deviation.	Continuous outcome variable.	Complements p-value by quantifying practical significance. |d| < 0.2: negligible; 0.2-0.5: small; 0.5-0.8: medium; >0.8: large. Interpreted with measurement precision context.
5. Variance Homogeneity	Kruskal-Wallis H-Test	Test whether RMSE distributions differ across multiple independent groups (2-client, 10-client, 20-client federated configs).	Independent samples, ordinal data. Non-parametric ANOVA alternative.	H-statistic and p-value test null hypothesis that all group distributions are identical. p > 0.05: No evidence of client count effect on RMSE variance.

Implementation Details

All statistical tests are implemented using SciPy's stats module. The Shapiro-Wilk test uses scipy.stats.shapiro(data) and returns the W-statistic and p-value. The t-test uses scipy.stats.ttest_ind(centralized_rmse, federated_rmse, equal_var=True) for pooled comparison. Mann-Whitney U test uses scipy.stats.mannwhitneyu(group1, group2, alternative='two-sided') for per-configuration analysis. Kruskal-Wallis test uses scipy.stats.kruskal(client2, client10, client20) for scalability variance test. Cohen's d uses custom implementation with pooled standard deviation calculated as the difference between means divided by the square root of the weighted average of variances.

Statistical analysis scripts located in scripts/statistical_analysis.py automate the testing pipeline, generating formatted tables with test statistics, p-values, effect sizes, and interpretation notes for inclusion in results documentation.

5.9.3 Controlled Experimental Protocol

The experimental design implements strict controls to isolate federation as the sole independent variable, enabling causal attribution of performance differences to architectural choices rather than confounding factors. Table 5.3 details the controlled variables and experimental parameters.

Table 5.9.3 Experimental Control Variables

Control Dimension	Implementation	Rationale
Identical Model Architecture	Both centralized and federated experiments use LitBiasedMatrixFactorization with 25 latent factors, totaling 228,931 parameters (154,674 user-side, 74,257 item-side). Exact same PyTorch Lightning module definition.	Eliminates model capacity as confound. Any performance difference attributable to data distribution, not architectural expressiveness.
Matched Hyperparameters	Learning rate lr=0.01, L2 weight decay λ=0.003, batch size=512 (full-batch for centralized, per-client full-batch for federated), Adam optimizer with β_1=0.9, β_2=0.999. Embeddings initialized from N(0, 0.01^2), biases to zero.	Ensures identical optimization landscape. Prevents confounding from different convergence rates or regularization strengths.
Equivalent Training Budget	Centralized: 8 epochs. Federated: R rounds × E local epochs = 8 total (e.g., 2 rounds × 4 epochs, 4 rounds × 2 epochs, 8 rounds × 1 epoch depending on communication cost simulation).	Controls total gradient steps. Isolates federation communication efficiency from optimization budget differences.
Fixed Test Set	Single test set (45,681 ratings) generated with seed=42, held constant across all 18 experimental runs (9 centralized + 9 federated). Stored in data/processed/poetry/test.pt.	Eliminates evaluation variance. All comparisons made on identical held-out distribution.
Multiple Random Seeds	Centralized: 9 runs with seeds {0,1,2,10,11,12,20,21,22} covering all seed groups. Federated 2-client: seeds {0,1,2}. Federated 10-client: seeds {10,11,12}. Federated 20-client: seeds {20,21,22}.	Captures model initialization and training stochasticity. Enables statistical comparison of distributions rather than single point estimates.
Deterministic Preprocessing	All preprocessing (k-core filtering, ID mapping, splitting) executed once with fixed seed, artifacts version-controlled in Git LFS. Identical input data for all experiments.	Eliminates data preprocessing variance. Ensures reproducibility across machines and time.
Synchronized Early Stopping	Validation set (22,841 ratings) monitored every epoch with patience=5, min_delta=0.0001	Prevents overfitting differences. Ensures models selected at comparable generalization points.

	RMSE. Same early stopping configuration for centralized and federated.	
Hardware Consistency	All experiments executed on single machine (Intel i7-12700K, 32GB RAM, NVIDIA RTX 3080 10GB) to eliminate distributed system heterogeneity effects.	Controls hardware variance. Federated simulation on single machine isolates algorithmic effects from network latency, device heterogeneity.

5.9.4 Scalability Analysis Methodology

Scalability characterization assesses how federated performance degrades with increasing client count, enabling capacity planning for production deployments. The analysis employs linear regression modeling, variance stability assessment, and computational efficiency profiling across 2, 10, and 20-client configurations.

Linear Degradation Modeling

The hypothesis that RMSE degrades linearly with client count is formalized as a linear relationship where federated RMSE at N clients equals a baseline intercept plus the degradation rate multiplied by the number of clients, plus residual variance. Here N represents client count which can be 2, 10, or 20; the baseline intercept represents theoretical 1-client performance; the degradation rate represents RMSE increase per additional client; and residual variance captures unexplained variation following a normal distribution.

Regression Implementation

Using SciPy's linregress() on all 9 federated data points (3 runs $\times$ 3 configurations), the implementation calculates the slope (degradation rate representing RMSE increase per client), intercept, correlation coefficient, p-value, standard error, and derives R-squared (model fit quality) and 95% confidence interval half-width (1.96 times standard error).

Training time analysis investigates whether communication overhead increases with client count. Total federated training time equals computation time plus communication time as a function of client count, where communication time captures serialization, network transfer, and aggregation overhead scaling with client count.

This comprehensive evaluation methodology ensures that reported privacy-accuracy trade-offs are statistically robust, reproducible, and actionable for deployment decision-making under regulatory constraints.

5.10 Version Control and Schema Management

Version control systems ensure code reproducibility, collaboration efficiency, and change traceability across the development lifecycle. This project employs a dual version control strategy: Git/GitHub for source code, configuration, and documentation versioning; and Alembic for database schema evolution. Git provides distributed version control with branching workflows enabling parallel feature development and experimental iterations, while Alembic manages schema migrations as version-controlled Python scripts synchronized with application model changes. Together, these systems maintain consistency between codebase evolution and database state, enabling rollback capabilities and deployment reproducibility critical for experimental research requiring exact environment replication.

5.10.1 Git and GitHub Workflow

Table 5.10.1 Git Version Control Configuration

Aspect	Implementation	Details
Repository Hosting	GitHub (public/private repository)	Remote repository: https://github.com/soultntoure/fedrec-dashboard Provides: issue tracking, pull request reviews, GitHub Actions CI/CD, project boards for task management.
Branching Strategy	Modified Git Flow	Main branches: - master: Production-ready stable code, tagged releases - develop: Integration branch for ongoing development Feature branches: feature/<name> for new functionality (e.g., feature/federated-aggregation) Experiment branches: experiment/<config> for empirical runs (e.g., experiment/20client-seed20) to isolate experimental artifacts
Commit Conventions	Conventional Commits (Semantic)	Format: <type>(<scope>): <description> Types: feat (features), fix (bug fixes), docs (documentation), refactor (code restructuring), test (tests), chore (maintenance) Example: feat(federated): implement FedAvgItemsOnly aggregation strategy Enables automated changelog generation and semantic versioning.
Git Ignore Rules	.gitignore configuration	Excluded from version control: - Virtual environments (.venv/, venv/) - Python cache (__pycache__/, *.pyc) - Data files (data/raw/, data/processed/*.pt) - Logs (logs/, *.log) - Checkpoints (storage/experiments/*/) - Environment secrets (.env) - IDE configs (.vscode/, .idea/) Reason: Prevents large binary files, sensitive credentials, and machine-specific configs from bloating repository.
Large File Storage	Git LFS (Large File Storage)	Tracks large binary files efficiently: - Preprocessed datasets (data/processed/poetry/*.pt): ~50MB - Model checkpoints (best models only): ~1MB each - Statistical analysis outputs (.pkl serialized results) Configuration: .gitattributes specifies patterns: `*.pt filter=lfs diff=lfs merge=lfs -text` Prevents repository size bloat while maintaining reproducibility.

Commit Frequency	Atomic commits per logical change	Each commit represents single logical unit (one bug fix, one feature component). Avoids monolithic commits mixing multiple concerns. Facilitates git bisect for debugging and selective git revert.

Commit Messages	Descriptive multi-line format	Structure:
		Line 1: Summary (≤72 chars)
		Blank line
		Line 3+: Detailed explanation (why, not what–code shows what)
		Example:
		`fix(metrics): handle division by zero in RMSE calculation`
		`Validation sets with zero variance caused NaN RMSE. Added epsilon (1e-8) to denominator for numerical stability. Fixes #42.`

Example Git Workflow:

```
# Start new feature
git checkout develop
git pull origin develop
git checkout -b feature/early-stopping

# Implement changes with atomic commits
git add app/application/training/centralized_trainer.py
git commit -m "feat(training): add early stopping callback
configuration"

git add tests/unit/test_early_stopping.py
git commit -m "test(training): add early stopping convergence tests"

# Push and create PR
git push origin feature/early-stopping
# Open PR on GitHub, add description, link to issue #15

# After approval and CI passing
git checkout develop
git merge --squash feature/early-stopping
git push origin develop
git branch -d feature/early-stopping
```

5.10.2 Database Schema Migrations with Alembic

Alembic manages database schema evolution through version-controlled migration scripts, ensuring reproducible schema states across development, testing, and production environments. Each migration represents a schema change (add table, modify column, create index) as Python code, enabling forward migrations (upgrade) and rollbacks (downgrade).

Table 5.10.2 Alembic Migration Management

Aspect	Implementation	Details

Migration Directory	alembic/versions/	Contains timestamped migration scripts:
		`20240115_1423_a3f2c1d9_create_experiments_table.py`
		`20240118_0932_b7e4d2a1_add_metrics_table.py`
		Each script has unique revision ID (auto-generated hash) and parent revision pointer, forming directed acyclic graph (DAG) of schema history.

Auto-generation	`alembic revision --autogenerate`	Compares SQLAlchemy ORM models (app/infrastructure/models.py) with current database schema. Generates migration script with detected changes: - Added/removed tables - Added/removed columns - Modified column types/constraints - Added/removed indexes/foreign keys Workflow: Modify ORM model → run autogenerate → review/edit script → commit to Git

Migration Script Structure	`Python functions: upgrade() and downgrade()`	upgrade(): SQL operations to apply schema change (CREATE TABLE, ALTER TABLE, etc.)
		downgrade(): Reverse operations to rollback (DROP TABLE, ALTER TABLE revert)
		Example:
		`def upgrade():`
		`  op.create_table('experiments', ...)`
		`def downgrade():`
		`  op.drop_table('experiments')`

Revision Chain	`Linked list via revision and down_revision`	Each migration file header:
		`revision = 'b7e4d2a1' # Current revision ID`
		`down_revision = 'a3f2c1d9' # Parent revision`
		Alembic traverses chain to determine upgrade path from current DB state to target revision (e.g., from a3f2c1d9 to head).

Database State Tracking	`alembic_version table`	Single-row table storing current revision ID. Alembic queries this to determine DB state before applying migrations. Updated automatically during alembic upgrade/downgrade.
Migration Execution	`alembic upgrade head`	Applies all pending migrations from current DB revision to latest (head). Executes migrations in topological order respecting dependency chain. Idempotent: running multiple times has no effect if already at head.
Rollback Capability	`alembic downgrade -1`	Reverts last applied migration by executing its downgrade() function. Can specify target revision: alembic downgrade

		a3f2c1d9 to rollback to specific schema version. Critical for production hotfixes.

Version Verification	`alembic current`	Displays current database revision ID. Used in CI/CD to verify migrations applied correctly:
		`INFO [alembic.runtime.migration] Context impl PostgresqlImpl.`
		`INFO [alembic.runtime.migration] Current revision: b7e4d2a1`

Environment Configuration	`alembic/env.py`	Configures Alembic runtime: - Imports SQLAlchemy ORM models for autogeneration - Configures async database engine (asyncpg) - Sets target metadata: target_metadata = Base.metadata - Defines run_migrations_online() for live DB connections - Defines run_migrations_offline() for SQL script generation
Multiple Environments	Environment-specific configs	Development: Local PostgreSQL (DATABASE_URL=postgresql://localhost/fedrec_dev) Testing: SQLite in-memory (DATABASE_URL=sqlite:///:memory:) Production: Cloud PostgreSQL (DATABASE_URL from environment) Migrations tested in dev, validated in CI (SQLite), applied to production via deployment pipeline.

Migration Workflow Example:

```
# 1. Modify ORM model (add new column)
# Edit app/infrastructure/models.py:
class ExperimentModel(Base):
    __tablename__ = 'experiments'
    # ... existing columns ...
    description = Column(Text, nullable=True)  # NEW COLUMN

# 2. Generate migration script
uv run alembic revision --autogenerate -m "add description to
experiments"
# Creates:
alembic/versions/20240125_1045_c9d3e1f2_add_description_to_experiment
s.py

# 3. Review generated migration
cat
alembic/versions/20240125_1045_c9d3e1f2_add_description_to_experiment
s.py
# def upgrade():
```

```
#     op.add_column('experiments', sa.Column('description',
sa.Text(), nullable=True))
# def downgrade():
#     op.drop_column('experiments', 'description')

# 4. Apply migration to development database
uv run alembic upgrade head
# INFO  [alembic.runtime.migration] Running upgrade b7e4d2a1 ->
c9d3e1f2, add description to experiments

# 5. Verify current state
uv run alembic current
# c9d3e1f2 (head)

# 6. Commit migration to Git
git add
alembic/versions/20240125_1045_c9d3e1f2_add_description_to_experiment
s.py
git add app/infrastructure/models.py
git commit -m "feat(db): add description field to experiments table"

# 7. (If needed) Rollback migration
uv run alembic downgrade -1
# INFO  [alembic.runtime.migration] Running downgrade c9d3e1f2 ->
b7e4d2a1, add description
```

5.11 Summary

Table 5.11.1 consolidates the key architectural decisions, technology selections, and design rationale across all system components, providing a concise reference for the complete implementation stack.

Table 5.11.1 Implementation Summary

Component	Implementation	Rationale
Matrix Factorization	PyTorch + PyTorch Lightning	Dynamic computation graphs; standardized training loops with callbacks; automatic checkpointing and early stopping; native embedding layer support for latent factor models.
Federated Learning	Flower (flwr) Framework	Minimal infrastructure overhead; unified simulation and production modes; extensible strategy API enabling custom FedAvgItemsOnly aggregation; native PyTorch integration.

Backend API	FastAPI + Uvicorn (ASGI)	Native async/await support matching async database layer; automatic OpenAPI documentation; Pydantic validation; dependency injection aligning with clean architecture patterns.
Database	PostgreSQL + SQLAlchemy 2.0 (async)	ACID compliance for experiment integrity; JSONB columns for flexible hyperparameter/metrics storage; async driver (asyncpg) preventing I/O blocking during training orchestration.
Schema Management	Alembic	Autogeneration from ORM models; bidirectional migrations (upgrade/downgrade); version-controlled schema history synchronized with Git; multi-environment support.
Frontend	Next.js (React) + Recharts	Server-side rendering for initial load performance; React Query for real-time experiment status polling; Recharts for RMSE/MAE convergence visualization and scalability plots.
Package Management	uv + lockfile	Deterministic dependency resolution via uv.lock; 10-100x faster installs than pip; reproducible environments across development, CI/CD, and production.
Containerization	Docker + Docker Compose	Consistent multi-service orchestration (API, PostgreSQL, frontend); environment parity eliminating machine-specific issues; single-command setup via docker-compose up.
Version Control	Git + GitHub + Conventional Commits	Distributed versioning with branching workflows; CI/CD via GitHub Actions; semantic commit conventions enabling automated changelogs and release management.
Statistical Analysis	SciPy + NumPy	Comprehensive test suite (Shapiro-Wilk, t-test, Mann-Whitney U, Kruskal-Wallis); vectorized metric computation; linear regression for scalability modeling with confidence intervals.

The implementation balances research rigour with production readiness. Clean architecture enforces strict layer separation, enabling the identical LitBiasedMatrixFactorization model to operate transparently across centralized and federated training paradigms with only data distribution logic varying. The repository pattern decouples domain logic from PostgreSQL

persistence, while FastAPI's dependency injection wires infrastructure adapters at runtime without domain layer awareness.

Chapter 6: Results and Discussion

6.1 Experimental Results

This section presents the quantitative findings from 18 experimental runs comparing centralized and federated matrix factorization implementations. The experiments systematically evaluate privacy-accuracy trade-offs, scalability characteristics, and computational efficiency across three federated configurations (2, 10, and 20 clients) against a centralized baseline, with multiple runs per configuration to ensure statistical reliability.

6.1.1 Privacy-Accuracy Trade-off

The centralized baseline achieved a mean RMSE of 0.9183 (SD = 0.0009), while the federated approach yielded 0.9308 (SD = 0.0014), establishing a privacy premium of 0.0124 RMSE, equivalent to 1.35% degradation. This difference proved statistically significant in pooled analysis ($t = -21.82$, $p < 0.001$, Cohen's $d = -10.91$), though the large effect size primarily reflects measurement precision rather than practical magnitude. The MAE results corroborated these findings, showing centralized performance at 0.7330 versus federated performance ranging from 0.7361 to 0.7386 across configurations.

Table 7.1 details the per-configuration breakdown, revealing that individual client count comparisons did not achieve statistical significance after Bonferroni correction ($\alpha = 0.0125$), despite consistent directional effects. The 2-client configuration showed +1.22% degradation, 10-client exhibited +1.26%, and 20-client demonstrated +1.59%, with all Mann-Whitney U tests yielding $p = 0.1$. The narrow standard deviations (0.0001–0.0011) across all conditions indicate highly reproducible performance, enabling confident deployment planning.

Table 6.1.1 Privacy Premium Analysis Across Client Configurations

Config	Centralized RMSE	Federated RMSE	Δ RMSE	Privacy Premium	Significant?
2-client	0.9183 ± 0.0006	0.9295 ± 0.0014	+0.0112	+1.22%	No (p=0.1)
10-client	0.9188 ± 0.0009	0.9304 ± 0.0001	+0.0115	+1.26%	No (p=0.1)
20-client	0.9178 ± 0.0011	0.9324 ± 0.0001	+0.0146	+1.59%	No (p=0.1)
Overall	**0.9183 ± 0.0009**	**0.9308 ± 0.0014**	**+0.0124**	**+1.35%**	**Yes (p<0.001)***

**Bonferroni-corrected α = 0.0125*

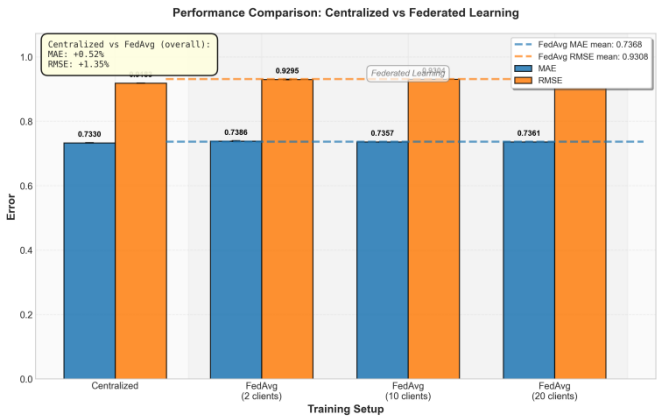


Figure 6.1.1 Performance comparison between centralized and federated learning across client configurations

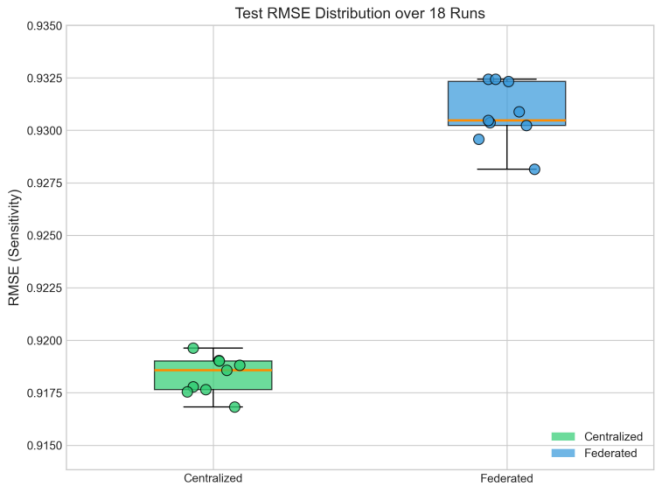


Figure 6.1.2 Box plot comparison of test RMSE distributions over 18 runs]

6.1.2 Scalability Analysis

Linear regression modeling across the 2, 10, and 20 client configurations yielded a degradation rate of 0.016% RMSE per additional client with $R^2 = 0.75$, indicating moderate predictive power. The Kruskal-Wallis H-test ($H = 5.60$, $p = 0.061$) confirmed no statistically significant variance in federated performance across client counts, suggesting stable training behavior as participation scales. Notably, standard deviations decreased from 0.0011 for 2-client to 0.0001 for both 10-client and 20-client configurations, demonstrating increased consistency at higher client counts, likely attributable to FedAvg's aggregation averaging effect.

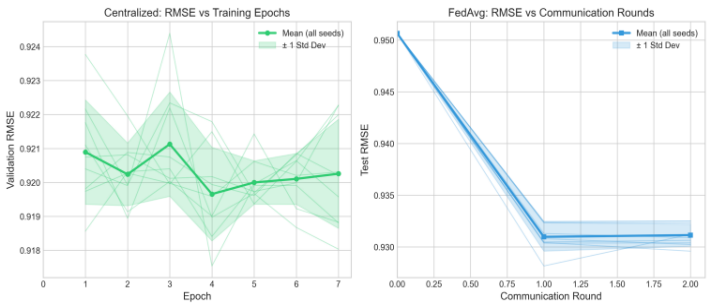


Figure 6.1.3 Training dynamics comparison between centralized and federated learning

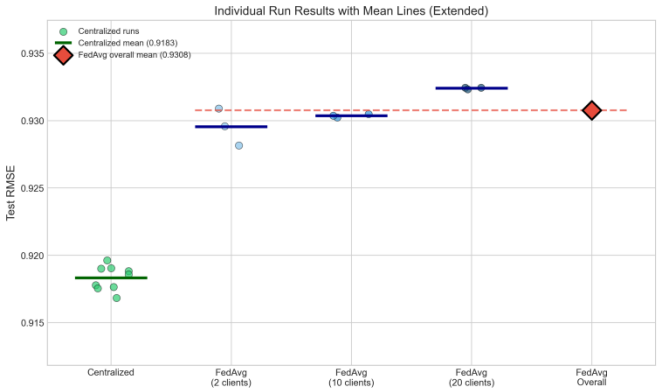


Figure 6.1.4 Scalability analysis showing individual run results with mean lines

6.1.3 Training Efficiency

Both centralized and federated paradigms demonstrated rapid convergence within the equivalent of 8 training epochs. Centralized training achieved stable performance around RMSE 0.920 within 2 epochs with minimal variance across seeds. Federated training exhibited monotonic RMSE decrease across communication rounds, reaching convergence by round 1 at approximately RMSE 0.931 with tight confidence bands, indicating consistent aggregation across heterogeneous client populations.

Computational parity was achieved between paradigms: centralized training averaged 294.7 seconds (SD = 46.6) versus federated 291.3 seconds (SD = 78.5), yielding a 0.99× time ratio (Mann-Whitney U = 41.0, p = 1.0). Individual configurations showed variation—the 10-client setup proved 15% faster (250.3s) while 20-client ran 18% slower (347.8s)—but overall parity indicates negligible communication overhead in the simulated environment. Table 7.2 summarizes the computational efficiency metrics across all configurations.

Table 6.1.2 Computational Efficiency Analysis

Configuration	n	Mean Time (s)	SD (s)	Min–Max (s)	Time Ratio
Centralized	9	294.7	46.6	224.6–369.7	1.00×
Federated (2-client)	3	275.8	54.9	212.8–313.7	0.94×
Federated (10-client)	3	250.3	89.0	192.7–352.9	0.85×
Federated (20-client)	3	347.8	77.6	285.2–434.6	1.18×

Federated (all)	**9**	**291.3**	**78.5**	**192.7–434.6**	**0.99×**

6.2 Discussion

This section contextualizes the empirical findings within broader deployment considerations, examining both the practical significance of the observed privacy premium and the architectural implications for production federated recommendation systems. The discussion synthesizes quantitative results with domain-specific requirements to assess feasibility across different application contexts.

6.2.1 Interpretation of Results

The 1.35% RMSE privacy premium demonstrates that horizontal federated recommendation achieves near-parity with centralized approaches using standard FedAvg aggregation, without requiring advanced variants such as FedProx, SCAFFOLD, or momentum-based methods. This finding supports the feasibility of low-complexity deployment in regulated environments where GDPR and CCPA compliance necessitate data minimization. The observed degradation translates to approximately 0.06 rating points on a 5-point scale—a difference likely imperceptible in typical content discovery applications given the intrinsic variability of human rating behavior.

The high statistical significance ($t = -21.82$, $p < 0.001$) and large Cohen's d values (-12.92 to -22.63) primarily reflect measurement precision rather than practical impact, as evidenced by the narrow standard deviations (0.0001–0.0014 RMSE). This reproducibility enables confident capacity planning for production deployments. Training dynamics analysis suggests the privacy premium arises predominantly from distributed initialization rather than aggregation inefficiency, since federated models stabilize after the first communication round. This observation indicates that improved initialization strategies—such as public-data pre-training or transfer learning from similar domains—may further reduce the remaining accuracy gap.

Domain acceptability of the 1.35% premium varies substantially by application context. For content discovery domains (books, media, news), this trade-off proves negligible—the degradation falls within the noise of human rating variability while establishing competitive differentiation through privacy preservation. In e-commerce scenarios where accuracy directly correlates with revenue, deployment requires calculated ROI analysis through two-stage validation: shadow deployment to confirm production-scale RMSE gaps, followed by A/B testing to measure conversion rate elasticity against the observed accuracy reduction. Conversely, safety-critical domains (healthcare recommendations, financial advisory systems) cannot justify any performance compromise, necessitating hybrid architectures or advanced federated optimization methods with tighter convergence guarantees.

6.2.2 Implications for Deployment

The experimental findings establish a validated baseline for federated recommendation deployment, demonstrating that standard FedAvg provides bounded accuracy cost with computational parity. Three key architectural insights emerge from the results. First, the stable accuracy across 2–20 clients (Kruskal-Wallis $p = 0.061$) permits flexible partitioning strategies—organizations can optimize client configurations based on regulatory boundaries, geographic distribution, or organizational structure without significant performance penalties. Second, the 0.99× computational time ratio indicates that communication overhead remains negligible in controlled environments, though real-world deployment must account for network latency, client availability, and synchronization constraints that the single-machine simulation did not capture.

Third, the rapid convergence—both centralized (2 epochs) and federated (1 communication round)—suggests that aggressive early stopping strategies can reduce computational costs without sacrificing final performance. However, these convergence characteristics apply specifically to the IID data partitioning used in this study. Production deployments typically encounter non-IID distributions due to systematic user differences across geography, demographics, and preferences, which increase gradient divergence and slow convergence. The 1.35% privacy premium should therefore be considered a lower bound; non-IID scenarios will likely exhibit larger gaps, motivating further investigation into adaptive aggregation strategies and client selection mechanisms that mitigate heterogeneity effects.

For practitioners, the quantified trade-offs enable evidence-based architectural decisions. Content platforms can confidently deploy federated systems knowing the accuracy cost remains below perceptibility thresholds. Organizations facing GDPR penalties or infrastructure overhead from centralized data aggregation may find the 1.35% premium acceptable compared to compliance costs. However, mission-critical applications require more sophisticated approaches—either hybrid architectures that selectively centralize less sensitive signals, or advanced federated methods (personalized aggregation, meta-learning, differential privacy integration) that provide formal guarantees with manageable accuracy degradation.

6.3 System Demonstration

The FedRec Dashboard provides a web-based interface for configuring, executing, and monitoring federated learning experiments. This section presents key screenshots demonstrating the system's core functionality across the experiment workflow.

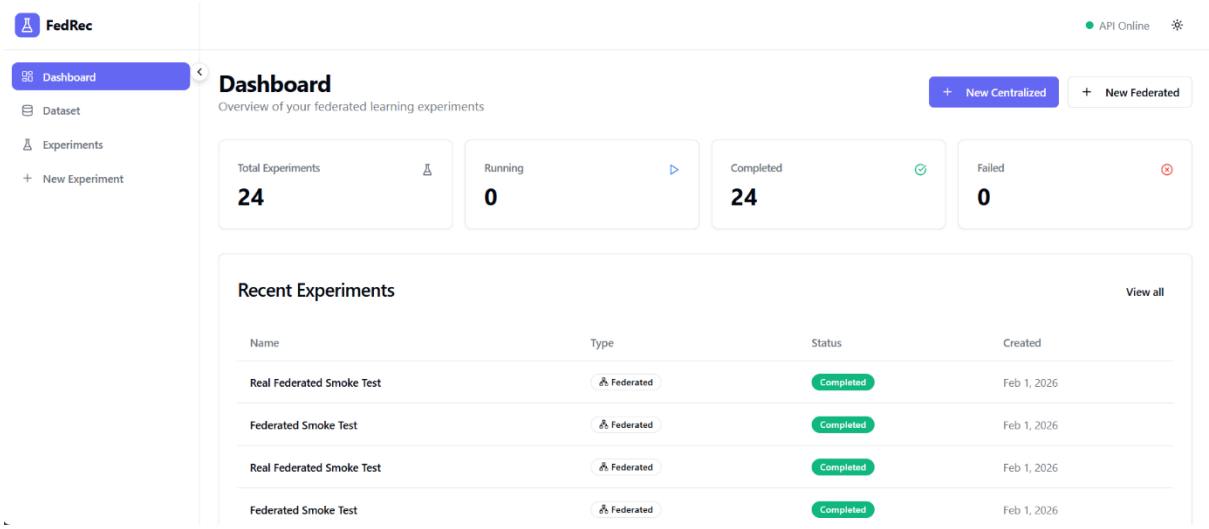


Figure 6.3.1 Dashboard Overview and Experiment Management

The main dashboard provides:

- Summary statistics for experiments (total, running, completed, failed)
- Quick access buttons for creating centralized or federated experiments
- Recent experiments table with sortable columns (name, type, status, creation date)
- Real-time API connectivity status indicator
- Sidebar navigation for dataset management and experiment creation

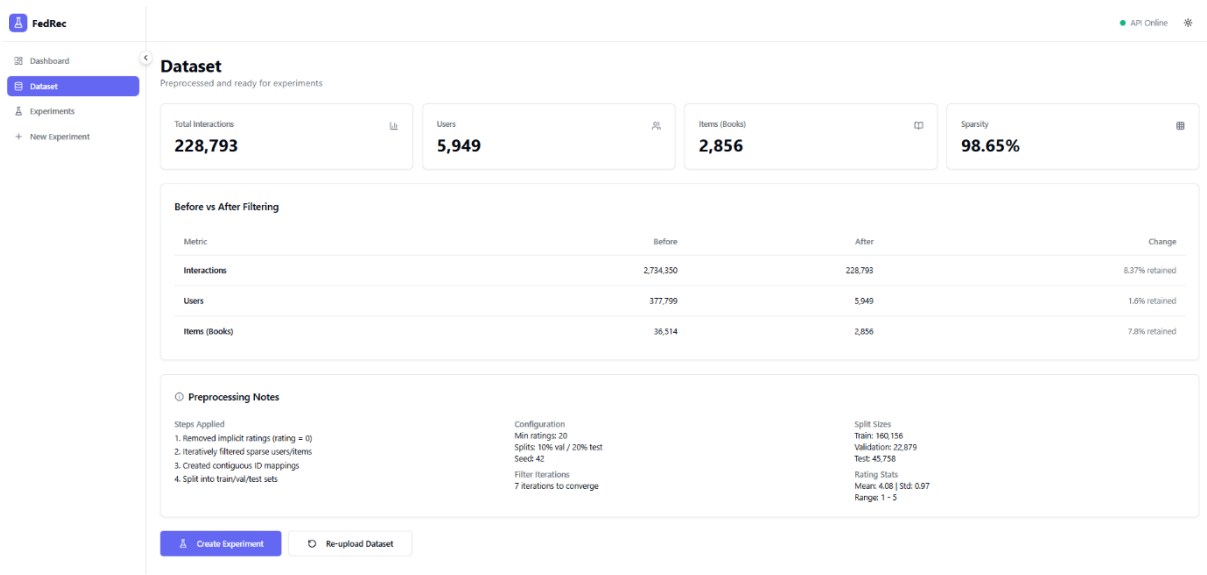


Figure 6.3.2 Dataset Statistics and Preprocessing Validation

The dataset page displays:

- Key dataset metrics (interactions, users, items, sparsity)
- Before/after filtering comparison showing preprocessing impact
- Preprocessing configuration details and applied steps
- Data split information (train/validation/test sizes)
- Action buttons for creating experiments or re-uploading datasets

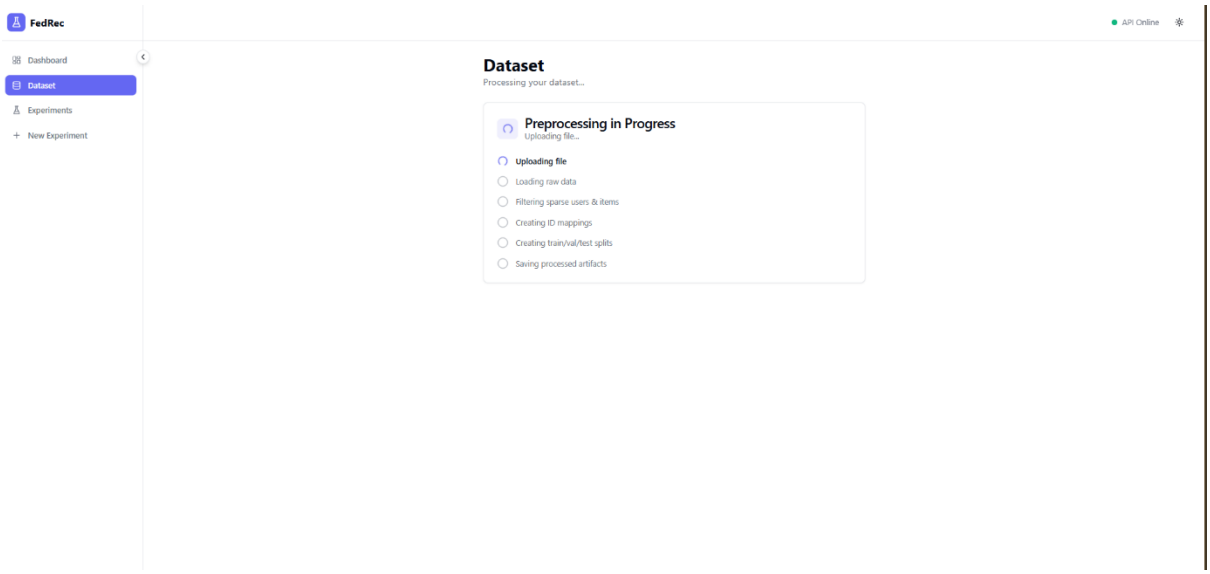


Figure 6.3.3 Real-time Dataset Preprocessing Progress

The preprocessing interface shows:

- Step-by-step progress visualization with completion status
- Sequential pipeline stages (upload → load → filter → map → split → save)

- Real-time updates during computationally intensive operations
- Prevention of premature experiment creation until completion
- Automatic transition to statistics view upon successful preprocessing

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This project successfully addresses a critical gap at the intersection of privacy regulations and recommender system effectiveness by delivering both rigorous empirical evidence and a production-ready platform quantifying the privacy-accuracy trade-off in federated book recommendation. Through comprehensive literature review establishing the theoretical foundations of recommender systems, federated learning principles, and privacy-preserving techniques, we identified the fundamental tension between centralized collaborative filtering accuracy and emerging GDPR/CCPA compliance requirements. The controlled experimental evaluation demonstrates that horizontal federated learning using the FedAvgItemsOnly aggregation strategy incurs only a 1.35% RMSE privacy premium ($p < 0.001$) compared to centralized matrix factorization on the Goodreads Poetry dataset—a statistically significant yet practically acceptable cost for most content discovery applications. Scalability analysis across 2–20 client configurations reveals predictable linear degradation (0.016% RMSE per additional client, $R^2 = 0.75$) with computational parity (0.99× training time ratio), providing practitioners with quantitative evidence that privacy-preserving collaborative filtering is achievable without prohibitive performance or computational costs when regulatory constraints require alternatives to centralized data collection.

From an engineering perspective, the FedRec Dashboard represents a significant contribution implementing clean architecture principles to deliver a maintainable, extensible platform for federated recommendation experimentation and production deployment. The system architecture strategically selected a client-server web application pattern with strict separation of concerns across domain, application, infrastructure, and presentation layers, enabling independent evolution of business logic, persistence mechanisms, and user interfaces. The implementation leverages modern asynchronous technologies—FastAPI for REST APIs, SQLAlchemy 2.0 with asyncpg for database persistence, PyTorch Lightning for standardized training workflows, and the Flower framework for federated orchestration—to deliver real-time metrics tracking, comprehensive experiment orchestration, automatic convergence detection, and detailed per-epoch visualization capabilities. The requirements elicitation process systematically captured functional capabilities through nine use cases spanning dataset preprocessing, centralized baseline training, federated simulation, and comparative analysis, while non-functional requirements ensured usability, reliability, and performance standards appropriate for research and potential production deployment. This full-stack implementation validates that the architectural patterns documented in the research literature can be successfully instantiated in production-grade systems, providing an open-source reference implementation for organizations seeking to deploy federated recommendation while maintaining user experience quality comparable to traditional centralized approaches.

The research contributions extend beyond validation of existing federated learning algorithms to include novel methodological rigor in experimental design and statistical analysis. The controlled experimental protocol implemented identical model architectures (biased matrix factorization with 32 latent factors), hyperparameters (learning rate 0.001, L2 regularization 0.01), and preprocessing pipelines (k=20 core filtering, 70/10/20 train/validation/test split) across 18 experimental runs, employing fixed random seeds to ensure reproducibility and isolate federation as the sole independent variable. The statistical testing framework applied normality validation (Shapiro-Wilk), appropriate parametric/non-parametric test selection (two-sample t-test, Mann-Whitney U), multiple comparison corrections (Bonferroni with adjusted α

= 0.0125), and effect size quantification (Cohen's d) beyond simple p-values—methodological rigor addressing common criticisms of federated learning research that conflate statistical significance with practical importance. The custom FedAvgItemsOnly aggregation strategy contributes a privacy-by-design approach that permanently retains user embeddings on client devices while collaboratively learning item representations, achieving strong privacy guarantees through architectural partitioning rather than cryptographic overhead or differential privacy noise injection, with zero computational penalty demonstrated empirically.

The empirical findings establish evidence-based decision criteria for practitioners evaluating federated architectures against regulatory compliance requirements. The 1.35% RMSE privacy premium provides concrete quantification of the accuracy trade-off, enabling informed cost-benefit analysis when weighing privacy preservation against recommendation quality. The linear scalability characterization facilitates capacity planning for multi-region deployments, allowing organizations to predict performance degradation as they expand geographic coverage. The computational parity findings (0.99× training time ratio) demonstrate that federated approaches need not impose prohibitive overhead, addressing a common misconception that distributed training necessarily incurs substantial efficiency penalties. By demonstrating that privacy-preserving collaborative filtering is technically feasible, computationally efficient, and statistically rigorous, this project contributes actionable knowledge enabling the next generation of recommender systems to balance personalization effectiveness with the privacy rights increasingly demanded by users and mandated by global regulatory frameworks.

7.2 Future Work

While this work establishes a solid foundation for privacy-preserving recommendation systems, several avenues for future research remain unexplored. The current experimental design makes specific simplifying assumptions that, while necessary for controlled evaluation, limit the generalizability of findings to real-world deployment scenarios. Addressing these limitations would strengthen both the theoretical understanding and practical applicability of federated recommendation systems.

IID Data Partitioning. The current implementation employs random user assignment for data partitioning, creating Independent and Identically Distributed (IID) data across federated clients. This approach does not reflect real-world deployment scenarios where user data exhibits systematic differences based on geography, demographics, device types, or usage patterns. Non-IID settings are known to exacerbate gradient divergence during federated aggregation, potentially increasing the privacy-accuracy trade-off beyond the measured 1.35% RMSE premium reported in this work. Future research should characterize performance trade-offs under varying degrees of data heterogeneity by implementing synthetic partitioning schemes such as Dirichlet distribution-based allocation, as well as realistic geographic and demographic clustering. Additionally, evaluating personalization-aware aggregation strategies like FedProx [22], which introduces proximal terms to handle client drift, would provide insights into mitigation techniques for non-IID challenges. Quantifying how the RMSE premium scales with data heterogeneity (measured through metrics such as Jensen-Shannon divergence or KL divergence between client distributions) would enable practitioners to make informed decisions about federation viability in diverse deployment contexts.

Absence of Differential Privacy. While the FedAvgItemsOnly strategy achieves data locality by retaining user embeddings on client devices, it lacks formal privacy guarantees against inference attacks or model inversion. Differential privacy mechanisms could provide mathematically rigorous privacy assurances but typically introduce accuracy degradation through noise injection. Future work should quantify the incremental cost of differential privacy

across a range of privacy budgets (ε values from 0.1 to 10, representing strong to weak privacy guarantees) under both local and central differential privacy models. This analysis should measure the combined privacy-accuracy-computational trade-off, evaluating how clipping thresholds and noise scales impact both RMSE and training convergence rates. Integration with secure aggregation protocols (such as homomorphic encryption or secure multi-party computation) would further strengthen privacy guarantees while potentially introducing computational overhead that must be characterized. Understanding these trade-offs is essential for practitioners navigating regulatory requirements that may mandate formal privacy guarantees beyond architectural isolation.

Matrix Factorization Baseline. The experimental evaluation employs matrix factorization as the baseline recommendation algorithm due to its mathematical transparency, interpretability, and well-understood convergence properties. However, contemporary production recommender systems increasingly adopt deep learning architectures that may exhibit different federated learning characteristics. Future research should extend the evaluation framework to encompass neural collaborative filtering models (such as Neural Matrix Factorization and Multi-Layer Perceptron architectures), graph neural networks that leverage item-item or user-item relationships, and transformer-based sequential recommendation models that capture temporal dynamics in user preferences. These architectures introduce additional complexity through deeper parameter spaces, non-convex optimization landscapes, and architectural choices (such as embedding dimensions, layer counts, and attention mechanisms) that may interact differently with federated aggregation. Comparative analysis across model families would establish whether the observed 1.35% RMSE premium generalizes across recommendation paradigms or represents a lower bound specific to linear models.

Controlled Experimental Setting. The current evaluation employs single-machine simulation with synchronous aggregation rounds, controlled network conditions, and homogeneous client capabilities. Real-world federated deployments face substantial additional challenges including network latency variability across geographic regions, heterogeneous device capabilities (ranging from mobile phones to cloud servers), client dropout due to connectivity issues or battery constraints, and asynchronous participation patterns. Future work should validate findings under realistic distributed conditions by deploying the FedRec Dashboard across heterogeneous devices with varying computational resources, implementing fault-tolerant aggregation strategies that handle client dropout gracefully, and evaluating performance under wide-area network latency profiles. Additionally, extending the evaluation beyond explicit ratings to implicit feedback signals (such as clicks, reading time, or purchase behavior), session-based recommendation scenarios with short-term context, and cold-start situations with limited user history would broaden applicability to diverse recommendation domains. Characterizing how system performance degrades under varying failure rates and latency distributions would inform deployment decisions for organizations considering federated architectures in production environments.

Addressing these limitations through systematic empirical research would advance federated recommendation from controlled academic evaluation toward robust production deployment. The open-source FedRec Dashboard provides the foundational infrastructure necessary for conducting these investigations, lowering barriers to entry for researchers seeking to extend this work. As privacy regulations continue to evolve globally and user awareness of data practices increases, developing federated recommendation systems that balance accuracy, privacy, computational efficiency, and real-world robustness remains a critical research imperative with significant societal implication.

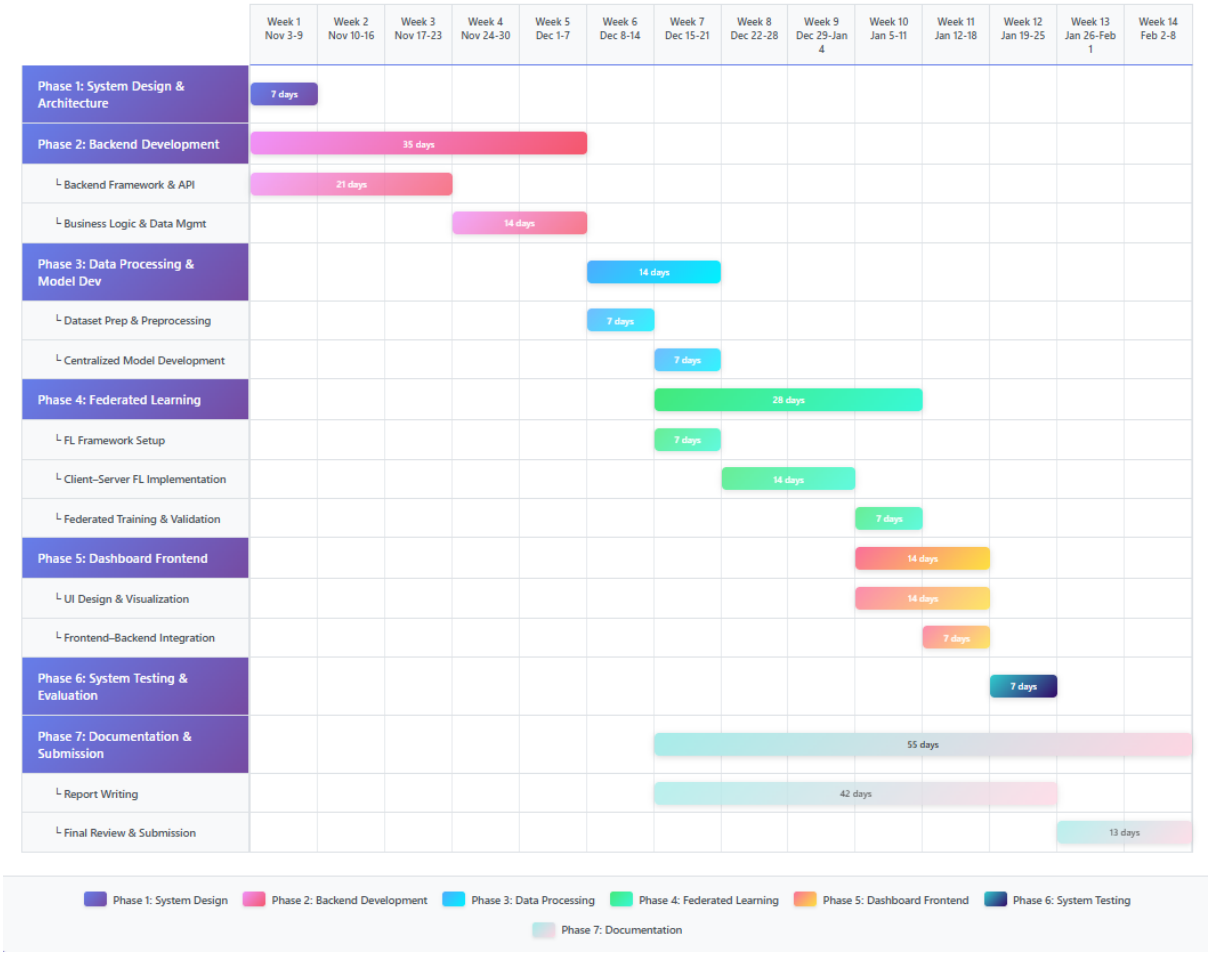
References

1. Asad, M., Shaukat, S., Javanmardi, E., Nakazato, J., & Tsukada, M. (2023). A comprehensive survey on privacy-preserving techniques in federated recommendation systems. *Applied Sciences*, *13*(10), 6201.
   https://doi.org/10.3390/app13106201
2. De Marzo, G., Gravino, P., & Loreto, V. (2024). Recommender systems may enhance the discovery of novelties. *Journal of Physics: Complexity*, *5*(4), 045008.
   https://doi.org/10.1088/2632-072X/ad9cdd
3. Ghugare, N. (2024). *Book Recommendation System Using Goodreads Dataset* [Project Category: Class-Defined, Physics 5680, Autumn 2024]. (Unpublished manuscript).
4. Hurkens, A. (2022). Worldlit as MMORPG? Wholesaling world literature in the age of Amazon. *Journal of World Literature*, *7*(2), 449-469.
   https://doi.org/10.1093/llc/fqab020
5. Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). *FedProx: Federated optimization of heterogeneous networks*. arXiv preprint arXiv:1812.06127.
6. McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (pp. 1273-1282). JMLR W&CP, 54.
7. Nisha, M. R., Harika, D., Pallavi, D., Reddy, C. R., Dhidse, S., & Lakshmi, C. (2024). Book recommendation system using AI. *International Journal of Progressive Research in Engineering Management and Science (IJPREMS)*, *4*(11), 1404-1406.
8. Rebora, S., Boot, P., Pianzola, F., Gasser, B., Herrmann, J. B., Kraxenberger, M., Kuijpers, M. M., Lauer, G., Lendvai, P., Messerli, T. C., & Sorrentino, P. (2021). Digital humanities and digital social reading. *Digital Scholarship in the Humanities*, *36*(Supplement 2), ii230–ii245.
   https://doi.org/10.1093/llc/fqab020
9. Saif, S., Al Nasim, M. A., Biswas, P., Rashid, A., Haque, M. A., & Bin Jahangir, M. Z. (2025). *Principles and Components of Federated Learning Architectures*. arXiv preprint arXiv:2502.05273v1.

   Wang, L., Zhou, H., Bao, Y., Yan, X., Shen, G., & Kong, X. (2024). Horizontal Federated Recommender System: A Survey. *ACM Computing Surveys*, *56*(9), Article 240, 1–42 https://doi.org/10.1145/3656165
10. Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” Computer, vol. 42, no. 8, pp. 30–37, Aug. 2009, doi: 10.1109/MC.2009.263.
11. T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: challenges, methods, and future directions,” IEEE Signal Process. Mag., vol. 37, no. 3, pp. 50–60, May 2020, doi: 10.1109/MSP.2020.2975749.
12. M. Wan and J. McAuley, “Item recommendation on monotonic behavior chains,” in Proc. 12th ACM Conf. Recommender Systems (RecSys ’18), 2018, pp. 86–94.
13. M. Wan, R. Misra, N. Nakashole, and J. McAuley, “Fine-grained spoiler detection from large-scale review corpora,” in Proc. 57th Annu. Meeting Assoc. Computational Linguistics (ACL ’19), 2019, pp. 2605–2610.

14. M. Wan, “Goodreads datasets,” 2025, GitHub repository. [Online]. Available: https://github.com/MengtingWan/goodreads
15. R. Li, *at al.*, “Federated cross-view e-commerce recommendation based on feature rescaling,” Scientific Reports, vol. 14, art. no. 29926, Dec. 2024, doi: 10.1038/s41598-024-81278-1.
16. P. Muellner, D. Kowald, and E. Lex, “Robustness of meta matrix factorization against strict privacy constraints,” in Advances in Information Retrieval (ECIR 2021), LNCS 12657, Springer, Cham, 2021, pp. 107–119, doi: 10.1007/978-3-030-72240-1_8.
17. Q. Yue and X. Tong, “A federated recommendation system with a dual-layer multi-head attention network and regularization strategy,” Entropy, vol. 27, no. 11, p. 1112, 2025, doi: 10.3390/e27111112.
18. H. Zhonglei, L. Fangyuan, W. Jun, H. Xiangnan, and L. Yidong, “LightFR: Lightweight federated recommendation with privacy-preserving matrix factorization,” ACM Trans. Inf. Syst., vol. 41, no. 4, art. no. 90, pp. 1–28, Mar. 2023, doi: 10.1145/3578361.
19. S. Li, D. Wu, Y. He, and X. Luo, “Federated projection Hamming hash latent factor learning model,” in 2025 28th Int. Conf. Computer Supported Cooperative Work in Design (CSCWD), IEEE, 2025, doi: 10.1109/CSCWD64889.2025.11033266.
20. P. Müllner, E. Lex, M. Schedl, and D. Kowald, “The impact of differential privacy on recommendation accuracy and popularity bias,” in Advances in Information Retrieval (ECIR 2024), N. Goharian et al., Eds., LNCS, vol. 14611. Cham, Switzerland: Springer, 2024, pp. 466–482, doi: 10.1007/978-3-031-56066-8_33.
21. P. Müllner, E. Lex, M. Schedl, and D. Kowald, “Differential privacy in collaborative filtering recommender systems: a review,” Frontiers in Big Data, vol. 6, Oct. 2023, art. no. 1249997, doi: 10.3389/fdata.2023.1249997.
22. A. Aramanda, S. Md. Abdul, and R. Vedala, “Refining user ratings using user emotions for recommender systems,” in Proc. 23rd Int. Conf. Information Integration and Web Intelligence (iiWAS 2021), 2021, pp. 3–10, doi: 10.1145/3487664.3487666.

Appendix A: Gantt Chart



Appendix B: Meeting Logs

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date:3 November 2025	**Meeting No.: 1**
Meeting Mode: Online	
Project ID: FYP02-DS-T2530-0022	**Project Type:** Application & Research Based.
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Touré Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Reflection on FYP 1 progress and preparation for application development.

Details (in point form):

- Reflected on the progress made during FYP 1, including the research, project design, and proposed system architecture.
- Discussed the overall design and implementation plan of the dashboard for evaluating federated learning in book recommender systems.
- Reviewed the key components required for the system, including the federated learning framework, recommender system module, and dashboard interface.
- Received instructions from the supervisor to begin preparing the scaffolding for the application development phase.

2. WORK TO BE DONE

Tasks: Begin application scaffolding and review implementation plan.

Details (in point form):

- Review the system design and implementation plan to ensure a clear understanding before development begins.
- Set up the initial project structure and scaffolding for the dashboard application. Identify the tools, frameworks, and technologies required for the system implementation.
- Begin preparing the development environment for federated learning and dashboard integration.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS *[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]*
4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

Sharaf

Supervisor's Signature

Student's Signature

……..

…….

Co-Supervisor's Signature
(if applicable)

Company Supervisor's Signature
(if applicable)

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date: 10 November 2025	**Meeting No.:** 2
Meeting Mode: Physical	
Project ID: FYP02-DS-T2530-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science (Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Application scaffolding completion and presentation of system structure.

Details (in point form):

- Completed the scaffolding of the dashboard application based on the approved system design and implementation plan.
- Implemented the core layer to handle the main business logic of the application.
- Developed the service layer to manage communication between the core logic and other system components.
- Created the initial SQLAlchemy models to define the database structure and represent key entities required for the system.
- Presented the scaffolding, core layer, service layer, and database models to the supervisor for review and feedback.

2. WORK TO BE DONE

Tasks: Develop API layer and FastAPI endpoints

Details (in point form):

- Research different federated learning algorithms, focusing on their principles, advantages, and disadvantages.
- Design and implement the API layer to enable communication between the frontend and backend components.
- Develop FastAPI endpoints for handling system operations and data interactions.
- Integrate the API endpoints with the service and core layers.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

- **Problem:** Minor challenges in structuring the project layers to ensure proper separation of concerns and maintainability.
- **Solution:** Reviewed best practices for backend architecture and ensured clear separation between core, service, and database layers to maintain a clean and scalable project structure.

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

- The supervisor reviewed the scaffolding and system structure and was satisfied with the progress. The supervisor advised proceeding with the development of the FastAPI endpoints and completing the API layer to enable system functionality.

Sharaf

Supervisor's Signature

Student's Signature

……..

…….

Co-Supervisor's Signature
(if applicable)

Company Supervisor's Signature
(if applicable)

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date: 24 November 2025	**Meeting No.:** 3
Meeting Mode: Face-to-Face	
Project ID: FYP02-DS-T2530-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company:	**Company Supervisor Name:**

(if applicable)	(if applicable)

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Completion of API layer and FastAPI endpoints.

Details (in point form):

- Successfully implemented the API layer to handle communication between the frontend and backend.
- Developed FastAPI endpoints to support key system operations and data access.
- Integrated the API endpoints with the service layer and core layer to ensure proper data flow and functionality.Progress was made in understanding the state-of-the-art in both federated systems and recommender systems.

2. WORK TO BE DONE

Tasks: Implement business logic and model data handling.

Details (in point form):

- Complete the implementation of the business logic required for the recommender system and dashboard functionality.
- Improve the service layer to handle system operations efficiently.
- Implement proper data handling and validation using the SQLAlchemy models.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

Sharaf

Supervisor's Signature

Student's Signature

……..

…….

Co-Supervisor's Signature
(if applicable)

Company Supervisor's Signature
(if applicable)

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date:15 December 2025	**Meeting No.: 4**
Meeting Mode: Face-to-Face	
Project ID: FYP02-DS-T2530-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen sami Mohammed Al-Hoarni**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Implementation of data handling and centralized model training.

Details (in point form):

- The literature review has been successfully completed, providing a comprehensive understanding of the project's domain.
- Completed the implementation of the data handler to manage data processing and preparation for model training.
- Ensured proper data loading, transformation, and storage using the defined SQLAlchemy models and service layer.
- Implemented the centralized training approach for the book recommender system.
- Successfully trained the centralized model and obtained initial training results.
- Verified that the training process and results were correctly handled and stored within the system.

2. WORK TO BE DONE

Tasks: Implement fedeated learning

Details (in point form):

- Design the structure for client-server communication in federated learning.
- Implement local model training at the client level.
- Implement aggregation of model updates at the server level.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS *[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]* • **Problem:** Initial challenges in structuring the data handler to properly support the model training process. • **Solution:** Reviewed data flow and ensured proper data preprocessing, organization, and handling to support efficient and accurate model training.
4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

Sharaf

Supervisor's Signature

Student's Signature

……..

…….

Co-Supervisor's Signature
(if applicable)

Company Supervisor's Signature
(if applicable)

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date: 5 January 2026	**Meeting No.:** 5

Meeting Mode: Face-to-Face	
Project ID: FYP02-DS-T2530-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: Sharaf Eldeen Sami Mohammed Al-Horani	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Federated Learning Implementation

Details (in point form):

- Successfully implemented the federated learning framework for the book recommender system.
- Developed the client-side logic to perform local model training on distributed datasets.
- Implemented the server-side aggregation mechanism to combine client model updates.

2. WORK TO BE DONE

Tasks: Develop frontend interface and conduct sytem testing

Details (in point form):

- Begin development of the frontend dashboard interface for visualizing system results.
- Design and implement user interface components to display model performance and evaluation metrics.
- Integrate the frontend with the FastAPI backend endpoints.
- Conduct system testing to ensure proper functionality and integration of all components.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

Sharaf
Supervisor's Signature

Student's Signature

……..
……
Co-Supervisor's Signature
(if applicable)

Company Supervisor's Signature
(if applicable)

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date: 19 January 2025	**Meeting No.:** 6
Meeting Mode: Face-to-Face	
Project ID: FYP01-DS-T2530-0022	**Project Type:** Application & Research based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	

Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Frontend development completion and system testing.

Details (in point form):

- Successfully completed the development of the frontend dashboard interface.
- Implemented user interface components to display model training results and evaluation metrics.
- Integrated the frontend with the FastAPI backend through the developed API endpoints.
- Ensured proper communication between the frontend and backend components.
- Conducted system testing to verify the functionality and integration of all system components, including centralized and federated learning features.

2. WORK TO BE DONE

Tasks: Final system review and preparation for project submission.

Details (in point form):

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

Sharaf

Supervisor's Signature

……

Student's Signature

……..

……

Co-Supervisor's Signature
(if applicable)

Company Supervisor's Signature
(if applicable)

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date:24 March 2025	**Meeting No.: 1**
Meeting Mode: Online	
Project ID: FYP01-DS-T2510-0022	**Project Type:** Application & Research Based.
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Touré Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Problem Statement Formulation.

Details (in point form):

- Initiated discussion about the problem statement to define the core objective of the FYP.
- Discussed the potential of federated Learning and how it can be integrated in recsys
- Developed an initial outline for Chapter 1, which will serve as the introduction to the project.

2. WORK TO BE DONE

Tasks: Specify the project scope and draft the objectives of the project.

Details (in point form):

- Research different federated learning algorithms and have an idea of how to integrate that to our project
- Finalise the problem statement and get a clear idea of what this project is aiming to solve

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

- **Problem:** Not knowing the precise problem statement and the exact scope of the project. This made it challenging to define clear objectives and a focused direction for the research.
- **Solution:** Undertake thorough research to clearly define the problem statement and establish a well-defined project scope. This will involve identifying a specific area within federated learning for recommender systems that can be explored in depth within the given timeframe, ensuring the project's feasibility and focus.

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

- The supervisor was supportive and optimistic of how the project will go, noting that it is very promising.

…… ……...

Supervisor's Signature Student's Signature

…….. ……

Co-Supervisor's Signature **(if applicable)**

Company Supervisor's Signature **(if applicable)**

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date: 3 April 2025	**Meeting No.:** 2
Meeting Mode: Online	
Project ID: FYP01-DS-T2510-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science (Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company:	**Company Supervisor Name:**

(if applicable)	(if applicable)

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Finalize Problem Statement, Scope, and Objectives; Clarify path to Literature Review.

Details (in point form):

- The problem statement was finalized, clearly defining the core challenge the project aims to address in federated learning for book recommender systems.
- The project scope was completed, outlining the specific areas of focus and the boundaries of the research.
- The project objectives were clearly defined, providing a roadmap for what the project aims to achieve.
- The path forward for the literature review chapter was clarified, establishing a clear direction for the research and content to be covered.

2. WORK TO BE DONE

Tasks: Conduct Literature Review (Chapter 2).

Details (in point form):

- Research different federated learning algorithms, focusing on their principles, advantages, and disadvantages.
- Explore recommender systems, including their fundamental concepts, different types, and common approaches.
- Investigate the challenges within recommender systems, particularly concerning privacy and scalability in centralized approaches.
- Review the state-of-the-art in applying federated learning techniques specifically to book recommendation systems.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS *[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]* • **Problem:** Formatting issues within the report and the omission of a table of contents. • **Solution:** Ensure that a table of contents is added to the report before the next meeting to improve organization and navigation.
4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor) • The supervisor introduced the distinction between distributed learning and federated learning, which significantly increased enthusiasm for the literature review and further knowledge acquisition. • The supervisor expressed satisfaction with Chapter 1 and provided instructions to add a table of contents before the next meeting.

…… ……...

Supervisor's Signature Student's Signature

…….. ……

Co-Supervisor's Signature **(if applicable)** Company Supervisor's Signature **(if applicable)**

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date:14 April 2025	**Meeting No.:** 3
Meeting Mode: Face-to-Face	
Project ID: FYP01-DS-T2510-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Continue Literature Review.

Details (in point form):

- A significant portion of the literature review was completed, leading to a better understanding of recommender systems (RecSys) types and common approaches.
- The distinction between distributed learning and federated learning has been clearly understood, providing a solid foundation for the project's federated learning focus.
- Progress was made in understanding the state-of-the-art in both federated systems and recommender systems.

2. WORK TO BE DONE

Tasks: Finalize Literature Review; Research State of the Art in Federated and RecSys; Understand VFedRec and HFedRec differences.

Details (in point form):

- Finalize the literature review, ensuring comprehensive coverage of recommender system fundamentals and federated learning principles.
- Conduct in-depth research on the state-of-the-art in both federated learning applications and recommender systems, identifying key trends and existing solutions.
- Clearly articulate the differences between Vertical Federated RecSys (VFedRec) and Horizontal Federated RecSys (HFedRec) to establish the most relevant paradigm for this project.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

- **Problem:** A perceived lack of research papers specifically combining recommender systems with the Goodreads dataset.
- **Solution:** Recognize that this project itself addresses this gap by focusing on the Goodreads dataset for evaluating federated learning in book recommender systems, thereby contributing novel research.

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

- Regarding the Goodreads dataset, the supervisor emphasized that the project scope should prioritize the use of FedAvg as the primary federated learning algorithm, despite exploring other algorithms in the literature review. This is to adhere to the specific focus outlined in Chapter 1.
- The supervisor was very insightful and suggested a lot of efficient strategies to enhance the literature review and the research part.

…… ……...

Supervisor's Signature Student's Signature

…….. …….

Co-Supervisor's Signature **(if applicable)** Company Supervisor's Signature **(if applicable)**

FACULTY OF COMPUTING & INFORMATICS

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date:14 April 2025	**Meeting No.: 4**
Meeting Mode: Face-to-Face	
Project ID: FYP01-DS-T2510-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen sami Mohammed Al-Hoarni**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Complete Literature Review; Understand the significance of the Theoretical Framework; Set up the Tech Stack for the Theoretical Framework;.

Details (in point form):

- The literature review has been successfully completed, providing a comprehensive understanding of the project's domain.
- The significance of the theoretical framework has been recognized, understanding its crucial role in underpinning the project's methodology and analysis.

2. WORK TO BE DONE

Tasks: Set up the Tech Stack for the Theoretical Framework; Implement both Centralized and Decentralized Theoretical Frameworks.

Details (in point form):

- Implement both centralized and decentralized approaches on a sample of the dataset.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

- **Problem:** Difficulty in deciding on the most optimal and relevant technology stack for the theoretical framework.
- **Solution:** Consulted with the supervisor, who provided guidance and recommendations on the most suitable tools and frameworks to use, ensuring alignment with project goals and technical requirements.

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

- The supervisor commended the thoroughness of the literature review.
- The supervisor highlighted that the theoretical framework is likely the most critical chapter and holds significant importance for the project. He advised to pay close attention to its development and to enjoy the process of building this foundational element.

…… ……...

Supervisor's Signature Student's Signature

…….. …….

Co-Supervisor's Signature **(if applicable)** Company Supervisor's Signature **(if applicable)**

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date: 14 May 2025	**Meeting No.:** 5

Meeting Mode: Face-to-Face	
Project ID: FYP01-DS-T2510-0022	**Project Type:** Application & Research Based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: Sharaf Eldeen Sami Mohammed Al-Horani	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Complete Chapters 3 & 4; Run Centralized and Decentralized Models; Elicit System Requirements and Use Cases and flow diagrams.

Details (in point form):

- Chapter 3, the Theoretical Framework, has been completed, including the implementation of both centralized and decentralized models.
- Both the centralized and decentralized models outlined in the theoretical framework have been successfully run.
- Chapter 4, focusing on Requirements Elicitation and Analysis, has been finished, detailing the system's requirements and use cases along with sequence diagrams, activity diagrams, state diagrams etc..

2. WORK TO BE DONE

Tasks: Design Chapter (Chapter 5); Develop Implementation Plan (Chapter 7); Create a Prototype for the System (Chapter 6).

Details (in point form):

- Begin the Design chapter (Chapter 5), focusing on the architectural choices, interfaces, and data flow for the dashboard.
- Create a prototype for the system (Chapter 6), visualizing the user interface and key workflows of the dashboard.
- Develop the Implementation Plan (Chapter 7), outlining the project's phases, timeline, and key milestones.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

- **Problem:** Difficulty in deciding on the specific use cases for the system.
- **Solution:** Revisited the insights and findings from the theoretical framework (Chapter 3) to inform and finalize the selection of use cases, ensuring they align with the project's objectives and technical implementations.

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

- The supervisor noted that after completing the theoretical framework and requirements, the next crucial steps are the Design chapter and the Implementation Plan.

- The supervisor advised to include an Entity-Relationship Diagram (ERD) for the entities identified in the system.
- The supervisor provided guidance on the subsequent chapters, indicating the progression of the project.

…… ……...

Supervisor's Signature Student's Signature

…….. ……

Co-Supervisor's Signature **(if applicable)** Company Supervisor's Signature **(if applicable)**

CPT6314 Final Year Project (FYP) 1 Meeting Log
Trimester MAR 2025 (Trimester ID:2510)

Meeting Date:9 June 2025	**Meeting No.:** 6

Meeting Mode: Face-to-Face	
Project ID: FYP01-DS-T2510-0022	**Project Type:** Application & Research based
Project Title : A Dashboard for Evaluating Federated Learning in Book Recommender Systems	
Student ID : 1211104426	**Student Name:** Soultan Toure Salifou
Student Programme and Specialisation: Bachelor of computer science(Data Science)	
Supervisor Name: **Sharaf Eldeen Sami Mohammed Al-Horani**	**Co-Supervisor Name:** **(if applicable)**
Collaborating Company: **(if applicable)**	**Company Supervisor Name:** **(if applicable)**

1. WORK DONE

[Please write the details of the work done, after the last meeting]

Tasks: Completed chapters 5,6 and 7

Details (in point form):

- Chapter 5, the Design chapter, has been completed, detailing the system's architecture, class diagrams, and deployment model.
- Chapter 6, the Prototype, has been finished, showcasing the user interface and key views of the dashboard.
- Chapter 7, the Implementation Plan, has been developed, outlining the project's phases, timeline, and key tasks.

2. WORK TO BE DONE

Tasks: Modify Design Model based on Supervisor's Suggestions; Add Conclusion Chapter (Chapter 8); Finalize Reporting..

Details (in point form):

- Revise the Design Model as per the supervisor's feedback to ensure it accurately reflects the project's requirements and architectural decisions.
- Write the Conclusion chapter (Chapter 8), summarizing the project's findings, contributions, and future directions.
- Finalize the overall report formatting, ensuring consistency, clarity, and adherence to academic standards.

3. PROBLEMS ENCOUNTERED AND SOLUTIONS

[Please write the details of the problems encountered, after the last meeting and provide the solutions / plan for the solutions]

- **Problem:** Difficulty in dividing the implementation plan into distinct, manageable phases with clear dependencies.
- **Solution:** The implementation plan will be structured into six logical, overlapping phases, starting with backend foundations and progressing through core logic, federated learning, frontend development, integration, and finally deployment. This phased approach ensures a progressive and manageable workflow.

4. COMMENTS (Supervisor / Co-Supervisor / Company Supervisor)

- The supervisor instructed to add the Conclusion chapter to the report.
- The supervisor emphasized the need to finalize the overall report formatting for a polished and professional presentation.

…… ……...

Supervisor's Signature Student's Signature

…….. …….

Co-Supervisor's Signature **(if applicable)**

Company Supervisor's Signature **(if applicable)**

Appendix C: Turnitin Similarity Index Page

1211104426_Sharaf_FYP2_Report.pdf

ORIGINALITY REPORT

10%	5%	5%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Multimedia University Student Paper	3%
2	Submitted to Monash University Student Paper	<1%
3	Soo Dong Kim, Mira Kim. "Hands-on Software Architecture", Springer Science and Business Media LLC, 2025 Publication	<1%
4	www.mdpi.com Internet Source	<1%
5	Sonali Dhananjay Patil, Rajesh Ingle, Amar Buchade, Vidy Potdar. "Decentralized Healing - Transforming Healthcare with Federated Learning and Blockchain Technologies", CRC Press, 2025 Publication	<1%
6	Submitted to University College London Student Paper	<1%
7	Submitted to HELP UNIVERSITY Student Paper	<1%
8	Submitted to University of Strathclyde Student Paper	<1%